

Search, Not Capacity: Evolving Ternary Protoneuron Populations for Balance and 3D Locomotion

A. Sohlen

Implementation, measurement and analysis assistance: Claude (Anthropic). All numbers reported here were produced by the code in the appendix; the corrections in Section 8 are part of the record.

Abstract. We study a deliberately undifferentiated recurrent population in which every unit is identical in form: exactly three incoming synapses, synaptic weights drawn from $\{-1, 0, +1\}$, and transmitted activations also in $\{-1, 0, +1\}$. The only continuous parameters are three per-unit quantities that set a firing threshold and an input gain. Graded motor output arises solely from a population vote at the actuators. Networks are evolved, not gradient-trained, and we evaluate them on three plants of increasing temporal structure: a triple inverted pendulum, a 3D biped with impulsive foot collisions, and a 3D quadruped with two-segment legs and penalty contact. Four results stand out. (i) Cost to solve grows as roughly the square of population size: eight units solve the pendulum about 130 times more cheaply than 128, and 512 units fail on 30 of 30 seeds despite trivially containing the eight-unit solution, which isolates search rather than capacity as the binding constraint. (ii) Robustness, out-of-sample generalisation and behavioural diversity all *decline* monotonically with population size, and all three follow from a single measurable quantity: the torque displaced by one mutated vote. (iii) Ablation shows that where the computation resides is a property of the task, not the architecture: on the balancing task the solution lives almost entirely in the per-unit thresholds and cutting every recurrent connection costs 15 percent, whereas on the quadruped the same cut removes 95 percent and the animal cannot stand. (iv) A ternary population of eight units matches a hand-tuned continuous controller on unseen cases, and evolved controllers outperformed our hand-tuned baselines on both legged plants. We also report the exploits, mis-specifications and retracted claims encountered along the way, since several would have silently invalidated the results.

1. Introduction

The question motivating this work is whether the smallest plausible computational unit, one that can neither hold state nor emit a graded value, can be composed into something that controls an unstable body. The unit we use is a caricature of a neuron: three synapses, each either excitatory, inhibitory or silent, and an output restricted to three symbols. Nothing crosses a synapse except one of those symbols. Every continuous quantity in the system lives inside a cell body or at the actuator.

This is a restrictive architecture by construction, and the interesting questions are what it cannot do, what it must reorganise into in order to do harder things, and how the difficulty of *finding* a solution scales with the size of the population. The last question turned out to dominate everything else we measured.

Our contributions are empirical. We give a scaling law for cost to solve as a function of population size, with a control that separates search difficulty from representational capacity. We show that four apparently independent quality measures move together and trace them to one mechanism. We use ablation across three plants to show that the locus of computation migrates from per-unit thresholds to recurrent loops exactly when the task begins to require phase. And we document the measurement failures candidly, because in a study of this kind most of the risk is in the instrumentation rather than the hypothesis.

1.1 Related work

The arithmetic of our unit is that of a ternary neural network. Ternary weight networks constrain weights to $\{-1, 0, +1\}$ against a threshold chosen to minimise distance to the original real-valued weights, and fully ternary networks quantise activations as well, using a two-threshold activation that maps an integer pre-activation to one of three levels. That activation is exactly what our argmax reduces to, derived independently (Section 2.1). GXNOR-style hardware exploits the zero state as a gate, so that a synapse with either a zero weight or a zero input costs no energy; 35 percent of the synapses in our evolved networks are in that state. The important difference is that ternary-network training keeps full-precision shadow weights and quantises on the forward pass, whereas our ternary values *are* the parameters and no

shadow exists.

The topology is a Kauffman random Boolean network: N nodes, each reading K fixed random inputs, updated synchronously, with state held only in the recurrence. Such networks are ordered at low connectivity and chaotic at high connectivity, with $K = 2$ the classical critical point for uniformly random Boolean functions. Our architecture declares $K = 3$, but the evolved networks silence a third of their synapses and arrive at a mean effective fan-in of 1.94. We flag the coincidence and the reasons it is not yet evidence in Section 9.

The benchmark lineage is neuroevolution on pole balancing, where cooperative synapse coevolution set the standard on hard variants with continuous state and hidden velocity information; those methods use continuous weights, which is the contrast that makes a ternary result interesting. Our diversity archive is MAP-Elites, with a genome-distance filter closer to novelty search with local competition. The closest architectural relative is weight-agnostic neural networks, which populate every connection with a single shared weight and search only over topology, motivated by precocial species whose competence appears to be structural rather than learned. Our legged plants follow the passive-dynamic-walking tradition, in which a limit cycle is sustained by small, well-timed energy injections rather than precise force regulation; that is a favourable regime for bang-bang ternary output.

We did not find prior work combining all of these: ternary weights *and* activations, fixed small fan-in, recurrent, evolved rather than gradient-trained, and driving an unstable plant through a population vote. A one-line framing is neuroevolution of a ternary-state Kauffman network as a controller.

2. The protoneuron model

Each unit i has three incoming slots. Slot k reads a source $s(i,k)$, which is either one of the ternary sensory features or the previous-step activation of another unit, and multiplies it by a weight $w(i,k)$ in $\{-1, 0, +1\}$. The pre-activation is

$$z_i = \text{bias}_i + \sum_k w(i,k) * \text{pre}(s(i,k)), \quad \text{pre} \in \{-1, 0, +1\}$$

Because both factors are ternary, a single synapse contributes at most plus or minus one to the sum. A weight is therefore not a strength but a sign convention: it decides whether an input counts for or against. The reachable ranges of every quantity are given in Table 1.

Quantity	Range	Step	Reachable values	Set by
Presynaptic value	-1, 0, +1	n/a	3	upstream unit or sensor
Synaptic weight	-1, 0, +1	n/a	3	mutation, uniform over the two it is not
One contribution	-1, 0, +1	n/a	3	consequence of the two above
Sum of 3 contributions	-3 .. +3	1	7	fan-in
bias	-2.5 .. +2.5	0.25	21	mutation, additive
fireBias, 3 channels	-5 .. +5	0.3	34 per channel	mutation, additive
fireGain	0.08 .. 3.5	x0.82 or x1.22	353 within 14 steps	mutation, multiplicative
Unit output h	-1, 0, +1	n/a	3	argmax of three affine pieces
Decoder weight	-1, 0, +1	n/a	3	mutation
Actuator vote sum	-fan-in .. +fan-in	1	57 / 79 / 73 measured	population tally
Torque	-22 .. +22 N.m	tanh-spaced	as the vote sum	$\tanh(\text{sum} / 0.55 f) * 22$

Table 1: Every quantity in the architecture. Everything that crosses a synapse is one of three symbols; everything continuous is inside a cell body. A unit compresses 45 reachable internal states of z into 3 outputs on every step. The only expansion in the system is the final tally.

2.1 The activation is an argmax of three affine pieces, and the softmax is vestigial

The output is the index of the largest of three quantities: a constant middle channel and two channels affine in z with slopes plus and minus gain/ T . Writing $U = (b_1 - b_2) T / \text{gain}$ and $L = (b_0 - b_1) T / \text{gain}$, the unit emits +1 above U , -1 below L , and 0 in between. With default thresholds both cut points sit at zero and the unit is exactly $\text{sign}(z)$. This is the standard two-threshold ternary activation, reached from a different direction.

Our original implementation computed a softmax over the three channels and then took the argmax. Since softmax is monotonic, the exponentials cannot change the answer. We verified this on 400,000 random draws of (z, fireBias, gain): **zero disagreements** against two plain comparisons, which ran **4.56 times faster** in isolation. At 128 units, 400 steps per episode and roughly 100 episodes per generation, the discarded work was about five million exponentials per generation. The softmax was left over from an abandoned gradient-based variant that genuinely sampled from those probabilities.

A practical consequence of the threshold geometry: with gain 0.9 and $T = 0.22$, fireBias must exceed 4.09 before the dead zone can suppress a vote sum of magnitude one, and the clamp is at 5. Across its entire legal range a unit therefore has only two achievable behaviours, "any nonzero sum fires" and "needs a margin of two", and only the top 18 percent of the range buys the second. Raising T widens that repertoire.

2.2 The population vote is the only place a magnitude survives

Each actuator j sums the votes of the units it reads and squashes the result: $\text{tau}_j = \tanh(\text{sum} / (0.55 f_j)) * \text{TAU_MAX}$, where f_j is the number of units with a nonzero decoder weight. In a measured 128-unit network the three actuators had fan-in 28, 39 and 36, giving 57, 79 and 73 distinguishable torque levels against the 3 available to any single unit. The graded control the plant requires exists only in the tally.

This has a consequence that runs through the whole paper. The torque displaced by flipping one vote is approximately $\text{TAU_MAX} / (0.55 f)$, so it falls as fan-in rises. Since evolution adds decoder connections and does so more in larger populations, the behavioural step size of a single mutation shrinks as the population grows. Section 4.3 shows this accounts for four separate trends at once.

2.3 A quantal reading

The analogy to quantal synaptic release is closer than the vocabulary suggests. A real bouton releases whole vesicles, so postsynaptic effect arrives in integer multiples of a quantum; our synapse delivers one quantum or none, with a sign. Quantal content is $m = n p$, with n the number of release sites and p the release probability. Here $n = 3$ by construction, and p is the fraction of slots carrying a nonzero weight. A measured 128-unit network had 136 of 384 slots silent, giving $p = 0.65$ and mean quantal content 1.94 of 3.

The analogy breaks in an informative place. A biological neuron integrates over time and space: the membrane has a time constant, quanta arriving in quick succession sum, and distal inputs are attenuated by the dendrite. Ours has none of that. Every input arrives simultaneously with identical weighting and the sum is discarded after one step. There is no leak, no decay and no cable. State can therefore exist only in the topology, which is why the ablation results in Section 7 come out as they do.

3. Plants

3.1 Triple inverted pendulum

Three point masses on massless rods, each joint angle measured from upright, with a torque at each joint. The mass matrix is $M(i,j) = m l^2 (3 - \max(i,j)) \cos(q_i - q_j)$ and the gravity term is $-m g l (3 - i) \sin(q_i)$; we integrate with RK4 at $dt = 0.025$ s. Failure is any joint angle beyond a configurable lane (default 77 degrees), any rate beyond 10 rad/s, or the episode cap of 10 s. Every evaluation applies scheduled velocity perturbations, so a solve is perturbation-tolerant by construction.

Sensing is 13 ternary features: three joint angles, three rates, three angle differences, three rate differences and a constant. Thresholds scale with the lane, so widening the lane does not silently change what the controller can see.

3.2 3D biped

A point-mass torso on rigid massless legs with point feet. During single support the mass is constrained to a sphere of radius L about the stance foot, solved with a Lagrange multiplier, so the model is a true spherical inverted pendulum with no small-angle approximation and no singularity at the vertical. A step places the swing foot at the radius the current CoM height allows and then destroys the velocity component along the new leg. That impulsive collision is the only energy sink and ankle torque the only source, so a steady gait must be a genuine limit cycle. Minimum swing time is 0.30 s, so the walker must actually balance on one leg. An impulse above 1.25 m/s ends the episode as an injury, and every landing is charged against reward.

3.3 3D quadruped

A 3 kg rigid torso with full six-degree-of-freedom dynamics; orientation is carried as a rotation matrix integrated by Rodrigues and re-orthonormalised each step, avoiding Euler singularities. Four massless two-segment legs provide hip swing, hip abduction and a knee. Joint servos are rate-limited to 5 rad/s so feet cannot teleport. Feet contact the ground through a normal spring-damper plus a stick-slip Coulomb friction anchor that drags when the tangential demand exceeds μN . Ground reaction is the only thing that can propel the animal, so no leg can pull the body through the air. A single foot above six times body weight counts as an injury. The controller drives twelve ternary actuators from eighteen ternary sensors and receives **no clock**: a quadruped gait is a four-phase rhythm, so any rhythm must come from the recurrence.

4. Experiment 1: population size on the triple pendulum

4.1 Protocol

We define **compute to half solved** as the amount of compute at which the median seed first crosses a criterion. It is the right statistic for this data because it is identifiable under right-censoring: provided more than half the seeds solve, the median is exact even though the slowest seeds never finish. Means are not identifiable, and means over solvers only are survivor-biased. Concretely it is the $\text{ceil}(n/2)$ -th smallest crossing with censored seeds entered as infinity, and we report a 90 percent bootstrap interval over seeds.

No compute currency is neutral across population size: fixed generations favour large populations, wall clock favours small ones, and episode counts ignore that a step costs $O(N)$. We rank on **neuron-updates**, the product of simulated physics steps and population size, and log generations, episodes and wall clock alongside. Our solve criterion is the ceiling of the robust score, which requires every one of 48 randomised validation cases to be held to the episode cap. Champions are additionally scored on a 48-case holdout suite built from a different seed stream.

4.2 Cost grows as the square of population size, and 512 units fail outright

N	seeds	solve rate	half-solved	90% interval	total spent	holdout
8	30	100%	0.01e9	0.01 .. 0.02e9	0.51e9	99%
12	30	100%	0.02e9	0.02 .. 0.03e9	1.18e9	99%
16	30	100%	0.05e9	0.03 .. 0.05e9	1.72e9	98%
24	30	100%	0.08e9	0.06 .. 0.08e9	2.90e9	98%
32	30	100%	0.10e9	0.08 .. 0.11e9	3.90e9	98%
48	30	100%	0.30e9	0.27 .. 0.34e9	10.42e9	99%
64	30	100%	0.59e9	0.49 .. 0.73e9	18.99e9	97%
128	11	100%	1.30e9	0.83 .. 1.46e9	13.50e9	97%
128*	30	100%	1.35e9	1.17 .. 1.53e9	45.13e9	97%
256*	30	80%	6.67e9	6.17 .. 7.89e9	207.81e9	96%
512*	30	0%	not reached	not reached	300.18e9	n/a

Table 2: Compute to half solved on the triple pendulum, in neuron-updates. Rows marked with an asterisk come from an earlier 300-trial sweep that extended to 512 units; the unmarked rows are a later 221-trial sweep. The two runs agree at $N = 128$ (1.30e9 with 11 seeds against 1.35e9

with 30). Total spent includes censored trials, so an arm that never solves still shows its full price.

Cost to half solved scales as N to the power 1.76 over the full range, and the exponent is stable across independent spans: 1.76 from 8 to 128 and 1.78 from 16 to 64. Since the cost of one generation is only $O(N)$, the number of generations required scales as roughly N to the 0.76, so larger populations need proportionally more generations rather than merely more expensive ones. Eight units are about 130 times cheaper than 128 and 667 times cheaper than 256. No floor was found: the smallest size we could test was limited by an implementation clamp, not by the task.

The decisive row is $N = 512$, which solved **0 of 30 seeds**. A 512-unit network can express everything an 8-unit network can, by silencing the other 504 units, so this cannot be a capacity limit. The rung ladder shows it stalling rather than failing at once: it reached a robust score of 9 at 8.99e9 neuron-updates and held half the validation cases, then never reached the higher rungs. This is as direct a demonstration as we can construct that **search, not representational capacity, is the binding constraint**. It is also expensive: $N = 256$ and $N = 512$ together consumed 86 percent of the 592e9 neuron-updates in that sweep and produced 24 solves out of 60, while the eight arms from 8 to 128 consumed 13 percent and produced 240 out of 240.

4.3 Robustness, generalisation and diversity all decline with population size

The perturbation ladder scores each arm champion on the holdout suite at perturbation multipliers from one to four. Every evaluation already perturbs at multiplier one, so a solve is perturbation-tolerant by construction; the ladder asks how much harder the plant can be kicked before recovery fails.

N	x1	x1.5	x2	x2.5	x3	x4	holdout	archive cells	voters	N.m per vote
8	100%	92%	65%	27%	13%	2%	99%	26.8	6.5	6.15
12	100%	90%	56%	25%	8%	2%	99%	28.0	9.8	4.08
16	100%	88%	52%	25%	6%	0%	98%	26.3	12.8	3.12
24	98%	83%	48%	19%	4%	0%	98%	20.7	15.6	2.56
32	98%	77%	31%	13%	2%	0%	98%	19.5	17.3	2.31
48	100%	85%	40%	15%	4%	0%	99%	19.3	19.6	2.04
64	98%	75%	38%	13%	4%	0%	97%	19.7	21.9	1.83
128	98%	69%	29%	8%	2%	0%	97%	14.9	31.2	1.28

Table 3: Median holdout survival under stronger perturbations, with out-of-sample generalisation, behavioural diversity and torque resolution alongside. Thirty seeds per arm except $N = 128$ (eleven). Spearman rank correlation against population size: survival at x1.5 and at x2 both -0.93, archive cells -0.91, holdout -0.50, and torque per vote -1.00.

Smaller populations are not merely cheaper to find; they are tougher, they generalise slightly better, and they discover more distinct behaviours. Survival under a double-strength kick falls monotonically from 65 percent at eight units to 29 percent at 128, with rank correlation -0.93. Archive coverage falls from 26.8 cells to 14.9 with correlation -0.91.

One mechanism accounts for all four trends. Decoder fan-in was targeted at 15 votes per actuator at initialisation, but evolution adds decoder connections and adds more of them in larger populations: measured fan-in runs 6.5, 9.8, 12.8, 15.6, 17.3, 19.6, 21.9 and 31.2 across the arms, reaching 60.1 at 256 units and 105.1 at 512. Because the tanh divisor is 0.55 f, the torque displaced by one mutated vote falls from 6.15 N.m at eight units to 1.28 at 128 and 0.38 at 512, a rank correlation of -1.00 by construction. A mutation in a large population is therefore both less visible to the plant and drawn from a larger genome. That single quantity predicts, with no further assumption, more generations to solve, less behaviour space covered per step, less decisive recovery from a perturbation, and more parameters available to fit the selection suite.

We stress that this makes our nominal fan-in control invalid: it fixed fan-in only at birth. The experiment that would settle the mechanism is to forbid mutations that add decoder connections once an actuator reaches its target fan-in, then re-run the ladder. If the robustness and coverage trends flatten, torque resolution was the cause; if they survive, something about small populations themselves is responsible.

4.4 Anatomy of an eight-unit solution

An eight-unit champion solved in 126 generations and 13 seconds on one core. Ablating it is informative. Resetting bias, fireBias and fireGain to defaults while leaving every wire and weight untouched collapses the robust score from 13.00 to **2.49**. Cutting every recurrent connection instead leaves 11.06 and 96 percent survival. The solution is therefore almost feedforward, and it lives in roughly nine small threshold adjustments: five of eight gains moved from 0.90 to 0.74 by a single multiplicative mutation each, two units carry a bias of plus or minus 0.25, and two have a widened dead zone.

Single-unit lesions, clamping one unit to zero output, give robust scores of 0.66, 1.91, 2.43, 2.52, 6.37, 8.13, 10.97 and 11.17. Four units are individually load-bearing, two are peripheral, and there is no redundancy anywhere, which is what one expects when only 19 of 24 slots carry weight.

We tested whether the hand-designed sensory encoder was doing the work, since it supplies pairwise joint differences that a linear controller would want. A network of eight pure relays, each passing one sensor through with decoder signs copied from the reference controller, achieves 0.69 s and 0 percent survival. A sign-only readout of the feature basis is therefore not sufficient; the thresholds are doing real work.

4.5 Structure of a larger network

A 128-unit network at generation 2209 was dumped and analysed as a graph. Of 384 declared slots, 136 carry weight zero, so mean effective fan-in is 1.94 and six units have no live input at all and emit a constant. Seventy-one units are read by at least one actuator, 108 can influence a torque at all, and 20 are computationally dead. Of the 71 voters, 35 are one hop from a sensor, 33 lie two to five hops deep, and three have no live sensor path and contribute only recurrent state. All thirteen sensors are used.

The functional core contains one 47-unit strongly connected component plus a 3-cycle and a 2-cycle, with 52 of 108 units inside a loop and no self-loops; 27 members of the large loop are read directly by actuators. At a sampled instant, actuator one read a vote sum of -12 out of plus or minus 28, producing -14.35 N.m from 6 units for, 18 against and 4 abstaining, while actuator two was deadlocked 15 to 15 and output exactly zero. That zero is a computed abstention, not an idle unit.

Two units illustrate how much scaffolding survives. Unit 39 has all three slots at weight zero and therefore emits zero forever, yet holds a decoder ballot on two actuators, where it is counted in the fan-in and abstains permanently. Unit 97 reads only unit 39, so it is also permanently zero, which reduces unit 10, nominally a two-input comparator, to a bare relay of one sensor into two actuators with opposite signs. Six units in this network are constant emitters and three of them hold decoder ballots. Nothing in the mutation operator prunes a slot once its weight reaches zero, and nothing notices that an actuator is polling a dead unit.

5. Experiment 2: the 3D biped

Balancing is a fixed point: one attractor, no sequencing, no phase. Walking is a limit cycle, which is why we built it. A cycled curriculum runs five rungs from standing to 0.8 m/s walking; the trainer keeps a frontier rung but always redraws 45 percent of its cases from earlier rungs, so progress on walking cannot quietly destroy standing, and promotion is judged on a fresh draw of cases.

rung	reference: distance	reference: steps	evolved: distance	evolved: time	evolved: score	injured
stand	0.01 m	11	3.05 m	4.00 s	1.000	0%
sway	-0.01 m	14	3.55 m	4.81 s	0.962	0%
creep	1.42 m	19	4.82 m	6.00 s	1.000	0%
walk slow	3.43 m	26	6.13 m	7.69 s	0.954	0%
walk	6.50 m	33.5	8.01 m	10.00 s	0.990	0%

Table 4: 3D biped. The reference is a hand-tuned capture-point controller with continuous gains; the evolved column is a 24-unit ternary population after 537 generations on one core, which cleared all five rungs by generation 120. It hits the 0.8 m/s target exactly and covers more ground than the reference at the same cadence. Single seed.

The evolved controller exceeded our hand-tuned baseline, which is worth stating plainly because the same thing happened on the quadruped. It is also a caution: our baselines are hand-tuned by a non-specialist, so "better than the reference" sets a low bar.

6. Experiment 3: the 3D quadruped

A quadruped gait is a four-phase rhythm and therefore a stronger test than the biped. With twelve ternary actuators, eighteen ternary sensors and no clock, a 32-unit population reached rung three of five in 494 generations, with promotions at generations 40, 48 and 376.

rung	distance	time	score	injured	peak foot force
stand	0.159 m	3.00 s	1.000	0%	35 N
settle	0.206 m	4.00 s	1.000	0%	36 N
creep	0.161 m	5.00 s	0.529	0%	35 N
walk	0.033 m	6.00 s	0.410	0%	35 N

Table 5: 3D quadruped, 32 units, 215 s on one core. Zero injuries at every rung and peak foot force 35 N against a 176 N cap. Our best hand-tuned crawl either survived and barely moved (0.035 m in 5 s) or covered 0.156 m and fell at 1.78 s.

This is a negative result stated positively. 0.16 m in 5 s is about 3 cm per second: the animal shuffles forward while staying upright, and rung three scored 0.410 against a 0.55 pass mark. The four-phase rhythm is not there. The reward function contains no gait shaping at all, no duty-factor term and no contact-sequence reward, which is the most likely route to an actual trot.

7. Where the computation lives is a property of the task

The same two ablations were applied to champions from all three plants: cut every recurrent connection by zeroing the weight on slots whose source is another unit, or reset every per-unit threshold parameter to its default while leaving wiring and weights intact. Both are applied to a copy.

Ablation	pendulum, N=8	biped, N=24	quadruped, N=32
baseline	robust 13.00	8.01 m, 10.00 s	0.033 m, 6.00 s
recurrence cut	11.06 (85% retained)	0.33 m, 1.12 s (93% lost)	0.30 s (95% lost)
thresholds reset	2.49 (81% lost)	0.79 m, 2.61 s (84% lost)	2.60 s (54% lost)
live slots	19 of 24	50 of 72	62 of 96
of which recurrent	4	31	37
recurrent share	21%	62%	60%

Table 6: Ablation across the three plants. On the balancing task the controller is almost feedforward and the solution resides in the thresholds. On the legged plants cutting recurrence is catastrophic: the quadruped cannot even stand, surviving 0.30 s instead of 6.00. Each column is a single champion.

The trend is monotone and, we think, the most interesting thing in the paper. The same unit model and the same mutation operators, presented with tasks of increasing temporal structure, move the solution out of the per-unit thresholds and into the recurrent loops. The recurrent share of live synapses rises from 21 percent to about 60 percent. This is what one should expect from a unit that cannot hold state: when the task needs phase, the only place phase can live is the topology, and the search puts it there. Because the pendulum column provides a case where recurrence is nearly irrelevant, the legged results are a contrast rather than an assertion.

8. Exploits, mis-specifications and retracted claims

Most of the risk in this study was instrumentation. We list the failures because several produced confident, wrong numbers, and because two of them are general hazards for anyone evolving controllers against a hand-written plant.

Selection artifacts. A novelty bonus added to fitness was cached along with the parent score, so a parent kept a bonus for occupying an empty behaviour cell after that cell filled; fitness then rose while episode duration fell and the search froze completely. A minimum-genome-distance rule intended to keep the champion pool diverse instead collapsed it to a single entry, because a four-mutation step is about 0.3 percent genome distance and a single lineage never produces two admissible members. The fix was to derive the pool from the behaviour archive rather than filter entrants.

Landscape artifacts. A dense random decoder places tanh at plus or minus 0.96, so single mutations were invisible to the plant and the fitness landscape was flat. Sparse initialisation with normalisation by active fan-in roughly doubled the climb rate. Separately, the sensor count constant exceeded the number of features the encoder returned, leaving one permanently-zero sensor, and the "hard" validation cases were generated identically to the easy ones.

Plant exploits. On the biped, the reflex step that fires before a certain fall had no refractory period, so a leaning walker re-planted its foot every 4 ms timestep, took 489 steps in 4 seconds and skated 4.66 m without ever balancing on one leg. On the same plant a 0.12 s swing refractory let the hand-tuned reference "walk" at 8.2 steps per second, which is a shuffle with no single-support phase; a realistic 0.30 s swing time was required before the task tested anything. On the quadruped, the servo rate limit drove feet into the ground at about 2 m/s and the contact damper spiked to 45 N, six times static load; 45 N at 0.13 m from the centre of mass against a pitch inertia of 0.0245 gives 240 rad/s^2 , so the animal launched itself. Softening the contact and slowing the servos was necessary before any gait was possible.

Curriculum fighting itself. The standing rungs originally scored proximity to the origin, which penalises the stepping every later rung depends on. Nothing passed rung zero in 982 generations. Scoring pure survival on those rungs instead, promotion through all five rungs took 120 generations. This was the single largest effect of any change we made.

Measurement artifacts. A calibration run that reported "N = 128 does not solve in 225 s" had three worker processes sharing one core, so each trial received a third of a CPU; the conclusion was an artifact of oversubscription, and oversubscribing a sweep silently censors it. In one sweep the wall-clock column gave N = 256 as cheaper than N = 128 (10.03 against 10.79 CPU-hours) while its neuron-update cost was 4.6 times higher, because throughput drifted by a factor of five across arms; wall clock is unusable as a currency here.

A retracted claim. We predicted, and initially reported as confirmed, that eight-unit controllers would tolerate perturbations poorly. That was based on one seed per arm. With thirty seeds the ordering reverses and the eight-unit arm is the most robust of all (Table 3). Single-seed results in this system are worthless: one 2-seed preview of N = 256 was 1.9 times optimistic against its own 30-seed measurement, and within a single arm the cost to solve spans a factor of eight.

9. Discussion and limitations

The strongest claim we can support is the scaling one, because it has a control. Cost rises as about the square of population size while a strictly larger network fails on every seed, and one measurable quantity, the torque displaced per mutated vote, predicts four otherwise unrelated trends. For this architecture and these tasks, the useful direction is downward.

The ablation result is the one we find most suggestive, and it is the weakest evidentially: each column of Table 6 is a single champion. It should be repeated across seeds before it is relied upon. The same applies to the legged locomotion numbers.

We have deliberately not claimed the edge-of-chaos reading. The evolved effective fan-in of 1.94 sits just under the classical critical connectivity of 2 for random Boolean networks, which is suggestive, but the classical value assumes uniformly random Boolean functions and ours are threshold functions on ternary inputs, so the true critical value for our ensemble is not 2. The test is cheap and we have not run it: perturb one unit mid-episode and measure the damage fraction after t steps. If evolved champions sit near critical, damage should neither die out nor engulf the population.

Other limitations. The sensory encoder is hand-designed and supplies pairwise joint differences, so "undifferentiated units self-organise" carries an asterisk, mitigated but not removed by the relay control in Section 4.4. Our behaviour descriptors for the pendulum are strongly correlated, which is why archive coverage stalls near 16 percent regardless of population size; gait descriptors such as duty factor and diagonality are genuinely independent and are the better use of the archive. The robust score saturates once every validation case is held to the cap, so a solved run stops improving by construction. And the holdout gap of two to three percent indicates the archive is mildly fitted to its fixed validation suite, since only the training cases rotate.

Finally, a note on the units themselves. It would be easy to read the architecture as elemental units composing into larger functional ones. The graph analysis in Section 4.5 supports composition in three specific senses: a graded actuator drive assembled from ternary votes, multi-hop feature construction, and cycles capable of holding state. It does not support clean modularity. What we observe is one large recurrent tangle wrapped in a shell of reflexes, with 20 of 128 units computationally dead and three actuator ballots held by permanently silent units.

10. Reproduction

All three plants, the evolutionary search, the diversity archive and the sweep harness are plain JavaScript with no dependencies, plus a small Flask dashboard for running sweeps. The appendix reproduces the load-bearing code verbatim. Practical notes for anyone repeating this: rank on neuron-updates rather than wall clock; do not set worker count above core count; use at least thirteen seeds and preferably thirty per arm, with an odd count so that the median is not decided by a single seed; and record crossings at a ladder of criteria so that a partial run still yields a usable curve. A lower rung is often a sufficient proxy: on the pendulum, measuring at a robust score of 9 costs a third of measuring at the ceiling and preserves the ranking of every arm.

Appendix A: the protoneuron forward pass

The unit model, verbatim. The decision is an argmax over three affine functions of the pre-activation; the actuator tally is the only place a magnitude appears. The commented softmax equivalence is discussed in Section 2.1.

```
function forward(b, s, prev) {
  const f = encode(s), h = new Array(b.N);
  for (let i = 0; i < b.N; i++) {
    let z = b.bias[i];
    for (let k = 0; k < 3; k++) { const sc = b.src[i][k]; z += b.w[i][k] * (sc < SENS ? f[sc] : prev[sc - SENS]); }
    const d = b.fireGain[i] * clamp(z, -12, 12) / DECT;
    const r0 = b.fireBias[i][0] - d, r1 = b.fireBias[i][1], r2 = b.fireBias[i][2] + d;
    h[i] = (r0 > r1 && r0 >= r2) ? -1 : (r2 > r1 && r2 > r0) ? 1 : 0;
  }
  const out = new Array(ACT);
  for (let j = 0; j < ACT; j++) {
    let sum = 0;
    for (let i = 0; i < b.N; i++) sum += b.decoder[j][i] * h[i];
    out[j] = Math.tanh(sum / Math.max(3, (b.dnz[j] || b.N * 0.12) * 0.55));
  }
  return { h, out };
}
```

A.1 Forward pass, from quad4.js. The pendulum and biped implementations are identical in form, differing only in the actuator count.

```
function sanitize(b) {
  for (let i = 0; i < b.N; i++) {
    for (let k = 0; k < 3; k++) {
      if (![-1, 0, 1].includes(b.w[i][k])) b.w[i][k] = 0;
      b.src[i][k] = clamp(b.src[i][k] | 0, 0, SENS + b.N - 1);
      b.fireBias[i][k] = clamp(fin(b.fireBias[i][k]), -5, 5);
    }
    b.fireGain[i] = clamp(fin(b.fireGain[i], 0.9), 0.08, 3.5);
    b.bias[i] = clamp(fin(b.bias[i]), -2.5, 2.5);
  }
  b.dnz = [];
  for (let j = 0; j < ACT; j++) {
    let nz = 0;
    for (let i = 0; i < b.N; i++) { if (![-1, 0, 1].includes(b.decoder[j][i])) b.decoder[j][i] = 0; if
      (b.decoder[j][i]) nz++; }
    b.dnz[j] = nz;
  }
  return b;
}
```

A.2 Invariant enforcement. Every weight is forced back into $\{-1, 0, +1\}$, every source into range, and the active decoder fan-in per actuator is recomputed, which is the quantity that sets torque resolution.

Appendix B: ternary sensory encoding and the pendulum plant

```
function encode(s){
  const f=[],qs=POS_LIMIT*.237,vs=VEL_LIMIT*.26,qd=POS_LIMIT*.259,qw=POS_LIMIT*.333,vd=VEL_LIMIT*.15;
  for(let i=0;i<3;i++)f.push(tri(s.q[i]/qs));
  for(let i=0;i<3;i++)f.push(tri(s.qd[i]/vs));
  f.push(tri((s.q[0]-s.q[1])/qd),tri((s.q[1]-s.q[2])/qd),tri((s.q[0]-s.q[2])/qw));
  f.push(tri((s.qd[0]-s.qd[1])/vd),tri((s.qd[1]-s.qd[2])/vd),tri((s.qd[0]-s.qd[2])/(VEL_LIMIT*.2)));
  f.push(1);
  return f;
}
```

B.1 The 13 ternary features. Thresholds are expressed as fractions of the configured lane, so widening the survival lane rescales what the controller can see rather than silently changing the task.

```
const PM={m:.55,l:.68,g:9.81,damp:.04};
function massMat(q){const M=Array.from({length:3},()=>[0,0,0]);for(let i=0;i<3;i++)for(let j=0;j<3;j++){const
  mult=3-Math.max(i,j);M[i][j]=PM.m*PM.l*PM.l*mult*Math.cos(q[i]-q[j])}return M}
function dM(q,a,b,k){if(a===b)return 0;const
  mult=3-Math.max(a,b),d=q[a]-q[b];if(k===a)return-PM.m*PM.l*PM.l*mult*Math.sin(d);if(k===b)return
  PM.m*PM.l*PM.l*mult*Math.sin(d);return 0}
function solve3(q,i){return-PM.m*PM.g*PM.l*(3-i)*Math.sin(q[i])}
function solve3(A,b){const a=A.map(r=>r.slice()),x=b.slice();for(let k=0;k<3;k++){let p=k;for(let
  i=k+1;i<3;i++)if(Math.abs(a[i][k])>Math.abs(a[p][k]))p=i;[a[k],a[p]]=[a[p],a[k]];[x[k],x[p]]=[x[p],x[k]];const
  piv=Math.abs(a[k][k])<1e-9?(a[k][k]<0?-1:1)*1e-9:a[k][k];for(let i=k+1;i<3;i++){const fac=a[i][k]/piv;for(let
  j=k;j<3;j++)a[i][j]-=fac*a[k][j];x[i]-=fac*x[k]}}const out=[0,0,0];for(let i=2;i>=0;i--){let s=x[i];for(let
  j=i+1;j<3;j++)s-=a[i][j]*out[j];out[i]=s/(Math.abs(a[i][i])<1e-9?(a[i][i]<0?-1:1)*1e-9:a[i][i])}return out}
function accel(q,qd,tau){const M=massMat(q),b=[-gravity(q,0)-PM.damp*qd[0],-gravity(q,1)-PM.damp*qd[1],-gravity(q,2)
  -PM.damp*qd[2]],c=[0,0,0];for(let i=0;i<3;i++)for(let j=0;j<3;j++)for(let k=0;k<3;k++){const
  G=.5*(dM(q,i,j,k)+dM(q,i,k,j)-dM(q,j,k,i));c[i]+=G*qd[j]*qd[k]}return
  solve3(M,[tau[0]-c[0]+b[0],tau[1]-c[1]+b[1],tau[2]-c[2]+b[2]])}
function physics(s,tau,dt=DT){
```

B.2 Triple pendulum mass matrix, its derivatives, the gravity term and the linear solve.

```
function physics(s, tau, dt=DT){
  const q=s.q, v=s.qd, a1=accel(q, v, tau), q2=q.map((x,i)=>x+.5*dt*v[i]), v2=v.map((x,i)=>x+.5*dt*a1[i]), a2=accel(q2, v2, tau), q3=q.map((x,i)=>x+.5*dt*v2[i]), v3=v.map((x,i)=>x+.5*dt*a2[i]), a3=accel(q3, v3, tau), q4=q.map((x,i)=>x+dt*v3[i]), v4=v.map((x,i)=>x+dt*a3[i]), a4=accel(q4, v4, tau);
  s.q=q.map((x,i)=>x+dt*(v[i]+2*v2[i]+2*v3[i]+v4[i])/6);
  s.qd=v.map((x,i)=>x+dt*(a1[i]+2*a2[i]+2*a3[i]+a4[i])/6);
  s.t+=dt; s.torque=tau.slice();
  if(!s.q.every(Number.isFinite)||!s.qd.every(Number.isFinite)){numericalFaults++;s.q=s.q.map(v2=>finiteNum(v2, POS_LIMIT*2));s.qd=s.qd.map(v2=>finiteNum(v2, 0));s.done=true;return}
  s.done=s.q.some(v2=>Math.abs(v2)>POS_LIMIT)||s.qd.some(v2=>Math.abs(v2)>VEL_LIMIT)||s.t>=MAXT;
}
```

B.3 RK4 integration with the failure lanes and a numerical-fault guard.

Appendix C: mutation operators

Most of an undifferentiated population is read by nothing, so a uniformly random mutation is usually invisible to the actuators. Biasing the target toward units the decoder actually reads, and biasing rewiring toward the sensor block, measurably improved the climb rate.

```
function relevant(b) {
  const read = new Set();
  for (let j = 0; j < ACT; j++) for (let i = 0; i < b.N; i++) if (b.decoder[j][i]) read.add(i);
  const deep = new Set(read);
  for (const i of read) for (let k = 0; k < 3; k++) { const sc = b.src[i][k]; if (sc >= SENS) deep.add(sc - SENS); }
  return [...deep];
}
```

C.1 The set of units that can influence an actuator within one hop.

```
function mutate(parent, rng, o) {
  const c = cloneBrain(parent), rel = relevant(c);
  const other = cur => { const a = [-1, 0, 1].filter(v => v !== cur); return a[Math.floor(rng() * a.length)]; };
  const target = () => (rel.length && rng() < 0.85 ? rel[Math.floor(rng() * rel.length)] : Math.floor(rng() * c.N));
  const source = () => (rng() < 0.5 ? Math.floor(rng() * SENS) : Math.floor(rng() * (SENS + c.N)));
  for (let z = 0; z < o.mut; z++) {
    const i = target(), k = Math.floor(rng() * 3), u = rng();
    if (u < 0.24) c.w[i][k] = other(c.w[i][k]);
    else if (u < 0.46) c.src[i][k] = source();
    else if (u < 0.58) c.bias[i] = clamp(c.bias[i] + (rng() < 0.5 ? -1 : 1) * 0.25, -2.5, 2.5);
    else if (u < 0.72 && !o.noThresholds) { const ci = Math.floor(rng() * 3); c.fireBias[i][ci] = clamp(c.fireBias[i][ci] + (rng() < 0.5 ? -1 : 1) * 0.3, -5, 5); }
    else if (u < 0.80 && !o.noThresholds) c.fireGain[i] = clamp(c.fireGain[i] * (rng() < 0.5 ? 0.82 : 1.22), 0.08, 3.5);
    else { const j = Math.floor(rng() * ACT), ni = (rng() < 0.85 && rel.length) ? rel[Math.floor(rng() * rel.length)] : Math.floor(rng() * c.N); c.decoder[j][ni] = other(c.decoder[j][ni]); }
  }
  return sanitize(c);
}
```

C.2 The mutation operator. Note that a weight mutation picks uniformly among the two values a slot does not currently hold, so it can never be a no-op, and that threshold mutations can be disabled for ablation.

Appendix D: diversity archive and cycled objectives

One elite per behaviour cell. Descriptors never span the full unit interval, so the axes track the range actually observed and rebin when it widens; rebinning is free because every elite carries its descriptor. Fixed axes were the difference between 2 percent and 10 percent coverage.

```
const AXES = ['duty', 'rhythm', 'diag'];
const cellKey = c => c.join('|');
let axisRange = { duty: [0.45, 0.95], rhythm: [0.0, 0.35], diag: [0.35, 0.75] };
let BINS = 4;
function resetAxes() { axisRange = { duty: [0.45, 0.95], rhythm: [0.0, 0.35], diag: [0.35, 0.75] }; }
function noteDesc(d) {
  let widened = false;
  for (const k of AXES) {
    const v = clamp(fin(d[k]), 0, 1), r = axisRange[k];
    if (v < r[0] - 1e-9) { r[0] = v; widened = true; }
    if (v > r[1] + 1e-9) { r[1] = v; widened = true; }
  }
  return widened;
}
function binOf(d) {
```

D.1 Adaptive behaviour axes.

```
function binOf(d) {
  return AXES.map(k => {
    const [lo, hi] = axisRange[k], span = Math.max(1e-6, hi - lo);
    return Math.min(BINS - 1, Math.max(0, Math.floor((clamp(fin(d[k]), lo, hi) - lo) / span * BINS * 0.999999)));
  });
}
```

D.2 Binning against the observed range.

```
function tryArchive(arc, brain, ev, gen, serial) {
  if (noteDesc(ev.desc)) rebinArchive(arc);
  const cell = binOf(ev.desc), key = cellKey(cell), cur = arc.get(key);
  const entry = { id: serial, gen, cell, brain: cloneBrain(brain), desc: { ...ev.desc },
    robust: robustScore(ev), dist: ev.dist, score: ev.score, safety: ev.safety,
    meanPeak: ev.meanPeak, injRate: ev.injRate };
  if (!cur || entry.robust > cur.robust + 1e-9) { arc.set(key, entry); return cur ? 'improved' : 'new'; }
  return false;
}
```

D.3 Archive admission.

```
function derivePool(arc, poolSize, minDist) {
  const all = [...arc.values()].sort((a, b) => b.robust - a.robust), picked = [];
  for (const e of all) {
    if (picked.length >= poolSize) break;
    if (minDist > 0 && picked.some(p => genomeDistance(p.brain, e.brain) < minDist)) continue;
    picked.push(e);
  }
  for (const e of all) { if (picked.length >= poolSize) break; if (!picked.includes(e)) picked.push(e); }
  return picked;
}
```

D.4 Deriving a diverse pool from the archive. This replaced a crowding filter that collapsed the pool to one entry (Section 8).

```
function genomeDistance(a, b) {
  if (a.N !== b.N) return 1;
  let diff = 0, tot = 0;
  for (let i = 0; i < a.N; i++) for (let k = 0; k < 3; k++) {
    tot += 2; if (a.src[i][k] !== b.src[i][k]) diff++; if (a.w[i][k] !== b.w[i][k]) diff++;
  }
  for (let j = 0; j < ACT; j++) for (let i = 0; i < a.N; i++) { tot++; if (a.decoder[j][i] !== b.decoder[j][i])
    diff++; }
  return diff / Math.max(1, tot);
}
```

D.5 Normalised genome distance.

```
const OBJECTIVES = [
  { name: 'safe walking', wDist: 1.0, wSafe: 3.5 },
  { name: 'distance', wDist: 5.0, wSafe: 0.8 },
];
const robustScore = e => 2.0 * e.dist + 1.0 * e.score + 1.5 * clamp(1 - e.meanPeak / FMAX, 0, 1) - 3.0 * e.injRate;
function fitness(brain, suite, objIdx) {
  const e = evaluate(brain, suite);
  const w = OBJECTIVES[objIdx || 0] % OBJECTIVES.length;
  return { f: e.rew + 4.0 * w.wDist * e.dist + 2.0 * w.wSafe * e.safety
}
```

D.6 Cycled objectives. Selecting on distance alone buys speed with hard landings; selecting on safety alone converges on standing still.

Appendix E: cycled curriculum

```
const STAGES = [
  { name: 'stand', vTarget: 0.00, T: 3.0, perturb: 0.0, pass: 0.80 },
  { name: 'settle', vTarget: 0.00, T: 4.0, perturb: 0.4, pass: 0.72 },
  { name: 'creep', vTarget: 0.15, T: 5.0, perturb: 0.0, pass: 0.60 },
  { name: 'walk', vTarget: 0.35, T: 6.0, perturb: 0.3, pass: 0.55 },
  { name: 'trot', vTarget: 0.60, T: 7.0, perturb: 0.5, pass: 0.50 },
];
function caseFor(stage, seed) {
```

E.1 The five rungs.

```
function buildSuite(frontier, seedBase, n, rng) {
  const out = [];
  for (let i = 0; i < n; i++) {
    const stage = (rng() < 0.55 || frontier === 0) ? frontier : Math.floor(rng() * frontier);
    out.push(caseFor(stage, (seedBase + i * 104729 + stage * 7919) >>> 0));
  }
  return out;
}
```

E.2 Suite construction. 55 percent of cases come from the frontier rung and the rest are spread over everything already learned.

Appendix F: the 3D biped plant

```
function accel(p, v, f, tau) {
  const r = sub(p, f);
  const Ft = [ (tau[1] * r[2] - 0) / (L * L),           // (tau x r)/L^2 with tau_z = 0
               (0 - tau[0] * r[2]) / (L * L),
               (tau[0] * r[1] - tau[1] * r[0]) / (L * L) ];
  const S = [Ft[0], Ft[1], Ft[2] - M * G];
  const lam = (-M * dot(v, v) - dot(r, S)) / L;
  return [ (S[0] + lam * r[0] / L) / M, (S[1] + lam * r[1] / L) / M, (S[2] + lam * r[2] / L) / M ];
}
```

F.1 Constrained acceleration. The Lagrange multiplier keeps the mass on a sphere of radius L about the stance foot without any small-angle approximation.

```
function integrate(s, tau) {
  const p = s.p, v = s.v, f = s.f;
  const a1 = accel(p, v, f, tau);
  const p2 = p.map((x, i) => x + 0.5 * DT * v[i]), v2 = v.map((x, i) => x + 0.5 * DT * a1[i]);
  const a2 = accel(p2, v2, f, tau);
  const p3 = p.map((x, i) => x + 0.5 * DT * v2[i]), v3 = v.map((x, i) => x + 0.5 * DT * a2[i]);
  const a3 = accel(p3, v3, f, tau);
  const p4 = p.map((x, i) => x + DT * v3[i]), v4 = v.map((x, i) => x + DT * a3[i]);
  const a4 = accel(p4, v4, f, tau);
  const px = p.map((x, i) => x + DT * (v[i] + 2 * v2[i] + 2 * v3[i] + v4[i]) / 6);
  const vx = v.map((x, i) => x + DT * (a1[i] + 2 * a2[i] + 2 * a3[i] + a4[i]) / 6);
  // drift control: re-project onto the sphere and strip radial velocity
  let r = sub(px, f), rn = norm(r);
  if (!Number.isFinite(rn) || rn < 1e-6) { s.done = true; s.fallen = true; return; }
  const k = L / rn;
  s.p = [f[0] + r[0] * k, f[1] + r[1] * k, f[2] + r[2] * k];
  r = sub(s.p, f);
  const rh = [r[0] / L, r[1] / L, r[2] / L], vr = dot(vx, rh);
  s.v = [vx[0] - vr * rh[0], vx[1] - vr * rh[1], vx[2] - vr * rh[2]];
  s.t += DT; s.tStep += DT;
  s.energy += (tau[0] * tau[0] + tau[1] * tau[1]) / (TAU_MAX * TAU_MAX) * DT;
  if (!s.p.every(Number.isFinite) || !s.v.every(Number.isFinite)) { s.done = true; s.fallen = true; return; }
  if (s.p[2] < FALL_H) { s.done = true; s.fallen = true; }
  if (norm(s.v) > VMAX) { s.done = true; s.fallen = true; }
}
```

F.2 RK4 with re-projection onto the sphere and removal of radial velocity, which prevents the constraint drifting.

```
function takeStep(s, dirx, diry) {
  const newSide = -s.side;
  const h = clamp(s.p[2], 0.05, L - 1e-4);
  const rad = Math.sqrt(Math.max(1e-6, L * L - h * h));
  let dx = clamp(dirx, -1, 1), dy = clamp(diry, -1, 1) + newSide * 0.75;
  const dn = Math.hypot(dx, dy) || 1;
  dx /= dn; dy /= dn;
  const f2 = [s.p[0] + rad * dx, s.p[1] + rad * dy, 0];
  const r2 = sub(s.p, f2), r2n = norm(r2);
  if (!Number.isFinite(r2n) || r2n < 1e-6) { s.done = true; s.fallen = true; return; }
  const rh = [r2[0] / r2n, r2[1] / r2n, r2[2] / r2n];
  const vr = dot(s.v, rh);
  s.lastImpulse = Math.abs(vr);
  s.peakImpulse = Math.max(s.peakImpulse, s.lastImpulse);
  if (s.lastImpulse > INJURY) { s.injured = true; s.done = true; return; }
  s.v = [s.v[0] - vr * rh[0], s.v[1] - vr * rh[1], s.v[2] - vr * rh[2]];
  s.f = f2; s.side = newSide; s.tStep = 0; s.steps++;
}
```

F.3 Foot placement and the impulsive collision. The destroyed radial velocity is the energy sink that makes a steady gait a limit cycle, and its magnitude is the injury measure.

Appendix G: the 3D quadruped plant

```
function footBody(leg, th, ph, kn) {
  const s = Math.sin(SY[leg] * ph), c = Math.cos(SY[leg] * ph);
  const d1 = [c * Math.sin(th), s, -c * Math.cos(th)];
  const a2 = th - kn;
  const d2 = [c * Math.sin(a2), s, -c * Math.cos(a2)];
  const h = HIP[leg];
  return [h[0] + L1 * d1[0] + L2 * d2[0], h[1] + L1 * d1[1] + L2 * d2[1], h[2] + L1 * d1[2] + L2 * d2[2]];
}
```

G.1 Two-segment leg kinematics with outward abduction and a backward-bending knee.

```

function physics(s, targets) {
  // servos move toward the commanded angles at a bounded rate
  const lim = RATE * DT;
  for (let i = 0; i < 4; i++) {
    s.th[i] += clamp(targets[3 * i] - s.th[i], -lim, lim);
    s.ph[i] += clamp(targets[3 * i + 1] - s.ph[i], -lim, lim);
    s.kn[i] += clamp(targets[3 * i + 2] - s.kn[i], -lim, lim);
    s.energy += (Math.abs(targets[3 * i] - s.th[i]) + Math.abs(targets[3 * i + 2] - s.kn[i])) * DT;
  }
  let F = [0, 0, -M * G], Tq = [0, 0, 0];
  for (let i = 0; i < 4; i++) {
    const q = footWorld(s, i);
    const fp = s.fprev[i];
    const vel = fp ? [(q[0] - fp[0]) / DT, (q[1] - fp[1]) / DT, (q[2] - fp[2]) / DT] : [0, 0, 0];
    s.fprev[i] = q.slice();
    if (q[2] < 0) {
      const pen = -q[2];
      const Fn = Math.max(0, KN * pen - DN * vel[2]);
      s.peakF = Math.max(s.peakF, Fn);
      if (Fn > FMAX) { s.injured = true; s.done = true; return; }
      if (!s.anchor[i]) s.anchor[i] = [q[0], q[1]];
      let dx = q[0] - s.anchor[i][0], dy = q[1] - s.anchor[i][1];
      const dm = Math.hypot(dx, dy), limF = MU * Fn;
      if (KT * dm > limF && dm > 1e-9) { // slip: drag the anchor
        const keep = limF / (KT * dm);
        s.anchor[i][0] += dx * (1 - keep); s.anchor[i][1] += dy * (1 - keep);
        dx *= keep; dy *= keep; s.slip += (1 - keep) * dm;
      }
      const Ft = [clamp(-KT * dx - DTC * vel[0], -limF, limF), clamp(-KT * dy - DTC * vel[1], -limF, limF)];
      const Fi = [Ft[0], Ft[1], Fn];
      F = [F[0] + Fi[0], F[1] + Fi[1], F[2] + Fi[2]];
      const rr = [q[0] - s.p[0], q[1] - s.p[1], q[2] - s.p[2]], tq = cross(rr, Fi);
      Tq = [Tq[0] + tq[0], Tq[1] + tq[1], Tq[2] + tq[2]];
      s.contact[i] = 1;
    } else { s.anchor[i] = null; s.contact[i] = 0; }
  }
  // linear
  const a = [F[0] / M, F[1] / M, F[2] / M];
  for (let k = 0; k < 3; k++) { s.v[k] += a[k] * DT; s.p[k] += s.v[k] * DT; }
  // angular, in the body frame
  const Tb = mult(s.R, Tq), I = [IX, IY, IZ];
  const Iw = [I[0] * s.w[0], I[1] * s.w[1], I[2] * s.w[2]], gy = cross(s.w, Iw);
  for (let k = 0; k < 3; k++) s.w[k] += ((Tb[k] - gy[k]) / I[k]) * DT;
  const om = s.w, th = Math.hypot(om[0], om[1], om[2]) * DT;
  if (th > 1e-9) { // Rodrigues, then re-orthonormalise
    const ax = [om[0] * DT / th, om[1] * DT / th, om[2] * DT / th];
    const ct = Math.cos(th), st = Math.sin(th), vt = 1 - ct;
    const Rr = [ct + ax[0] * ax[0] * vt, ax[0] * ax[1] * vt - ax[2] * st, ax[0] * ax[2] * vt + ax[1] * st,
      ax[1] * ax[0] * vt + ax[2] * st, ct + ax[1] * ax[1] * vt, ax[1] * ax[2] * vt - ax[0] * st,
      ax[2] * ax[0] * vt - ax[1] * st, ax[2] * ax[1] * vt + ax[0] * st, ct + ax[2] * ax[2] * vt];
    const R = s.R, out = new Array(9);
    for (let i2 = 0; i2 < 3; i2++) for (let j = 0; j < 3; j++) {
      out[i2 * 3 + j] = R[i2 * 3] * Rr[j] + R[i2 * 3 + 1] * Rr[3 + j] + R[i2 * 3 + 2] * Rr[6 + j];
    }
    s.R = fixR(out);
  }
  s.t += DT;
  if (!s.p.every(Number.isFinite) || !s.v.every(Number.isFinite) || !s.R.every(Number.isFinite)) {
    s.done = true; s.fallen = true; return;
  }
  const up = [s.R[2], s.R[5], s.R[8]]; // body z in world
  const tilt = Math.acos(clamp(up[2], -1, 1));
  s.tilt = tilt;
  if (s.p[2] < FALL_H || tilt > MAX_TILT || Math.hypot(s.v[0], s.v[1], s.v[2]) > VMAX) { s.done = true; s.fallen = true; }
}

```

G.2 Rate-limited servos, penalty contact with a stick-slip friction anchor, and full 6-DOF torso dynamics with the rotation matrix integrated by Rodrigues and re-orthonormalised.

Appendix H: the compute-to-half-solved statistic

```

function halfSolved(vals) {
  const s = vals.slice().sort((a, b) => a - b);
  return s[Math.ceil(s.length / 2) - 1];
}

```

H.1 The order statistic. Censored seeds enter as infinity and the median remains exact provided more than half the seeds cross.

```

function boot(vals) {
  const out = [];
  let censored = 0;

```

```

for (let b = 0; b < B; b++) {
  const r = Array.from({ length: vals.length }, () => vals[(Math.random() * vals.length) | 0]);
  const h = halfSolved(r);
  if (!isFinite(h)) censored++; else out.push(h);
}
out.sort((a, b) => a - b);
return {
  lo: out.length ? out[Math.floor(out.length * 0.05)] : Infinity,
  hi: out.length ? out[Math.floor(out.length * 0.95)] : Infinity,
  censoredFrac: censored / B,
};
}

```

H.2 Percentile bootstrap over seeds, reporting the fraction of resamples in which half solved is never reached.

Appendix I: episode scoring and the robust score

```

function scoreSet(brain, suite, opts){
  sanitizeBrain(brain);
  const durations=[]; let total=0, eff=0, act=0, tempo=0, top=0, match=0;
  for(let i=0; i<suite.length; i++){
    const ep=runEpisode(brain, suite[i], opts);
    durations.push(ep.duration); total+=ep.total; eff+=ep.effort; act+=ep.activity; tempo+=ep.tempo; top+=ep.topShare; match+=ep.match;
  }
  const n=Math.max(1, durations.length), sorted=durations.slice().sort((a,b)=>a-b);
  const mean=sorted.reduce((a,b)=>a+b, 0)/n;
  const median=n%2?sorted[(n-1)>>1]:.5*(sorted[n/2-1]+sorted[n/2]);
  const worst=sorted[0]||0;
  const survival=sorted.filter(v=>v>=SURV).length/n;
  const sc=Math.max(.2, MAXT/10);
  // Robust score deliberately rewards broad survival rather than one long run; time terms are
  // normalised by the episode cap so scores stay comparable when you change episode length.
  const robust=(.55*mean+.25*median+.20*worst)/sc+3*survival;
  return{mean, median, worst, survival, total, robust, durations:sorted, teacherMatch:match/n,
    desc:{effort:clamp(eff/n, 0, 1), activity:clamp(act/n, 0, 1), tempo:clamp(tempo/n, 0, 1), topShare:clamp(top/n, 0, 1)}};
}

```

I.1 Suite scoring. The robust score deliberately rewards broad survival over one long run, and the time terms are normalised by the episode cap so that scores remain comparable when the cap changes. Its ceiling is 13.000, which a solved run reaches exactly.

Appendix J: complete source

Appendices A to I reproduce the load-bearing routines with commentary. This appendix is the complete contents of every file, verbatim, so that the results above can be reproduced without reference to any external archive. Long lines are wrapped to the page width; a wrapped continuation is indented four spaces relative to its parent line. Two files are browser documents and include their own markup and styling; the computational content of the pendulum lab is the script element roughly two thirds of the way through the file.

J.1 protoneuron-champion-lab-v12.html — the triple pendulum lab: plant, protoneuron model, evolutionary search, MAP-Elites archive, connectome biopsy, and browser interface.

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Protoneuron Triple-Pendulum Lab v12 – Champion Evolution</title>
<style>
:root{color-scheme:dark;--bg:#06101b;--panel:#0f1b2c;--panel2:#091522;--line:#29415f;--text:#eaf3ff;--muted:#9fb1c8;
--cyan:#53d9ff;--green:#55e58b;--gold:#ffd166;--red:#ff6f7d;--violet:#b9a0ff}
*{box-sizing:border-box}body{margin:0;background:radial-gradient(circle at top,#112540,#06101b
62%);font-family:Segoe UI,Arial,sans-serif;color:var(--text)}.wrap{max-width:1500px;margin:auto;padding:16px}.gr
id{display:grid;grid-template-columns:1.15fr .85fr;gap:14px}.grid2{display:grid;grid-template-columns:1fr
1fr;gap:14px}@media(max-width:1080px){.grid,.grid2{grid-template-columns:1fr}}
.card{background:rgba(15,27,44,.97);border:1px solid var(--line);border-radius:14px;padding:14px;box-shadow:0 12px
30px #000055}.sub{color:var(--muted);line-height:1.5;margin:5px 0 12px}h1{margin:0;font-size:28px}h3{margin:4px 0
10px}.controls{display:flex;flex-wrap:wrap;gap:7px;margin:10px 0}button,input,select{border:1px solid
var(--line);border-radius:8px;padding:9px 11px;background:#142844;color:var(--text)}button{cursor:pointer;font-w
eight:700}button.primary{background:var(--cyan);color:#04121b}button.good{background:var(--green);color:#062014}
button.warn{background:var(--red);color:#241904}button.bad{background:var(--red);color:#25040a}button.violet{ba
ckground:var(--violet);color:#160f29}button.disabled{opacity:.45;cursor:not-allowed}input{width:88px;textarea{wi
dth:100%;min-height:96px;background:#07101d;color:var(--text);border:1px solid
var(--line);border-radius:9px;padding:9px;font:11px Consolas,monospace}.small{font-size:12px;color:var(--muted)}
.tiny{font-size:11px;color:var(--muted)}label{font-size:12px;color:var(--muted);display:inline-flex;align-items:
center;gap:4px}.metrics{display:grid;grid-template-columns:repeat(5,1fr);gap:7px}@media(max-width:950px){.metric
s{grid-template-columns:repeat(3,1fr)}}.m{background:var(--panel2);border:1px solid
var(--line);border-radius:9px;padding:8px}.m span{display:block;color:var(--muted);font-size:11px}.m
b{font-size:18px}.status{padding:9px 10px;border-radius:9px;background:#081523;border:1px solid var(--line);min-
height:40px}.ok{color:var(--green)}.warn{color:var(--gold)}.bad{color:var(--red)}.pill{display:inline-bl
ock;border:1px solid var(--line);border-radius:999px;padding:3px 7px;margin:2px;font-size:11px}.pill
b{color:var(--gold)}.legend{line-height:1.55}.head{border-top:1px solid
#1b3048;margin-top:12px;padding-top:9px;color:var(--cyan);font-size:11px;letter-spacing:.04em}
canvas{display:block;width:100%;height:auto;background:#081321;border:1px solid
var(--line);border-radius:10px}.vizWrap{position:relative}.vizLabel{position:absolute;left:14px;top:12px;padding
:5px 8px;border-radius:999px;background:#071321dd;border:1px solid
var(--line);font-size:11px;color:var(--muted)}#history,#archiveGrid{width:100%;background:#081321;border:1px
solid
var(--line);border-radius:10px;display:block}#history{min-height:260px}#archiveGrid{min-height:300px}#history
.axis,#archiveGrid .axis{stroke:#29415f;stroke-width:1}#history
.gridline{stroke:#1b3048;stroke-width:1;stroke-dasharray:4 6}#history
.evo{fill:none;stroke:var(--cyan);stroke-width:2.2}#history
.champ{fill:none;stroke:var(--gold);stroke-width:2.2}#history
.bpt{fill:none;stroke:var(--violet);stroke-width:2.2}
pre{background:#07101d;border:1px solid var(--line);border-radius:9px;padding:10px;min-height:240px;max-height:380px
;overflow:auto;white-space:pre-wrap;font:12px Consolas,monospace}.section{padding-top:8px;border-top:1px solid
#1b3048;margin-top:10px}.callout{border-left:3px solid
var(--cyan);padding-left:10px;color:#cddbea;line-height:1.5}.methodGrid{display:grid;grid-template-columns:1fr
1fr;gap:8px}@media(max-width:850px){.methodGrid{grid-template-columns:1fr}}
</style>
</head>
<body><div class="wrap">
<h1>Protoneuron Triple-Pendulum Connectome Lab v12</h1>
<div class="sub">A deliberately undifferentiated recurrent population controls a three-link inverted pendulum. Every
protoneuron has exactly <b>3 incoming slots</b>; synaptic weights and communicated activations are ternary
<b>-1, 0, +1</b>. This build runs one method only – <b>evolution with champions</b> – and adds a diversity
archive so the champions you keep are behaviourally different brains rather than eight copies of the same one.
The survival lane, episode length, and every champion-feed ratio are editable while it runs.</div>

<div class="grid">
<div class="card">
<div class="vizWrap"><canvas id="world" width="900" height="420"></canvas><div class="vizLabel"
id="modeBadge">IDLE</div></div>
<div class="controls">
<button class="warn" id="start">Start fresh run</button>
<button class="violet" id="continue">Continue from champions</button>
<button id="pause">Pause</button><button id="resume">Resume</button>
<button class="good" id="test">Test paused current</button><button id="evalCurrent">Robust eval
current</button><button id="perturb">Perturb now</button><button class="bad" id="reset">Reset</button>
</div>
<div class="metrics">
```

```

<div class="m"><span>Mode</span><b id="mode">idle</b></div><div class="m"><span>Generation</span><b
id="gen">0</b></div><div class="m"><span>Live time</span><b id="time">0.00 s</b></div><div
class="m"><span>Neurons</span><b id="nCount">128</b></div><div class="m"><span>Numerical faults</span><b
id="faults">0</b></div>
<div class="m"><span>Champion mean</span><b id="best">0.00 s</b></div><div class="m"><span>Champion
worst</span><b id="worst">0.00 s</b></div><div class="m"><span>Champion survival</span><b
id="survival">0%</b></div><div class="m"><span>Archive cells</span><b id="cells">0</b></div><div
class="m"><span>Archive coverage</span><b id="coverage">0%</b></div>
</div>
<div class="status" id="status">Press Start fresh run. Training is accelerated; Test pauses learning and exposes
only the frozen current brain.</div>
</div>

<div class="card">
<h3>Champion evolution setup</h3>
<div class="head">Brain and search</div>
<div class="controls">
<label>Protoneurons <input id="neurons" type="number" min="8" max="512" step="8" value="128"></label>
<label>Seed <input id="seed" type="number" value="47017"></label>
<label>Mutations/candidate <input id="mutations" type="number" min="1" max="64" value="4"></label>
<label>Candidates/gen <input id="candidates" type="number" min="2" max="16" value="8"></label>
<label>Strong jump every <input id="strongEvery" type="number" min="0" max="500" value="20"></label>
<label>Mutate read neurons % <input id="focus" type="number" min="0" max="100" step="1" value="85"></label>
<label>Rewire to sensors % <input id="sensorBias" type="number" min="0" max="100" step="1" value="50"></label>
<label>Decision temp <input id="dect" type="number" min="0.05" max="2" step="0.01" value="0.22"></label>
<label>Decoder density % <input id="decDensity" type="number" min="1" max="100" step="1" value="12"></label>
</div>
<div class="head">Survival lane and episodes</div>
<div class="controls">
<label>Angle lane &plusmn;&deg; <input id="laneDeg" type="number" min="10" max="150" step="1" value="77"></label>
<label>Velocity lane <input id="velLane" type="number" min="2" max="40" step="0.5" value="10"></label>
<label>Max torque <input id="torque" type="number" min="1" max="40" step="0.5" value="22"></label>
<label>Episode cap s <input id="maxT" type="number" min="2" max="60" step="0.5" value="10"></label>
<label>Survival &e; s <input id="survS" type="number" min="0.5" max="60" step="0.5" value="9"></label>
<label>Start &plusmn;&deg; <input id="startDeg" type="number" min="0.5" max="60" step="0.5" value="10"></label>
<label>Start vel &plusmn; <input id="startVel" type="number" min="0" max="6" step="0.05" value="0.75"></label>
<label>Hard case &times; <input id="hardMul" type="number" min="1" max="3" step="0.05" value="1.45"></label>
<label>Perturb &times; <input id="perturbMul" type="number" min="0" max="4" step="0.1" value="1"></label>
</div>
<div class="head">Champion feeds</div>
<div class="controls">
<label>Champion pool <input id="poolSize" type="number" min="1" max="24" value="8"></label>
<label>Revisit threshold <input id="threshold" type="number" min="5" max="500" value="60"></label>
<label>Max parent from champions % <input id="revisitPct" type="number" min="0" max="100" value="35"></label>
<label>&hellip;of those, archive % <input id="archivePct" type="number" min="0" max="100" value="45"></label>
<label>Crossover % <input id="crossPct" type="number" min="0" max="100" value="20"></label>
<label>Selection exponent <input id="selExp" type="number" min="0.3" max="8" step="0.1" value="1.8"></label>
</div>
<div class="head">Diversity archive and validation</div>
<div class="controls">
<label>Bins per axis <input id="bins" type="number" min="2" max="10" value="5"></label>
<label>Archive cap <input id="archiveCap" type="number" min="8" max="400" value="120"></label>
<label>Min genome distance % <input id="minDist" type="number" min="0" max="60" step="0.5" value="4"></label>
<label>Novelty bonus <input id="novelty" type="number" min="0" max="20" step="0.1" value="0"></label>
<label>Training cases <input id="trainCases" type="number" min="2" max="32" value="12"></label>
<label>Rotate cases every <input id="rotate" type="number" min="0" max="500" step="1" value="25"></label>
<label>Validation trials <input id="validationTrials" type="number" min="12" max="128" value="48"></label>
<label>Validate every <input id="valEvery" type="number" min="1" max="25" value="3"></label>
</div>
<div class="methodGrid" style="margin-top:10px">
<div class="callout"><b>Champion pool</b><br><span class="small">Derived from the archive: best first, skipping
any brain closer than the minimum genome distance to one already chosen.</span></div>
<div class="callout"><b>Diversity archive</b><br><span class="small">One elite per behaviour cell (control effort
&times; firing activity &times; top-joint share). Each generation also validates the best candidate that
landed in an empty cell, so losing strategies still get kept.</span></div>
<div class="callout"><b>Feeds</b><br><span class="small">Each candidate's parent is the current brain, a pool
champion, or a random archive cell; crossover splices a second champion's neuron block.</span></div>
<div class="callout"><b>Lane</b><br><span class="small">Angle and velocity lanes define both failure and the
sensory scale, so the encoder rescales itself when you widen the lane.</span></div>
</div>
<div class="controls"><button id="crossval" class="good">Cross-validate champions</button><button id="audit">Audit
connectome</button><button id="copyBiopsy">Copy biopsy</button><button id="rescore">Rescore +
rebin</button><button id="reseed">Revisit champion</button><button id="loadTop">Load top
champion</button><button class="bad" id="clearArchive">Clear champions</button></div>
<div class="status" id="auditOut">Not audited.</div>
<div class="status" id="championStatus" style="margin-top:7px">Champions: 0</div>
<div class="tiny" style="margin-top:8px">Neuron count and seed take effect after Reset. Lane, episode, and suite
changes rebuild the case suites immediately – use Rescore + rebin to re-place existing champions under the new
settings so scores stay comparable.</div>
</div>
</div>

<div class="grid2" style="margin-top:14px">
<div class="card"><h3>Validation history</h3><svg id="history" viewBox="0 0 860 260"
preserveAspectRatio="none"></svg><div id="historyReadout" class="small">No validation samples yet.</div><div

```

```

        class="tiny" style="margin-top:6px">Gold = mean duration • cyan = worst case • violet = survival share scaled to
        the episode cap.</div></div>
<div class="card"><h3>Run log</h3><pre id="log">Ready. One method: evolution with champions.</pre></div>
</div>

<div class="grid2" style="margin-top:14px">
<div class="card"><h3>Behaviour archive</h3>
<svg id="archiveGrid" viewBox="0 0 430 330" preserveAspectRatio="xMidYMid meet"></svg>
<div class="tiny" style="margin-top:6px">Each filled cell holds the best brain found with that behaviour. Brighter
    is a higher robust score; the number is how many third-axis cells sit behind that square.</div>
<div id="archiveList" class="legend" style="margin-top:8px"><span class="small">No champions yet.</span></div>
</div>
<div class="card"><h3>Save and load champions</h3>
<div class="small">Champions and the whole archive are saved as one payload: pool, behaviour cells, descriptors,
    and packed ternary genomes. Browser storage is used when available; the text box below always works.</div>
<div class="controls"><button class="primary" id="saveStore">Save to browser</button><button id="loadStore">Load
    from browser</button><button id="exportBox">Write to box</button><button id="importBox">Load from
    box</button><button id="copyBox">Copy box</button><button id="downloadJson">Download file</button></div>
<textarea id="ioBox" spellcheck="false" placeholder="Champion payload JSON appears here. Paste a saved payload and
    press Load from box."></textarea>
<div class="status" id="ioStatus" style="margin-top:8px">Nothing saved yet this session.</div>
<div class="tiny" style="margin-top:6px">Payloads are keyed to seed, neuron count, trial count, lane, and bin
    count. Loading a payload from different settings still works, but rescore before trusting the numbers.</div>
</div>
</div>

<div class="card" id="networkCard" style="margin-top:14px;display:none">
<h3>Live connectome – visible only during frozen-current testing</h3>
<div class="small">Green node = +1 firing • red node = &minus;1 firing • gray = 0. Green edge = +1 weight • red =
    &minus;1 • dim edge = 0. Pulses show currently active recurrent transmission.</div>
<canvas id="netview" width="1200" height="760" style="margin-top:8px"></canvas>
</div>
</div>

<script>
'use strict';
const $id=>document.getElementById(id), world=$( 'world' ), wctx=world.getContext( '2d' ), netview=$( 'netview' ),
    nctx=netview.getContext( '2d' );
let rng,net,state,mode='idle',paused=true,running=false,generation=0;
let bestMean=0,bestWorst=0,bestSurvival=0,history=[],trainTimer=0;
let champions=[],archive=new Map(),championSerial=0,stagnation=0;
let trainingPool=[],validationPool=[],holdoutPool=[];
let currentTestRng=null,lastFrame=performance.now(),layoutCache=null,numericalFaults=0;
let lastParentTag='current',lastAdmit='none yet',busy=false;
const SENS=13, DT=0.025;
let MAX_TORQUE=22, MAXT=10, POS_LIMIT=1.35, VEL_LIMIT=10, SURV=9, DECT=.22;
const CFG={};

function PRNG(seed){let s=(seed>>>0)||1;return()=>{s=(s+0x6D2B79F5)>>>0;let
    t=s;t=Math.imul(t^(t>>>15),t|1);t^=t+Math.imul(t^(t>>>7),t|61);return((t^(t>>>14))>>>0)/4294967296}}
const clamp=(x,a,b)>=>Math.max(a,Math.min(b,x)), tri=x>=x>0.33?1:x<0.33?-1:0,
    choice=(r,a)>=>a[Math.floor(r()*a.length)];
function finiteNum(x,f=0){return Number.isFinite(x)?x:f}
function otherTern(r,cur){const o=[-1,0,1].filter(v=>v!==cur);return o[Math.floor(r()*o.length)]}

function readCfg(){
    CFG.N=clamp(Math.round(+$('neurons').value|128),8,512);
    CFG.seed=(+$('seed').value|47017)>>>0;
    CFG.mut=clamp(Math.round(+$('mutations').value|4),1,64);
    CFG.cand=clamp(Math.round(+$('candidates').value|8),2,16);
    CFG.strongEvery=clamp(Math.round(+$('strongEvery').value|0),0,500);
    CFG.focus=clamp(+$('focus').value|0,0,100)/100;
    CFG.sensorBias=clamp(+$('sensorBias').value|0,0,100)/100;
    CFG.dect=clamp(+$('dect').value|.22,.05,2);
    CFG.decDensity=clamp(+$('decDensity').value|12,1,100)/100;
    CFG.laneDeg=clamp(+$('laneDeg').value|77,10,150);
    CFG.vellLane=clamp(+$('vellLane').value|10,2,40);
    CFG.torque=clamp(+$('torque').value|9,1,40);
    CFG.maxT=clamp(+$('maxT').value|10,2,60);
    CFG.survS=clamp(+$('survS').value|9,.5,60);
    CFG.startDeg=clamp(+$('startDeg').value|10,.5,60);
    CFG.startVel=clamp(+$('startVel').value|0,0,6);
    CFG.hardMul=clamp(+$('hardMul').value|1.45,1,3);
    CFG.perturbMul=clamp(+$('perturbMul').value|0,0,4);
    CFG.pool=clamp(Math.round(+$('poolSize').value|8),1,24);
    CFG.threshold=clamp(Math.round(+$('threshold').value|60),5,500);
    CFG.revisitPct=clamp(+$('revisitPct').value|0,0,100);
    CFG.archivePct=clamp(+$('archivePct').value|0,0,100);
    CFG.crossPct=clamp(+$('crossPct').value|0,0,100);
    CFG.selExp=clamp(+$('selExp').value|1.8,.3,8);
    CFG.bins=clamp(Math.round(+$('bins').value|5),2,10);
    CFG.archiveCap=clamp(Math.round(+$('archiveCap').value|120),8,400);
    CFG.minDist=clamp(+$('minDist').value|0,0,60)/100;
    CFG.novelty=clamp(+$('novelty').value|0,0,20);
    CFG.trainCases=clamp(Math.round(+$('trainCases').value|12),2,32);

```

```

CFG.rotate=clamp(Math.round(+$('rotate').value||0),0,500);
CFG.valTrials=clamp(Math.round(+$('validationTrials').value||48),12,128);
CFG.valEvery=clamp(Math.round(+$('valEvery').value||3),1,25);
POS_LIMIT=CFG.laneDeg*Math.PI/180; VEL_LIMIT=CFG.velLane; MAX_TORQUE=CFG.torque; MAXT=CFG.maxT;
    SURV=Math.min(CFG.survS,CFG.maxT); DECT=CFG.dect;
}

function sanitizeBrain(b){
  for(let i=0;i<b.N;i++){
    for(let k=0;k<3;k++){
      if(![-1,0,1].includes(b.w[i][k]))b.w[i][k]=0;
      b.src[i][k]=Math.min(SENS+b.N-1,Math.max(0,b.src[i][k]|0));
      b.fireBias[i][k]=clamp(finiteNum(b.fireBias[i][k]|0),-5,5);
    }
    b.fireGain[i]=clamp(finiteNum(b.fireGain[i]|.9),.08,3.5);
    b.bias[i]=clamp(finiteNum(b.bias[i]|0),-2.5,2.5);
  }
  b.dnz=[0,0,0];
  for(let j=0;j<3;j++){let nz=0;for(let i=0;i<b.N;i++){if(![-1,0,1].includes(b.decoder[j][i]))b.decoder[j][i]=0;if(b.decoder[j][i])nz++;}b.dnz[j]=nz}
  return b;
}

function deepCloneNet(n){return{N:n.N,src:n.src.map(r=>r.slice()),w:n.w.map(r=>r.slice()),decoder:n.decoder.map(r=>r.slice()),bias:n.bias?n.bias.slice():Array(n.N).fill(0),fireBias:n.fireBias?n.fireBias.map(r=>r.slice()):Array(n.N).fill(0),fireGain:n.fireGain?n.fireGain.slice():Array(n.N).fill(.9),seed:n.seed}}
function makeNet(){
  const N=CFG.N,total=SENS+N,src=[],w=[];
  for(let i=0;i<N;i++){src.push([Math.floor(rng()*total),Math.floor(rng()*total),Math.floor(rng()*total)]);w.push([Math.floor(rng()*3)-1,Math.floor(rng()*3)-1,Math.floor(rng()*3)-1]);}
  const dens=CFG.decDensity;
  const decoder=Array.from({length:3},()=>Array.from({length:N},()=>rng()<dens?(rng()<.5?-1:1):0));
  return sanitizeBrain({N,src,w,decoder,bias:Array(N).fill(0),fireBias:Array(N).fill(0),fireGain:Array(N).fill(.9),seed:CFG.seed});
}

function freshState(r,hard=false){const m=hard?CFG.hardMul:1,a=CFG.startDeg*Math.PI/180,v=CFG.startVel*m;return{q:[(r()-1.5)*2*a,(r()-1.5)*2*a,(r()-1.5)*2*v],qd:[(r()-1.5)*2*v,(r()-1.5)*2*v,(r()-1.5)*2*v],t:0,torque:[0,0,0],done:false,impulse:0}
function cloneState(s){return{q:s.q.slice(),qd:s.qd.slice(),t:s.t,torque:s.torque.slice(),done:s.done,impulse:s.impulse,caseSeed:s.caseSeed||0}}
function encode(s){
  const f=[],qs=POS_LIMIT*.237,vs=VEL_LIMIT*.26,qd=POS_LIMIT*.259,qw=POS_LIMIT*.333,vd=VEL_LIMIT*.15;
  for(let i=0;i<3;i++)f.push(tri(s.q[i]/qs));
  for(let i=0;i<3;i++)f.push(tri(s.qd[i]/vs));
  f.push(tri((s.q[0]-s.q[1])/qd),tri((s.q[1]-s.q[2])/qd),tri((s.q[0]-s.q[2])/qw));
  f.push(tri((s.qd[0]-s.qd[1])/vd),tri((s.qd[1]-s.qd[2])/vd),tri((s.qd[0]-s.qd[2])/(VEL_LIMIT*.2)));
  f.push(1);
  return f;
}

// Point-mass triple pendulum. Rods are massless; each joint angle is measured from upright.
const PM={m:.55,l:.68,g:9.81,damp:.04};
function massMat(q){const M=Array.from({length:3},()=>[0,0,0]);for(let i=0;i<3;i++)for(let j=0;j<3;j++){const mult=3-Math.max(i,j);M[i][j]=PM.m*PM.l*PM.l*mult*Math.cos(q[i]-q[j])}return M}
function dM(q,a,b,k){if(a===b)return 0;const mult=3-Math.max(a,b),d=q[a]-q[b];if(k===a)return-PM.m*PM.l*PM.l*mult*Math.sin(d);if(k===b)return PM.m*PM.l*PM.l*mult*Math.sin(d);return 0}
function gravity(q,i){return-PM.m*PM.g*PM.l*(3-i)*Math.sin(q[i])}
function solve3(A,b){const a=A.map(r=>r.slice()),x=b.slice();for(let k=0;k<3;k++){let p=k;for(let i=k+1;i<3;i++)if(Math.abs(a[i][k])>Math.abs(a[p][k]))p=i;[a[k],a[p]]=[a[p],a[k]];[x[k],x[p]]=[x[p],x[k]];const piv=Math.abs(a[k][k])<1e-9?(a[k][k]<0?-1:1)*1e-9:a[k][k];for(let i=k+1;i<3;i++){const fac=a[i][k]/piv;for(let j=k;j<3;j++){a[i][j]-=fac*a[k][j];x[i]-=fac*x[k]}}const out=[0,0,0];for(let i=2;i>=0;i--){let s=x[i];for(let j=i+1;j<3;j++)s-=a[i][j]*out[j];out[i]=s/(Math.abs(a[i][i])<1e-9?(a[i][i]<0?-1:1)*1e-9:a[i][i])}return out}
function accel(q,qd,tau){const M=massMat(q),b=[-gravity(q,0)-PM.damp*qd[0],-gravity(q,1)-PM.damp*qd[1],-gravity(q,2)-PM.damp*qd[2]],c=[0,0,0];for(let i=0;i<3;i++)for(let j=0;j<3;j++)for(let k=0;k<3;k++){const G=.5*(dM(q,i,j,k)+dM(q,i,k,j)-dM(q,j,k,i));c[i]+=G*qd[j]*qd[k]}return solve3(M,[tau[0]-c[0]+b[0],tau[1]-c[1]+b[1],tau[2]-c[2]+b[2]])}
function physics(s,tau,dt=DT){
  const q=s.q,v=s.qd,a1=accel(q,v,tau),q2=q.map((x,i)=>x+.5*dt*v[i]),v2=v.map((x,i)=>x+.5*dt*a1[i]),a2=accel(q2,v2,t,tau),q3=q.map((x,i)=>x+.5*dt*v2[i]),v3=v.map((x,i)=>x+.5*dt*a2[i]),a3=accel(q3,v3,tau),q4=q.map((x,i)=>x+.5*dt*v3[i]),v4=v.map((x,i)=>x+.5*dt*a3[i]),a4=accel(q4,v4,tau);
  s.q=q.map((x,i)=>x+dt*(v[i]+2*v2[i]+2*v3[i]+v4[i])/6);
  s.qd=v.map((x,i)=>x+dt*(a1[i]+2*a2[i]+2*a3[i]+a4[i])/6);
  s.t+=dt;s.torque=tau.slice();
  if(!s.q.every(Number.isFinite)||!s.qd.every(Number.isFinite)){numericalFaults++;s.q=s.q.map(v2=>finiteNum(v2,POS_LIMIT*2));s.qd=s.qd.map(v2=>finiteNum(v2,0));s.done=true;return}
  s.done=s.q.some(v2=>Math.abs(v2)>POS_LIMIT)||s.qd.some(v2=>Math.abs(v2)>VEL_LIMIT)||s.t>MAXT;
}

function reward(s,tau){const qc=s.q.reduce((a,v)=>a+v*v,0)/3,vc=s.qd.reduce((a,v)=>a+v*v,0)/3,uc=tau.reduce((a,v)=>a+v*v,0)/(3*MAX_TORQUE*MAX_TORQUE);let r=1-3.2*qc-.032*vc-.06*uc;if(Math.max(...s.q.map(Math.abs))<POS_LIMIT*.089)r+=.55;return clamp(r,-4,2)}
function teacherAction(s){const q=s.q,v=s.qd;return[clamp(-(42*q[0]+10*v[0]+10*q[1]+2.5*v[1]+6*q[2]+1.2*v[2]),-MAX_TORQUE,MAX_TORQUE),clamp(-(36*q[1]+9*v[1]+8*q[0]+2*v[0]+7*q[2]+1.8*v[2]),-MAX_TORQUE,MAX_TORQUE),clamp(-(28*q[2]+7*v[2]+7*q[1]+1.6*v[1]),-MAX_TORQUE,MAX_TORQUE)]}
function teacherMatch(a,t){return 1-(Math.abs(a[0]-t[0])+Math.abs(a[1]-t[1])+Math.abs(a[2]-t[2]))/(6*MAX_TORQUE)}

```

```

function classProbs(brain,i,z,temp){const b=(brain.fireBias?.[i]||[0,0,0]).map(v=>finiteNum(v,0));const gain=clamp(finiteNum(brain.fireGain?.[i],.9),.05,4),T=Math.max(.05,finiteNum(temp,1)),zz=clamp(finiteNum(z,0),-12,12),drive=gain*zz/T,raw=[b[0]-drive,b[1],b[2]+drive],m=Math.max(...raw),e=raw.map(v=>Math.exp(clamp(v-m,-18,18))),d=Math.max(1e-12,e[0]+e[1]+e[2]);return[e[0]/d,e[1]/d,e[2]/d]}
function deterministicForward(brain,s,prev){
  const f=encode(s),h=[],z=[],p=[];
  for(let i=0;i<brain.N;i++){
    let q=finiteNum(brain.bias?.[i],0);
    for(let k=0;k<3;k++){const src=brain.src[i][k];const
      pre=src<SENS?finiteNum(f[src],0):finiteNum(prev[src-SENS],0);q+=finiteNum(brain.w[i][k],0)*pre
    }
    z[i]=q;const pp=classProbs(brain,i,q,DECT);p[i]=pp;
    let im=1;if(pp[0]>pp[1]&&pp[0]>=pp[2])im=0;else if(pp[2]>pp[1]&&pp[2]>pp[0])im=2;
    h[i]=im-1;
  }
  const tor=[0,0,0];
  for(let j=0;j<3;j++){let sum=0;for(let i=0;i<brain.N;i++)sum+=brain.decoder[j][i]*h[i];tor[j]=Math.tanh(sum/Math.max(3,(brain.dnz?.[j]||brain.N*.12)*.55))*MAX_TORQUE}
  return{f,h,z,p,tor};
}

function makeCase(seed,hard=false){const s=freshState(PRNG(seed>>>0),hard);s.caseSeed=seed>>>0;return s}
let trainEpoch=0;
function buildTraining(){
  const base=(CFG.seed+trainEpoch*2654435761)>>>0;
  trainingPool=Array.from({length:CFG.trainCases},(_,i)=>makeCase((base+7919*(i+1))>>>0,i>=Math.floor(CFG.trainCases*.72)));
}
function buildSuites(){
  const base=CFG.seed;
  buildTraining();
  validationPool=Array.from({length:CFG.valTrials},(_,i)=>makeCase((base+1000003+i*104729)>>>0,i>=Math.floor(CFG.valTrials*.72)));
  holdoutPool=Array.from({length:CFG.valTrials},(_,i)=>makeCase((base+7700017+i*15485863)>>>0,i>=Math.floor(CFG.valTrials*.72)));
}
function perturbSchedule(caseSeed){const r=PRNG((caseSeed||1)>>>0);return{rng:r,next:1.0+r()*1.8}}
function applyScheduledPerturb(s,plan){
  const j=Math.floor(plan.rng()*3),sg=plan.rng()<.5?-1:1;
  s.qd[j]+=sg*VEL_LIMIT*(.15+plan.rng()*12)*CFG.perturbMul;
  plan.next=s.t+1.5+plan.rng()*2.0;s.impulse=1;
}

// One episode of the frozen deterministic policy, with behaviour descriptors collected on the way.
function runEpisode(brain,base,opts){
  const o=opts||{};perturbing=o.perturb!==false;
  let prev=Array(brain.N).fill(0),s=cloneState(base),total=0,t=0,steps=0;
  let eff=0,act=0,tempo=0,top=0,tot=0,match=0;
  const plan=perturbSchedule(s.caseSeed||1);
  while(!s.done&&t<MAXT){
    const fw=deterministicForward(brain,s,prev);prev=fw.h;
    if(o.match)match+=teacherMatch(fw.tor,teacherAction(s))*DT;
    physics(s,fw.tor,DT);
    total+=reward(s,fw.tor)*DT;t+=DT;steps++;
    const a0=Math.abs(fw.tor[0]),a1=Math.abs(fw.tor[1]),a2=Math.abs(fw.tor[2]),sum=a0+a1+a2;
    eff+=sum/(3*MAX_TORQUE);tot+=sum;top+=a2;
    let nz=0;for(let i=0;i<brain.N;i++)if(fw.h[i]!==0)nz++;
    act+=nz/brain.N;
    tempo+=(Math.abs(s.qd[0])+Math.abs(s.qd[1])+Math.abs(s.qd[2]))/(3*VEL_LIMIT);
    if(perturbing&&s.t>=plan.next)applyScheduledPerturb(s,plan);
  }
  const n=Math.max(1,steps);
  return{duration:s.t,total,match:match/Math.max(t,DT),effort:eff/n,activity:act/n,tempo:tempo/n,topShare:tot>1e-9?tot/tot:1/3,last:s};
}
function scoreSet(brain,suite,opts){
  sanitizeBrain(brain);
  const durations=[];let total=0,eff=0,act=0,tempo=0,top=0,match=0;
  for(let i=0;i<suite.length;i++){
    const ep=runEpisode(brain,suite[i],opts);
    durations.push(ep.duration);total+=ep.total;eff+=ep.effort;act+=ep.activity;tempo+=ep.tempo;top+=ep.topShare;match+=ep.match;
  }
  const n=Math.max(1,durations.length),sorted=durations.slice().sort((a,b)=>a-b);
  const mean=sorted.reduce((a,b)=>a+b,0)/n;
  const median=n%2?sorted[(n-1)>>1]:.5*(sorted[n/2-1]+sorted[n/2]);
  const worst=sorted[0]||0;
  const survival=sorted.filter(v=>v>=SURV).length/n;
  const sc=Math.max(.2,MAXT/10);
  // Robust score deliberately rewards broad survival rather than one long run; time terms are
  // normalised by the episode cap so scores stay comparable when you change episode length.
  const robust=(.55*mean+.25*median+.20*worst)/sc+3*survival;
  return{mean,median,worst,survival,total,robust,durations:sorted,teacherMatch:match/n,
    desc:{effort:clamp(eff/n,0,1),activity:clamp(act/n,0,1),tempo:clamp(tempo/n,0,1),topShare:clamp(top/n,0,1)}};
}

```

```

}
const scoreSuite=(brain,suite)=>scoreSet(brain,suite,{});
function fitness(brain){
  const ts=scoreSet(brain,trainingPool,{match:true});
  let f=ts.total+2*ts.mean+1.5*ts.worst+8*ts.survival;
  if(CFG.novelty>0&&!archive.has(cellKey(binOf(ts.desc))))f+=CFG.novelty;
  return Object.assign({fitness:f},ts);
}
function scoreReference(suite){
  const durations=[];let survive=0;
  for(const c of suite){
    let s=cloneState(c),t=0;const plan=perturbSchedule(s.caseSeed||1);
    while(!s.done&&t<MAXT){physics(s,teacherAction(s),DT);t+=DT;if(s.t>=plan.next)applyScheduledPerturb(s,plan)}
    durations.push(s.t);if(s.t>=SURV)survive++;
  }
  const n=Math.max(1,durations.length),sorted=durations.slice().sort((a,b)=>a-b);
  return{mean:sorted.reduce((a,b)=>a+b,0)/n,worst:sorted[0]||0,survival:survive/n};
}

// Most of an undifferentiated population is read by nothing, so a uniformly random mutation is
// usually invisible to the actuators. Bias the target toward neurons the decoder actually reads
// (and their direct inputs), and bias rewiring toward the sensor block, which is only 9% of the
// source space by uniform draw.
function relevantSet(b){
  const read=new Set();
  for(let j=0;j<3;j++)for(let i=0;i<b.N;i++){if(b.decoder[j][i])read.add(i);
  const deep=new Set(read);
  for(const i of read)for(let k=0;k<3;k++){const sc=b.src[i][k];if(sc>=SENS)deep.add(sc-SENS)}
  return[...deep];
}
function mutate(parent,strong=false){
  const c=deepCloneNet(parent),r=rng||PRNG(1),count=CFG.mut;
  const rel=relevantSet(c),focus=CFG.focus;
  const target=()=>rel.length&&r()<focus?rel[Math.floor(r()*rel.length)]:Math.floor(r()*c.N);
  const source=()=>r()<CFG.sensorBias?Math.floor(r()*SENS):Math.floor(r()*(SENS+c.N));
  for(let z=0;z<count;z++){
    const i=target(),k=Math.floor(r()*3),u=r();
    if(u<.26){
      c.w[i][k]=otherTern(r,c.w[i][k]); // local weight flip
    }else if(u<.50){
      c.src[i][k]=source(); // rewire, keep the sign
    }else if(u<.62){
      c.bias[i]=clamp((c.bias[i]||0)+(r()<.5?-1:1)*.25,-2.5,2.5); // small drive offset
    }else if(u<.72){
      const ci=Math.floor(r()*3);
      c.fireBias[i][ci]=clamp(c.fireBias[i][ci]+(r()<.5?-1:1)*.3,-5,5); // firing threshold / zero band
    }else if(u<.78){
      c.fireGain[i]=clamp(c.fireGain[i]*(r()<.5?.82:1.22),.08,3.5); // sensitivity
    }else{
      const j=Math.floor(r()*3),ni=r()<focus&&rel.length?rel[Math.floor(r()*rel.length)]:Math.floor(r()*c.N);
      c.decoder[j][ni]=otherTern(r,c.decoder[j][ni]); // actuator wiring
    }
  }
  if(strong){
    const extra=Math.max(2,Math.round(count*1.5));
    for(let z=0;z<extra;z++){const i=target(),k=Math.floor(r()*3);c.src[i][k]=source();c.w[i][k]=choice(r,[-1,0,1])}
  }
  return sanitizeBrain(c);
}
function crossover(a,b){
  if(a.N!=b.N)return deepCloneNet(a);
  const c=deepCloneNet(a),r=rng,cut=Math.floor(r()*c.N),len=Math.max(1,Math.floor(c.N*(.2+r)*.4));
  for(let x=0;x<len;x++){
    const i=(cut+x)%c.N;
    c.src[i]=b.src[i].slice();c.w[i]=b.w[i].slice();c.bias[i]=b.bias[i];
    c.fireBias[i]=b.fireBias[i].slice();c.fireGain[i]=b.fireGain[i];
    for(let j=0;j<3;j++)c.decoder[j][i]=b.decoder[j][i];
  }
  return sanitizeBrain(c);
}
function genomeDistance(a,b){
  if(a.N!=b.N)return 1;
  let diff=0,tot=0;
  for(let i=0;i<a.N;i++)for(let k=0;k<3;k++){tot+=2;if(a.src[i][k]!=b.src[i][k])diff++;if(a.w[i][k]!=b.w[i][k])diff++;}
  for(let j=0;j<3;j++)for(let i=0;i<a.N;i++){tot++;if(a.decoder[j][i]!=b.decoder[j][i])diff++;}
  return diff/Math.max(1,tot);
}

// ---- diversity archive: one elite per behaviour cell (MAP-Elites) ----
// Descriptors never use the whole 0-1 range: every brain that can balance at all ends up with a
// similar control effort. So the axes track the range actually observed and rebin when it widens,
// which costs nothing because descriptors are stored alongside each elite.
const AXES=['effort','activity','topShare'];
const cellKey=cell=>cell.join('|');

```

```

let axisRange={effort:[.30,.55],activity:[.35,.65],topShare:[.20,.45]};
function resetAxes(){axisRange={effort:[.30,.55],activity:[.35,.65],topShare:[.20,.45]}}
function noteDesc(d){
  let widened=false;
  for(const k of AXES){
    const v=clamp(finiteNum(d[k],0),0,1),r=axisRange[k];
    if(v<r[0]-1e-9){r[0]=v;widened=true}
    if(v>r[1]+1e-9){r[1]=v;widened=true}
  }
  return widened;
}
function binOf(desc){
  const b=CFG.bins;
  return AXES.map(k=>{
    const [lo,hi]=axisRange[k],span=Math.max(1e-6,hi-lo);
    return Math.min(b-1,Math.max(0,Math.floor((clamp(finiteNum(desc[k],0),lo,hi)-lo)/span*b*.999999))));
  });
}
function rebinArchive(){
  const e=[...archive.values()];archive=new Map();
  for(const x of e){
    x.cell=binOf(x.validation.desc);
    const k=cellKey(x.cell),cur=archive.get(k);
    if(!cur||x.validation.robust>cur.validation.robust)archive.set(k,x);
  }
}
function tryArchive(brain,val){
  if(noteDesc(val.desc))rebinArchive();
  const cell=binOf(val.desc),key=cellKey(cell),cur=archive.get(key);
  if(!cur||val.robust>cur.validation.robust+1e-9){
    archive.set(key,{id:++championSerial,generation,brain:deepCloneNet(brain),validation:val,cell});
    pruneArchive();return cur?'improved':'new';
  }
  return false;
}
function pruneArchive(){
  if(archive.size<=CFG.archiveCap)return;
  const e=[...archive.entries()].sort((a,b)=>a[1].validation.robust-b[1].validation.robust);
  for(let i=0;i<archive.size-CFG.archiveCap;i++)archive.delete(e[i][0]);
}
function refreshPool(){
  const all=[...archive.values()].sort((a,b)=>b.validation.robust-a.validation.robust),picked=[];
  // Best first, but skip anything that is structurally a near-copy of a brain already picked, so
  // the pool holds different brains rather than eight variants of one lineage.
  for(const e of all){
    if(picked.length>=CFG.pool)break;
    if(CFG.minDist>0&&picked.some(p=>genomeDistance(p.brain,e.brain)<CFG.minDist))continue;
    picked.push(e);
  }
  for(const e of all){if(picked.length>=CFG.pool)break;if(!picked.includes(e))picked.push(e)}
  champions=picked;
  updateChampionUI();
  return champions.length;
}
function pickChampion(){if(!champions.length)return null;return
  champions[Math.min(champions.length-1,Math.floor(Math.pow(rng(),CFG.selExp)*champions.length))]}
function pickArchive(){if(!archive.size)return null;const a=[...archive.values()];return
  a[Math.floor(rng()*a.length)]}
function pickParent(){
  if(rng()*100<CFG.revisitPct){
    if(rng()*100<CFG.archivePct){const a=pickArchive();if(a)return{brain:a.brain,from:'archive #' +a.id}}
    const c=pickChampion();if(c)return{brain:c.brain,from:'champion #' +c.id};
  }
  return{brain:net,from:'current'};
}
function revisitChampion(){
  const c=(rng()*100<CFG.archivePct?pickArchive():null)||pickChampion();
  if(!c){setStatus('No champion exists yet. Train first.');

```

```

const d=hofDiversity();
${'championStatus'}.textContent=`Champions: ${champions.length} • top #${top.id} • robust
  ${top.validation.robust.toFixed(2)} • mean ${top.validation.mean.toFixed(2)} s • median
  ${top.validation.median.toFixed(2)} s • worst ${top.validation.worst.toFixed(2)} s • survival
  ${top.validation.survival*100}.toFixed(0)}% • pool genome spread min ${d.min*100}.toFixed(1)}% / mean
  ${d.mean*100}.toFixed(1)}%`;
}else ${'championStatus'}.textContent='Champions: 0';
const list=${'archiveList'};
const entries=[...champions.map(c=>['pool',c]),...[...archive.values()].sort((a,b)=>b.validation.robust-a.validation.robust).slice(0,10).map(c=>['cell',c])];
if(!entries.length){list.innerHTML=<span class="small">No champions yet.</span>;return}
const seen=new Set();
list.innerHTML=entries.filter(([,c])=>!seen.has(c.id)&&seen.add(c.id)).map(([,kind,c])=>{
  const v=c.validation,d=v.desc||{effort:0,activity:0,topShare:0};
  return `<span class="pill">${kind==='pool'?'pool':'cell'} #${c.id} • <b>${v.robust.toFixed(2)}</b> •
    ${v.mean.toFixed(1)}s • ${v.survival*100}.toFixed(0)}% • effort ${d.effort*100}.toFixed(0)}% • active
    ${d.activity*100}.toFixed(0)}% • top ${d.topShare*100}.toFixed(0)}%</span>`;
}).join('');
}

// ---- training loop ----
function trainGeneration(){
  if(!running||paused||mode!=='train')return;
  readCfg();
  if(!trainingPool.length)buildSuites();
  if(CFG.rotate>0&&generation%CFG.rotate===0){trainEpoch++;buildTraining()}
  const parentF=fitness(net);
  let winner=net,wF=parentF,tag='current (kept)',explorer=null,explorerF=-Infinity;
  for(let i=0;i<CFG.cand;i++){
    const p=pickParent();
    let base=p.brain,note=p.from;
    if(champions.length>1&&rng()*100<CFG.crossPct){
      const other=pickChampion();
      if(other&&other.brain.N===base.N&&other.brain!==base){base=crossover(base,other.brain);note+='#'+other.id}
    }
    const strong=CFG.strongEvery>0&&i===CFG.cand-1&&(generation%CFG.strongEvery)===CFG.strongEvery-1;
    const c=mutate(base,strong),f=fitness(c);
    // A candidate whose behaviour lands in an empty archive cell is worth validating even if it
    // loses this generation: that is how the archive fills with strategies the winner never tried.
    if(!archive.has(cellKey(binOf(f.desc)))&&f.fitness>explorerF){explorer=c;explorerF=f.fitness}
    if(f.fitness>wF.fitness){winner=c;wF=f;tag=note+(strong?' + strong jump':'')}
  }
  const improved=winner!==net;
  net=winner;generation++;lastParentTag=tag;
  if(improved)stagnation=0;else stagnation++;

  let v=null;
  if(generation%CFG.valEvery===0){
    v=scoreSuite(net,validationPool);
    const admitted=tryArchive(net,v);
    let probe='none';
    if(explorer){
      const pv=scoreSuite(explorer,validationPool),pa=tryArchive(explorer,pv);
      probe=`cell ${binOf(pv.desc).join('/')}` $pa?pa+ - kept': 'already better - dropped'`;
    }
    refreshPool();
    const inPool=champions.some(c=>c.generation===generation);
    lastAdmit=`winner ${admitted?'took a '+admitted+' cell': 'changed nothing'}` $inPool?, in pool:'' • probe
      ${probe}`;
    history.push({g:generation,d:v.mean,w:v.worst,s:v.survival});
    if(history.length>240)history.shift();
  }
  if(stagnation>=CFG.threshold&&revisitChampion())stagnation=0;

  const cov=coverage(),div=hofDiversity(),rec=net.src.flat().filter(x=>x>=SENS).length;
  ${'log'}.textContent=`generation ${generation}
  method: evolution with champions + diversity archive
  winning parent: ${lastParentTag}
  train fitness: ${wF.fitness.toFixed(4)} (mean ${wF.mean.toFixed(2)} s, worst ${wF.worst.toFixed(2)} s, survival
    ${wF.survival*100}.toFixed(0)}%)
  reference match: ${wF.teacherMatch*100}.toFixed(1)}%
  ${v?'validation mean: ${v.mean.toFixed(2)} s
  validation median: ${v.median.toFixed(2)} s
  validation worst: ${v.worst.toFixed(2)} s
  survival >= ${SURV.toFixed(1)} s: ${v.survival*100}.toFixed(1)}%
  robust score: ${v.robust.toFixed(3)}
  behaviour: effort ${(v.desc.effort*100).toFixed(0)}% • active ${(v.desc.activity*100).toFixed(0)}% • tempo
    ${(v.desc.tempo*100).toFixed(0)}% • top-joint ${(v.desc.topShare*100).toFixed(0)}%
  admission: ${lastAdmit}` validation: skipped (runs every ${CFG.valEvery} generations)`
  stagnation: ${stagnation}/${CFG.threshold}
  champions: ${champions.length}/${CFG.pool} pool genome spread: min ${d.min*100}.toFixed(1)}%
  archive: ${cov.cells} cells of ${cov.total} ${cov.pct.toFixed(1)}% coverage, cap ${CFG.archiveCap}
  feeds: ${CFG.revisitPct}% champion parents, ${CFG.archivePct}% of those from archive, ${CFG.crossPct}% crossover
  lane: ±${CFG.laneDeg.toFixed(0)}° angle, ±${VEL_LIMIT.toFixed(1)} rad/s, episode cap ${MAXT.toFixed(1)} s
  recurrent slots: ${rec}/${net.N*3}

```

```

weights ∈ {-1,0,+1}`;
drawHistory();drawArchive();updateUI();
trainTimer=setTimeout(trainGeneration,18);
}

function startTrain(keep){
clearTimeout(trainTimer);readCfg();
rng=PRNG(CFG.seed);
if(keep&&champions.length){net=mutate(champions[0].brain,true)}else{net=makeNet()}
generation=0;stagnation=0;history=[];numericalFaults=0;trainEpoch=0;
buildSuites();
if(!keep){champions=[];archive=new Map();championSerial=0;resetAxes()}
mode='train';paused=false;running=true;layoutCache=null;
$('networkCard').style.display='none';
setStatus(keep&&champions.length
? Continuing from champion #${champions[0].id}. Pool and archive stay as parent feeds; generation counter
restarts.`
: `Fresh champion-evolution run from seed ${CFG.seed}. Pool and archive cleared.`);
trainGeneration();updateUI();
}
function
pauseTrain(){if(mode==='train'&&!paused){paused=true;running=false;clearTimeout(trainTimer);setStatus(`Paused at
generation ${generation}. Test the frozen current brain or press Resume.`);updateUI()}}
function resumeTrain(){
if(mode==='test'&&state?.done){mode='train';paused=true;running=false;$('networkCard').style.display='none';state=
freshState(rng,false);setStatus('Returned from test. Exact same brain is frozen; press Resume to continue
training.');

```

```

state.qd[j]+=sg*VEL_LIMIT*(.15+rr()*.12)*Math.max(.25,CFG.perturbMul);
state.impulse=1;state._nextPerturb=state.t+1.5+rr()*2.0;
}
function audit(){
  let ok=true,tern=true,valid=true,finite=true;
  for(const row of net.src){if(row.length!==3)ok=false;for(const s of row){if(s<0||s>=SENS+net.N)valid=false}
  for(const row of net.w){for(const v of row){if(![-1,0,1].includes(v))tern=false;
  for(let j=0;j<3;j++)for(const v of net.decoder[j]){if(![-1,0,1].includes(v))tern=false;
  for(let i=0;i<net.N;i++)if(!Number.isFinite(net.bias[i])||!Number.isFinite(net.fireGain[i]))finite=false;
  const rec=net.src.flat().filter(x=>x>=SENS).length,pass=ok&&tern&&valid&&finite;
  $('auditOut').innerHTML+=`<b class="${pass?'ok':'badtext'}">${pass?'PASS':'FAIL'}</b> - ${net.N} protoneurons,
  ${net.N*3} incoming slots, every neuron has 3; recurrent ${rec} (${(100*rec/(net.N*3)).toFixed(1)}%); ternary
  weights and decoder ${tern?'yes':'no'}; thresholds finite ${finite?'yes':'no'}; lane
  ±${CFG.laneDeg.toFixed(0)}°.`;
  }
}
function loadTopChampion(){
  if(mode==='test'||(mode==='train'&&!paused)){setStatus('Pause training before loading a champion into the current
  brain.');
  if(!champions.length){setStatus('No champions to load yet.');
  const c=champions[0];net=deepCloneNet(c.brain);layoutCache=null;
  $('neurons').value=net.N;readCfg();
  setStatus(`Champion #${c.id} (generation ${c.generation}, robust ${c.validation.robust.toFixed(2)}) is now the
  current brain. Test or evaluate it directly.`);
  audit();updateUI();
}
function rescoreArchive(){
  if(busy)return;
  if(mode==='train'&&!paused){setStatus('Pause training before rescoring.');
  busy=true;setStatus('Rescoring and rebinning every champion under the current settings...');
  setTimeout(()=>{
    readCfg();buildSuites();
    const entries=[...archive.values()];
    for(const e of entries)e.validation=scoreSuite(e.brain,validationPool);
    resetAxes();for(const e of entries)noteDesc(e.validation.desc);
    archive=new Map();
    for(const e of entries){
      e.cell=binOf(e.validation.desc);
      const k=cellKey(e.cell),cur=archive.get(k);
      if(!cur||e.validation.robust>cur.validation.robust)archive.set(k,e);
    }
    pruneArchive();
    refreshPool();
    const cov=coverage(),merged=entries.length-archive.size;
    setStatus(`Rescored ${entries.length} archive entries and re-derived a pool of ${champions.length}.${merged>0?`
    ${merged} entr${merged===1?'y':'ies'} rebinned into an occupied cell and only the better one was kept.`:''}
    ${cov.cells} cells occupied (${cov.pct.toFixed(1)}% coverage).`);
    drawArchive();updateUI();busy=false;
  },40);
}
function crossValidate(){
  if(busy)return;
  if(mode==='train'&&!paused){setStatus('Pause training before cross-validating.');
  if(!champions.length&&!archive.size){setStatus('No champions to cross-validate yet.');
  busy=true;setStatus('Cross-validating champions on the holdout suite...');
  setTimeout(()=>{
    readCfg();buildSuites();
    const seen=new Set(),rows=[];
    const list=[...champions.map(c=>['pool',c]),...[...archive.values()].sort((a,b)=>b.validation.robust-a.validation.robust).slice(0,10).map(c=>['cell',c])];
    for(const [kind,c] of list){
      if(seen.has(c.id))continue;seen.add(c.id);
      const h=scoreSuite(c.brain,holdoutPool),d=c.validation.desc||{effort:0,activity:0,topShare:0};
      rows.push({kind,id:c.id,val:c.validation.robust,h,d,dist:champions.length?genomeDistance(c.brain,champions[0].brain):0});
    }
    rows.sort((a,b)=>b.h.robust-a.h.robust);
    const ref=scoreReference(holdoutPool);
    $('log').textContent=`CHAMPION CROSS-VALIDATION - holdout suite, ${holdoutPool.length} unseen cases
    =====
    src id   val.rob hold.rob mean worst surv effort active top dist
    ${rows.map(r=>`${r.kind.padEnd(4)} #${String(r.id).padEnd(4)} ${r.val.toFixed(2).padStart(7)}
    ${r.h.robust.toFixed(2).padStart(9)} ${r.h.mean.toFixed(2).padStart(6)}s ${r.h.worst.toFixed(2).padStart(6)}s
    ${r.h.survival*100).toFixed(0).padStart(4)}% ${r.d.effort*100).toFixed(0).padStart(6)}%
    ${r.d.activity*100).toFixed(0).padStart(6)}% ${r.d.topShare*100).toFixed(0).padStart(4)}%
    ${r.d.dist*100).toFixed(1).padStart(5)}%
    `).join('`');
    reference controller (hand-tuned, not evolved): mean ${ref.mean.toFixed(2)} s, worst ${ref.worst.toFixed(2)} s,
    survival ${ref.survival*100).toFixed(0)}%

    val.rob is the score the brain was archived with; hold.rob is on cases it has never seen.
    dist is genome distance from the top pool champion, so a spread of distances means the
    pool really is holding different brains rather than one brain eight times.`;
    setStatus(`Cross-validation finished on ${rows.length} champions. Best holdout robust
    ${rows[0]?rows[0].h.robust.toFixed(2):'-'}`);
    updateUI();busy=false;

```

```

    },40);
}

// ---- persistence: packed ternary genomes, browser storage with a text-box fallback ----
const T3='-0+',MEM={};
const packTern=arr=>arr.map(v=>T3[(v|0)+1]).join('');
const unpackTern=s=>[...s].map(ch=>Math.max(0,T3.indexOf(ch))-1);
function packBrain(b){return{N:b.N,src:b.src.flat(),w:packTern(b.w.flat()),dec:b.decoder.map(packTern),bias:b.bias.map(v=>v.toFixed(3)),fb:b.fireBias.flat().map(v=>v.toFixed(3)),fg:b.fireGain.map(v=>v.toFixed(3)),seed:b.seed|0}}
function unpackBrain(p){
  const N=p.N|0,src=[],w=[],fb=[],flatW=unpackTern(p.w|''),flatFb=p.fb|[];
  for(let i=0;i<N;i++){
    src.push([p.src[i*3]|0,p.src[i*3+1]|0,p.src[i*3+2]|0]);
    w.push([flatW[i*3]??0,flatW[i*3+1]??0,flatW[i*3+2]??0]);
    fb.push([finiteNum(flatFb[i*3],0),finiteNum(flatFb[i*3+1],0),finiteNum(flatFb[i*3+2],0)]);
  }
  const decoder=[0,1,2].map(j=>{const a=unpackTern((p.dec|[])[j]|'');while(a.length<N)a.push(0);return a.slice(0,N)});
  const bias=Array.from({length:N},(_,i)=>finiteNum((p.bias|[])[i],0));
  const fireGain=Array.from({length:N},(_,i)=>finiteNum((p.fg|[])[i],.9));
  return sanitizeBrain({N,src,w,decoder,bias,fireBias:fb,fireGain,seed:p.seed|0});
}
const slimVal=v=>({mean:+v.mean.toFixed(4),median:+v.median.toFixed(4),worst:+v.worst.toFixed(4),survival:+v.survival.toFixed(4),robust:+v.robust.toFixed(4),total:+v.total.toFixed(3),
  desc:{effort:+v.desc.effort.toFixed(4),activity:+v.desc.activity.toFixed(4),tempo:+v.desc.tempo.toFixed(4),topShare:+v.desc.topShare.toFixed(4)}});
const storeKey=()=>`protonlab-champ-v12-${CFG.seed}-${CFG.N}-${CFG.valTrials}-${CFG.laneDeg}-${CFG.bins}`;
function serializeAll(){
  return JSON.stringify({v:12,kind:'champion-evolution',key:storeKey(),generation,serial:championSerial,
    cfg:{seed:CFG.seed,N:CFG.N,laneDeg:CFG.laneDeg,veLLane:CFG.veLLane,torque:MAX_TORQUE,maxT:CFG.maxT,survS:SURV,bins:CFG.bins,valTrials:CFG.valTrials},
    axes:axisRange,
    champions:champions.map(c=>({id:c.id,generation:c.generation,validation:slimVal(c.validation),brain:packBrain(c.brain)})),
    archive:[...archive.values()].map(e=>({id:e.id,generation:e.generation,cell:e.cell,validation:slimVal(e.validation),brain:packBrain(e.brain)}))));
}
function deserializeAll(txt){
  const d=JSON.parse(txt);
  if(!d||d.v!==12)throw new Error('payload is not a v12 champion set');
  // Pool and archive entries both go into the archive map; the pool is then re-derived, so a
  // payload saved under a different bin count still lands in sensible cells.
  archive=new Map();const loaded=[...(d.archive|[]),...(d.champions|[])];
  // Saved axis ranges come back with the payload, otherwise two elites can rebin into the same
  // cell on load and one champion disappears.
  resetAxes();
  if(d.axes)for(const k of AXES){const r=d.axes[k];if(Array.isArray(r)&&r.length===2&&r.every(Number.isFinite)&&r[0]<=r[1])axisRange[k]=[clamp(r[0],0,1),clamp(r[1],0,1)]}
  for(const e of loaded)if(e.validation?.desc)noteDesc(e.validation.desc);
  for(const e of loaded){
    const entry={id:e.id|0,generation:e.generation|0,brain:unpackBrain(e.brain),validation:e.validation,cell:binOf(e.validation.desc)};
    const k=cellKey(entry.cell),cur=archive.get(k);
    if(!cur||entry.validation.robust>cur.validation.robust)archive.set(k,entry);
  }
  championSerial=Math.max(d.serial|0,...[...archive.values()].map(e=>e.id),0);
  pruneArchive();refreshPool();drawArchive();updateUI();
  return{champions:champions.length,cells:archive.size,cfg:d.cfg|{}};
}
async function storeSet(k,v){
  if(window.storage?.set){try{await window.storage.set(k,v);return'browser storage'}catch(_){}}
  try{localStorage.setItem(k,v);return'local storage'}catch(_){}
  MEM[k]=v;return'session memory only';
}
async function storeGet(k){
  if(window.storage?.get){try{const r=await window.storage.get(k);if(r&&r.value)return r.value}catch(_){}}
  try{const v=localStorage.getItem(k);if(v)return v}catch(_){}
  return MEM[k]||null;
}
async function saveStore(){
  readCfg();
  if(!champions.length&&!archive.size){$('ioStatus').textContent='Nothing to save yet – train until champions appear.';return}
  const txt=serializeAll(),where=await storeSet(storeKey(),txt);
  $('ioStatus').textContent=`Saved ${champions.length} pool champions and ${archive.size} archive cells to ${where} (${(txt.length/1024).toFixed(0)} KB) under key ${storeKey()}`;
}
async function loadStore(silent){
  readCfg();
  const txt=await storeGet(storeKey());
  if(!txt){if(!silent)$('ioStatus').textContent='No saved champions under key ${storeKey()}`;return}
  try{const r=deserializeAll(txt);$('ioStatus').textContent=`Loaded ${r.champions} pool champions and ${r.cells} archive cells saved at lane ±${r.cfg.laneDeg}°.`}
  catch(e){$('ioStatus').textContent='Could not read the saved payload: '+e.message}
}

```

```

}
function exportBox(){
  readCfg();
  if(!champions.length&&!archive.size){$('ioStatus').textContent='Nothing to export yet – train until champions
  appear.';return}
  const txt=serializeAll();$('ioBox').value=txt;
  $('ioStatus').textContent='Wrote ${champions.length} pool champions and ${archive.size} archive cells to the box
  (${txt.length/1024}.toFixed(0)) KB. Copy it somewhere safe.';
}
function importBox(){
  const txt=$('ioBox').value.trim();
  if(!txt){$('ioStatus').textContent='Paste a saved payload into the box first.';return}
  try{const r=deserializeAll(txt);$('ioStatus').textContent='Loaded ${r.champions} pool champions and ${r.cells}
  archive cells from the box. Rescore + rebin if your lane differs.'}
  catch(e){$('ioStatus').textContent='Could not read that payload: '+e.message}
}
async function copyBox(){
  if(!$('ioBox').value)exportBox();
  const txt=$('ioBox').value;if(!txt)return;
  try{await navigator.clipboard.writeText(txt);$('ioStatus').textContent='Payload copied to the clipboard.'}
  catch(_){
    let ok=false;
    try{const ta=$('ioBox');ta.select();ok=document.execCommand('copy')}catch(_){ok=false}
    $('ioStatus').textContent=ok?'Payload copied to the clipboard.': 'Clipboard unavailable – select the box contents
    and copy manually.';
  }
}
function downloadJson(){
  if(!$('ioBox').value)exportBox();
  const txt=$('ioBox').value;if(!txt)return;
  try{
    const b=new Blob([txt],{type:'application/json'}),u=URL.createObjectURL(b),a=document.createElement('a');
    a.href=u;a.download='protoneuron-champions-${CFG.seed}-${CFG.N}.json';document.body.appendChild(a);a.click();a.r
    emove();
    setTimeout(()=>URL.revokeObjectURL(u),2000);
    $('ioStatus').textContent='Download started.';
  }catch(_){$('ioStatus').textContent='Download blocked here – use Copy box instead.'}
}

function neutralBaseline(brain){
  const s={q:[0,0,0],qd:[0,0,0],t:0,torque:[0,0,0],done:false,impulse:0};
  const o=deterministicForward(brain,s,Array(brain.N).fill(0));
  const c={neg:0,zero:0,pos:0};for(const v of o.h){if(v<0)c.neg++;else if(v>0)c.pos++;else c.zero++;}
  return{active:c,torque:o.tor};
}

function biopsyText(){
  const rec=net.src.flat().filter(x=>x===SENS).length,wc={'-1':0,'0':0,'1':0};
  for(const row of net.w)for(const v of row)wc[String(v)]++;
  const active=state&&state._prev?state._prev:[];
  const ac={neg:active.filter(v=>v<0).length,zero:active.filter(v=>v===0).length,pos:active.filter(v=>v>0).length};
  const nb=neutralBaseline(net),cov=coverage(),div=hofDiversity();
  const a={'PROTONEURON CONNECTOME BIOPSY v12 – champion evolution','timestamp':new Date().toISOString(),'
  'mode':mode,'generation':generation,'neurons':net.N,'incomingSlots':net.N*3,'recurrentSlots':rec,'
  recurrentPct':(100*rec/(net.N*3)).toFixed(1)}%,
  'laneAngleDeg':CFG.laneDeg.toFixed(1),'laneAngleRad':POS_LIMIT.toFixed(4),'laneVelocity':VEL_LIMIT.toFixed
  (2),'maxTorque':MAX_TORQUE.toFixed(2),'decoderDensityInit':(CFG.decDensity*100).toFixed(1)}%, 'decoderF
  anIn':(net.dnz||[]).join('/'),'mutateReadNeuronsPct':(CFG.focus*100).toFixed(0),'rewireToSensorsPct':(C
  CFG.sensorBias*100).toFixed(0),'mutationsPerCandidate':CFG.mut,'candidatesPerGen':CFG.cand,'trainingC
  ases':trainingPool.length,'rotateCasesEvery':CFG.rotate,'trainEpoch':trainEpoch,'episodeCap':MAXT.t
  oFixed(2)}s,'survivalThreshold':SURV.toFixed(2)}s,
  'startSpreadDeg':CFG.startDeg.toFixed(2),'startVel':CFG.startVel.toFixed(2),'hardCaseMultiplier':CFG.hardM
  ul.toFixed(2),'perturbMultiplier':CFG.perturbMul.toFixed(2),'decisionTemp':DECT.toFixed(3)},
  'bestMeanValidation':bestMean.toFixed(3)}s,'bestWorstValidation':bestWorst.toFixed(3)}s,'bestSurvival':((best
  Survival*100).toFixed(1)}%,
  'weightCounts':-1:{wc['-1']} 0:{wc['0']} +1:{wc['1']},'active':-1:{ac.neg} 0:{ac.zero} +1:{ac.pos},
  'championPool':{champions.length}/CFG.pool,'poolGenomeSpreadMin':((div.min*100).toFixed(2)}%, 'poolGenomeSpre
  adMean':(div.mean*100).toFixed(2)}%,
  'archiveCells':cov.cells/cov.total,'axisRanges':effort:{axisRange.effort.map(v=>v.toFixed(3)).join('-')}
  activity:{axisRange.activity.map(v=>v.toFixed(3)).join('-')} topShare:{axisRange.topShare.map(v=>v.toFixed(
  3)).join('-')},'archiveCoverage':cov.pct.toFixed(2)}%, 'archiveCap':CFG.archiveCap,'binsPerAxis':CFG.bi
  ns,'minGenomeDistance':(CFG.minDist*100).toFixed(2)}%, 'noveltyBonus':CFG.novelty.toFixed(2)},
  'feedChampionPct':CFG.revisitPct,'feedArchivePct':CFG.archivePct,'crossoverPct':CFG.crossPct,'selectionE
  xponent':CFG.selExp,'revisitThreshold':CFG.threshold,'stagnation':stagnation},
  'validationTrials':validationPool.length,'holdoutTrials':holdoutPool.length,'validateEvery':CFG.valEvery}
  , 'testCase':state?(state._caseIndex??-1)+1:'n/a',
  'numericalFaults':numericalFaults,'deterministicTieBreak':neutral_zero,
  'neutralBaselineActive':-1:{nb.active.neg} 0:{nb.active.zero}
  +1:{nb.active.pos}, 'neutralBaselineTorque':nb.torque.map(v=>v.toFixed(4)).join(',')},
  'CHAMPION POOL id, generation, robust, mean, worst, survival, cell, descriptors'};
  for(const c of champions){const v=c.validation,d=v.desc;a.push(`#${c.id} gen=${c.generation}
  robust=${v.robust.toFixed(3)} mean=${v.mean.toFixed(3)}s worst=${v.worst.toFixed(3)}s
  surv=${(v.survival*100).toFixed(1)}% cell=${(c.cell||[]).join('/')}` effort=${d.effort.toFixed(3)}
  active=${d.activity.toFixed(3)} tempo=${d.tempo.toFixed(3)} topShare=${d.topShare.toFixed(3)}')
  a.push('','BEHAVIOUR ARCHIVE cell -> id, robust, descriptors');
  for(const [k,e] of [...archive.entries()].sort()){const d=e.validation.desc;a.push(`${k} -> #${e.id}

```

```

    robust=${e.validation.robust.toFixed(3)} effort=${d.effort.toFixed(3)} active=${d.activity.toFixed(3)}
    topShare=${d.topShare.toFixed(3)}))}
a.push('', 'FIRING THRESHOLDS (zero-drive reference, decision temperature '+DECT.toFixed(2)+'')');
for(let i=0; i<net.N; i++){const pp=classProbs(net, i, 0, DECT); a.push(' N${i+1}: P(-1)=$pp[0].toFixed(4)}
P(0)=$pp[1].toFixed(4)} P(+1)=$pp[2].toFixed(4)} gain=$net.fireGain[i].toFixed(4)}
bias=$net.bias[i].toFixed(4)}))}
a.push('', 'CONNECTOME target <- source, weight');
for(let i=0; i<net.N; i++){for(let k=0; k<3; k++){const s=net.src[i][k]; a.push(' N${i+1} <-
$S<SENS>`S${s+1}`: N${s-SENS+1}` $net.w[i][k]`)}
a.push('', 'ACTUATOR DECODER');
for(let j=0; j<3; j++){const d=$net.decoder[j].map((v,i)=>`N${i+1}:$v`).join(' ')});
if(mode==='test') a.push('', 'CURRENT ACTIVATIONS', active.map((v,i)=>`N${i+1}:$v`).join(' '));
return a.join('\n');
}
}
async function copyBiopsy(){
const txt=biopsyText();
try{await navigator.clipboard.writeText(txt); setStatus('Biopsy copied.')}
catch(_){
let ok=false;
try{const ta=document.createElement('textarea'); ta.value=txt; ta.style.position='fixed'; ta.style.left='-9999px'; d
ocument.body.appendChild(ta); ta.select(); ok=document.execCommand('copy'); ta.remove()} catch(_){ok=false}
setStatus(ok?'Biopsy copied.': 'Clipboard unavailable – the biopsy is in the log instead.')}
if(!ok)$( 'log').textContent=txt;
}
}

function updateUI(){
const cov=covage();
$('mode').textContent=mode; $('gen').textContent=generation; $('time').textContent=(state?.t||0).toFixed(2)+' s';
$('best').textContent=bestMean.toFixed(2)+' s'; $('worst').textContent=bestWorst.toFixed(2)+'
s'; $('survival').textContent=(bestSurvival*100).toFixed(0)+'%';
$('nCount').textContent=net.N; $('faults').textContent=numericalFaults;
$('cells').textContent=cov.cells; $('coverage').textContent=cov.pct.toFixed(0)+'%';
$('modeBadge').textContent=mode.toUpperCase()+ ' • lane ±$CFG.laneDeg.toFixed(0)}°';
$('pause').disabled=mode!=='train' || paused;
$('resume').disabled=!((mode==='train'&&paused) || (mode==='test'&&state?.done));
$('test').disabled=!((mode==='train'&&paused));
$('continue').disabled=!champions.length;
$('loadTop').disabled=!champions.length;
}

function setStatus(s){$('status').textContent=s}
function drawWorld(){
const c=wctx, W=world.width, H=world.height;
c.clearRect(0, 0, W, H);
c.strokeStyle='#29415f'; c.lineWidth=2; c.beginPath(); c.moveTo(25, H-45); c.lineTo(W-25, H-45); c.stroke();
const ox=W/2, oy=H-60, l=72;
c.save(); c.setLineDash([5, 8]); c.strokeStyle='#294a6d'; c.lineWidth=1.5;
for(const sgn of [-1, 1]){c.beginPath(); c.moveTo(ox, oy); c.lineTo(ox+3*l*Math.sin(sgn*POS_LIMIT), oy-3*l*Math.cos(sgn*
POS_LIMIT)); c.stroke()}
c.restore();
let x=ox, y=oy, pts=[[x, y]];
for(let i=0; i<3; i++){x+=l*Math.sin(state?.q?.[i]||0); y-=l*Math.cos(state?.q?.[i]||0); pts.push([x, y])}
c.strokeStyle='#53d9ff'; c.lineWidth=7; c.beginPath(); c.moveTo(ox, oy); for(let
i=1; i<pts.length; i++){c.lineTo(pts[i][0], pts[i][1]); c.stroke()}
for(let i=0; i<pts.length; i++){c.fillStyle=i===0?'#8aa6c7':Math.abs((state?.q?.[Math.min(2, i-1)]||0)<POS_LIMIT*.13
3?'#55e58b': '#ffd166'); c.beginPath(); c.arc(pts[i][0], pts[i][1], i===0?9:13, 0, Math.PI*2); c.fill()}
c.fillStyle='#eaf3ff'; c.font='15px Segoe UI';
c.fillText('q' $((state?.q||[0, 0, 0]).map(v=>(v*180/Math.PI).toFixed(1)+'°').join(' ')), 18, 27);
c.fillText('q■' $((state?.qd||[0, 0, 0]).map(v=>v.toFixed(2)).join(' ')), 18, 50);
c.fillText('τ' $((state?.torque||[0, 0, 0]).map(v=>v.toFixed(1)).join(' ')), 18, 73);
if(mode==='test'){c.fillStyle='#55e58b'; c.fillText('FROZEN CURRENT
TEST', W-205, 27); if(state.impulse){c.fillStyle='#ff6f7d'; c.fillText('PERTURBATION', W-175, 50)}}
else{c.fillStyle='#9fb1c8'; c.fillText(mode==='train'? (paused?'PAUSED': 'TRAINING'): 'IDLE', W-110, 27)}
}

function drawHistory(){
const svg=$( 'history'), W=860, H=260, p=34, ns='http://www.w3.org/2000/svg';
while(svg.firstChild)svg.removeChild(svg.firstChild);
const add=(t, attrs)=>{const e=document.createElementNS(ns, t); for(const [k, v] of
Object.entries(attrs))e.setAttribute(k, v); svg.appendChild(e); return e};
add('rect', {x:0, y:0, width:W, height:H, fill:'#081321'});
const max=Math.max(1, MAXT);
for(let i=0; i<5; i++){const y=p+i*(H-p*1.45)/4; add('line', {x1:p, y1:y, x2:W-12, y2:y, class:'gridline'}); const
tx=add('text', {x:5, y:y+4, fill:'#8fa4bd', 'font-size':11}); tx.textContent=((4-i)*max/4).toFixed(1)+'s';
add('line', {x1:p, y1:H-p, x2:W-12, y2:H-p, class:'axis'}); add('line', {x1:p, y1:p, x2:W-12, y2:H-p, class:'axis'});
if(!history.length){const t=add('text', {x:W/2-95, y:H/2, fill:'#9fb1c8', 'font-size':14}); t.textContent='No
validation samples yet.'; $('historyReadout').textContent='No samples yet.'; return}
const xa=i-p+(W-p-12)*(i/Math.max(1, history.length-1)), ya=v=>H-p-(clamp(v, 0, max)/max)*(H-p*1.4);
for(const [key, cls, col] of [['d', 'champ', '#ffd166'], ['w', 'evo', '#53d9ff'], ['s', 'bp', '#b9a0ff']]){
let d=''; history.forEach((h, i)=>{const v=key==='s'?h.s*max:h[key]; d+=i?' L ':'M ') +xa(i)+' '+ya(v)});
add('path', {d, class:cls});
if(history.length<90)for(let i=0; i<history.length; i++){const
v=key==='s'?history[i].s*max:history[i][key]; add('circle', {cx:xa(i), cy:ya(v), r:2.2, fill:col})}
}
const last=history[history.length-1];
const t=add('text', {x:W-245, y:22, fill:'#eaf3ff', 'font-size':12}); t.textContent=`last mean ${last.d.toFixed(2)} s`;

```

```

const r=add('text',{x:W-245,y:40,fill:'#9fb1c8','font-size':12});r.textContent=`samples ${history.length}`;
$('historyReadout').textContent=`${history.length} samples • latest mean ${last.d.toFixed(2)} s • worst
  ${last.w.toFixed(2)} s • survival ${last.s*100}.toFixed(0)}%`;
}
function drawArchive(){
const svg=$( 'archiveGrid' ),W=430,H=330,ns='http://www.w3.org/2000/svg',b=CFG.bins||5;
while(svg.firstChild)svg.removeChild(svg.firstChild);
const add=(t,attrs,text)=>{const e=document.createElementNS(ns,t);for(const[k,v]of
  Object.entries(attrs))e.setAttribute(k,v);if(text!==undefined)e.textContent=text;svg.appendChild(e);return e};
add('rect',{x:0,y:0,width:W,height:H,fill:'#081321'});
const px=56,py=18,pb=52,cw=(W-px-16)/b,ch=(H-py-pb)/b;
const cells=new Map();let mn=Infinity,mx=-Infinity;
for(const e of archive.values()){
  const key=e.cell[0]+'|'+e.cell[1];let c=cells.get(key);
  if(!c){c={count:0,best:-Infinity,top:null};cells.set(key,c)}
  c.count++;if(e.validation.robust>c.best){c.best=e.validation.robust;c.top=e}
  mn=Math.min(mn,e.validation.robust);mx=Math.max(mx,e.validation.robust);
}
for(let i=0;i<b;i++)for(let j=0;j<b;j++){
  const c=cells.get(i+'|'+j),x=px+i*cw,y=py+(b-1-j)*ch;
  const t=c&&mx>mn?(c.best-mn)/(mx-mn):c?1:0;
  add('rect',{x:x+1,y:y+1,width:cw-2,height:ch-2,rx:4,
    fill:c?'hsl((${(196-150*t).toFixed(0)},82%,${(26+26*t).toFixed(0)}%)`:'#0c1826',
    stroke:c?'#7fb6dd': '#1b3048','stroke-width':c?1:1,'fill-opacity':c?.95:1});
  if(c)add('text',{x:x+cw/2,y:y+ch/2+4,fill:'#06121d','font-size':Math.min(13,cw*.36),'text-anchor':'middle','font
    -weight':'700'},String(c.count));
}
add('line',{x1:px,y1:py,x2:px,y2:py+b*ch,class:'axis'});
add('line',{x1:px,y1:py+b*ch,x2:px+b*cw,y2:py+b*ch,class:'axis'});
const fr=k=>`${axisRange[k][0]*100}.toFixed(0)} → ${axisRange[k][1]*100}.toFixed(0)}%`;
add('text',{x:px+b*cw/2,y:H-30,fill:'#9fb1c8','font-size':12,'text-anchor':'middle','control effort
  ${fr('effort')}`});
const yl=add('text',{x:14,y:py+b*ch/2,fill:'#9fb1c8','font-size':12,'text-anchor':'middle','firing activity
  ${fr('activity')}`});
yl.setAttribute('transform','rotate(-90 14 ${py+b*ch/2})');
const cov=coverage();
add('text',{x:px,y:H-10,fill:'#eaf3ff','font-size':12},`${cov.cells} cells • ${cov.pct.toFixed(1)}% coverage •
  robust ${mn===Infinity?'':mn.toFixed(2)} → ${mx===-Infinity?'':mx.toFixed(2)} • top-joint axis
  ${fr('topShare')}`});
}
function nodeLayout(){const
  N=net.N,W=netview.width,H=netview.height,rad=Math.min(W,H)*.35,cx=W*.5,cy=H*.53,p=[];for(let i=0;i<N;i++){const
  a=(i/N)*Math.PI*2,rr=rad*(.72+.22*((i*17)%13)/13);p.push({x:cx+Math.cos(a)*rr,y:cy+Math.sin(a)*rr})}return p}
function drawNetwork(){
  if(mode!=='test')return;
  const
    pts=layoutCache&&layoutCache.length===net.N?layoutCache:(layoutCache=nodeLayout()),h=state?state._prev||[]:[];
  nctx.clearRect(0,0,netview.width,netview.height);
  for(let i=0;i<net.N;i++)for(let k=0;k<3;k++){
    const src=net.src[i][k],w=net.w[i][k];if(src<SENS)continue;
    const a=pts[src-SENS],bq=pts[i],col=w>0?'#55e58b':w<0?'#ff6f7d': '#42546c';
    nctx.strokeStyle=col;nctx.globalAlpha=w===0?.15:(h[src-SENS]===0?.75:.25);nctx.lineWidth=h[src-SENS]!==0?1.8:1;
    nctx.beginPath();nctx.moveTo(a.x,a.y);nctx.lineTo(bq.x,bq.y);nctx.stroke();
    if(w!==0&&h[src-SENS]!==0){const f=(performance.now()/420+i*.071)%1;nctx.globalAlpha=.95;nctx.fillStyle=col;nctx
      .beginPath();nctx.arc(a.x+(bq.x-a.x)*f,a.y+(bq.y-a.y)*f,2.4,0,Math.PI*2);nctx.fill()}
  }
  nctx.globalAlpha=1;
  for(let i=0;i<net.N;i++){
    const a=pts[i],v=h[i]||0;
    nctx.fillStyle=v>0?'#55e58b':v<0?'#ff6f7d': '#52647c';
    nctx.beginPath();nctx.arc(a.x,a.y,Math.min(5,Math.max(2.5,260/net.N)),0,Math.PI*2);nctx.fill();
    if(net.N<=170){nctx.fillStyle='#d9e7f7';nctx.font='8px Segoe UI';nctx.fillText(String(i+1),a.x+6,a.y+3)}
  }
  nctx.fillStyle='#eaf3ff';nctx.font='13px Segoe UI';
  nctx.fillText(`N=${net.N} • 3N=${net.N*3} incoming slots • ${net.src.flat().filter(x=>x>=SENS).length} recurrent •
    ternary activations`,18,24);
}
$('start').onclick=(=>startTrain(false);
$('continue').onclick=(=>startTrain(true);
$('pause').onclick=pauseTrain;$('resume').onclick=resumeTrain;
$('test').onclick=testCurrent;$('evalCurrent').onclick=evalCurrent;$('perturb').onclick=perturbNow;$('reset').oncl
  ick=resetAll;
$('crossval').onclick=crossValidate;$('audit').onclick=audit;$('copyBiopsy').onclick=copyBiopsy;$('rescore').oncl
  ick=rescoreArchive;
$('reseed').onclick=(=>{if(revisitChampion())updateUI()});
$('loadTop').onclick=loadTopChampion;
$('clearArchive').onclick=(=>{champions=[];archive=new Map();championSerial=0;resetAxes();bestMean=bestWorst=bestSu
  rival=0;updateChampionUI();drawArchive();updateUI();setStatus('Champion pool and behaviour archive cleared. The
    current brain is untouched.'});
$('saveStore').onclick=saveStore;$('loadStore').onclick=(=>loadStore(false);
$('exportBox').onclick=exportBox;$('importBox').onclick=importBox;$('copyBox').onclick=copyBox;$('downloadJson').onc
  lick=downloadJson;
for(const id of ['laneDeg','velLane','torque','maxT','survS','startDeg','startVel','hardMul','perturbMul','trainCase
  s','validationTrials'])

```

```

$(id).onchange={()=>{readCfg();buildSuites();drawHistory();drawArchive();updateUI();setStatus('Case suites rebuilt
for the new lane or episode settings. Existing champion scores came from the old settings – press Rescore +
rebin, or Reset for a clean run.')}};
for(const id of ['bins','archiveCap'])
  $(id).onchange={()=>{readCfg();pruneArchive();drawArchive();updateUI();setStatus('Archive geometry changed. Press
Rescore + rebin to re-place existing champions into the new cells.')}};
for(const id of ['poolSize','minDist'])
  $(id).onchange={()=>{readCfg();refreshPool();updateUI();setStatus('Pool re-derived from the archive:
${champions.length} champions at least ${CFG.minDist*100}.toFixed(1)}% apart.')}};
for(const id of ['threshold','revisitPct','archivePct','crossPct','selExp','novelty','mutations','candidates','strongEvery','valEvery','dect','focus','sensorBias','rotate'])
  $(id).onchange={()=>{readCfg();updateUI();setStatus('Search and feed settings applied live from the next
generation.')}};
$('decDensity').onchange={()=>setStatus('Decoder density changed. Press Reset to build a brain with the new actuator
fan-in.')}};
$('neurons').onchange={()=>setStatus('Neuron count changed. Press Reset to create the new-sized brain.')}};
$('seed').onchange={()=>setStatus('Seed changed. Press Reset to create the new brain and suites.')}};

function frame(now){
  const dt=Math.min(.05,(now-lastFrame)/1000);lastFrame=now;
  if(mode==='test'&&state&&state.done){
    if(state.t>=state._nextPerturb)perturbNow();
    const o=deterministicForward(net,state,state._prev||Array(net.N).fill(0));
    state._prev=o.h;state.torque=o.tor;physics(state,o.tor,DT);
    state.impulse=Math.max(0,state.impulse-dt*2.5);
    state._trace.push(o);if(state._trace.length>200)state._trace.shift();
    if(state.done)setStatus('Test ended at ${state.t.toFixed(2)} s on validation case ${state._caseIndex??0}+1.
    Frozen brain preserved; Resume is ready.'));
  }
  drawWorld();if(mode==='test')drawNetwork();updateUI();requestAnimationFrame(frame);
}

readCfg();rng=PRNG(CFG.seed);net=makeNet();buildSuites();state=freshState(rng,false);
audit();drawHistory();drawArchive();updateChampionUI();updateUI();requestAnimationFrame(frame);
loadStore(true);
</script>
</body></html>

```

J.2 walker3d.js — the 3D biped: constrained spherical-pendulum plant, foot placement with impulsive collision, cycled curriculum, evolution, ablation modes.

```

'use strict';
// 3D walker for the ternary protoneuron architecture.
//
//   node walker3d.js --mode selftest
//   node walker3d.js --mode reference
//   node walker3d.js --mode evolve --seconds 120 --neurons 24
//   node walker3d.js --mode ablate --seconds 120
//
// PLANT: point-mass torso on rigid massless legs with point feet, in 3D. During single
// support the mass is constrained to a sphere of radius L about the stance foot, so the
// state is a spherical inverted pendulum – no small-angle approximation, no singularity
// at the vertical. A step places the swing foot on the ground at the radius the current
// CoM height allows, then destroys the velocity component along the new leg: that
// impulsive collision is the only energy sink, and ankle torque is the only source, so a
// steady gait has to be a genuine limit cycle.
//
// The controller is identical in kind to the pendulum lab: every neuron has 3 incoming
// slots, weights and activations are ternary, and the only continuous parameters are the
// per-neuron bias, firing thresholds and gain.

const M = 1.0, G = 9.81, L = 0.9; // mass, gravity, leg length
const DT = 0.004; // contact-free but collisional, so keep it small
const TAU_MAX = 0.28 * M * G * L; // point feet cannot produce much ankle torque
const W0 = 0.085; // half stance width, anatomical
const MIN_STEP = 0.30; // realistic swing time: it must truly balance on one leg
const FORCE_LEAN = 0.52; // reflex step before a certain fall
const FALL_H = 0.55 * L; // CoM this low means fallen
const INJURY = 1.25; // collision impulse per unit mass, m/s
const VMAX = 4.0; // tau_x, tau_y, trigger, dir_x, dir_y
const ACT = 5;
const SENS = 14;
const DECT = 0.22;

const clamp = (x, a, b) => Math.max(a, Math.min(b, x));
const tri = x => (x > 0.33 ? 1 : x < -0.33 ? -1 : 0);
const finite = (x, f = 0) => (Number.isFinite(x) ? x : f);
function PRNG(seed) {
  let s = (seed >>> 0) || 1;
  return () => { s = (s + 0x6D2B79F5) >>> 0; let t = s; t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
}
const dot = (a, b) => a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
const sub = (a, b) => [a[0] - b[0], a[1] - b[1], a[2] - b[2]];

```

```

const norm = a => Math.sqrt(dot(a, a));

// ----- plant
function freshState(vTarget, r) {
  const lean = (r() - 0.5) * 0.06, latl = (r() - 0.5) * 0.04;
  const h = Math.sqrt(Math.max(0.01, L * L - lean * lean - latl * latl));
  return {
    p: [lean, latl, h], v: [0.15 * vTarget + (r() - 0.5) * 0.08, (r() - 0.5) * 0.06, 0],
    f: [0, 0, 0], side: r() < 0.5 ? 1 : -1, tStep: 0, t: 0,
    steps: 0, dist: 0, peakImpulse: 0, injured: false, done: false, fallen: false,
    vTarget, lastImpulse: 0, energy: 0,
  };
}

function cloneState(s) {
  const c = { ...s, p: s.p.slice(), v: s.v.slice(), f: s.f.slice() };
  c._pr = null; c._nextKick = 0; // rebuilt deterministically from caseSeed
  return c;
}

// Constrained acceleration: gravity plus ankle torque plus the rod's radial multiplier.
function accel(p, v, f, tau) {
  const r = sub(p, f);
  const Ft = [ (tau[1] * r[2] - 0) / (L * L), // (tau x r)/L^2 with tau_z = 0
    (0 - tau[0] * r[2]) / (L * L),
    (tau[0] * r[1] - tau[1] * r[0]) / (L * L) ];
  const S = [Ft[0], Ft[1], Ft[2]] - M * G;
  const lam = (-M * dot(v, v) - dot(r, S)) / L;
  return [ (S[0] + lam * r[0] / L) / M, (S[1] + lam * r[1] / L) / M, (S[2] + lam * r[2] / L) / M ];
}

function integrate(s, tau) {
  const p = s.p, v = s.v, f = s.f;
  const a1 = accel(p, v, f, tau);
  const p2 = p.map((x, i) => x + 0.5 * DT * v[i]), v2 = v.map((x, i) => x + 0.5 * DT * a1[i]);
  const a2 = accel(p2, v2, f, tau);
  const p3 = p.map((x, i) => x + 0.5 * DT * v2[i]), v3 = v.map((x, i) => x + 0.5 * DT * a2[i]);
  const a3 = accel(p3, v3, f, tau);
  const p4 = p.map((x, i) => x + DT * v3[i]), v4 = v.map((x, i) => x + DT * a3[i]);
  const a4 = accel(p4, v4, f, tau);
  const px = p.map((x, i) => x + DT * (v[i] + 2 * v2[i] + 2 * v3[i] + v4[i]) / 6);
  const vx = v.map((x, i) => x + DT * (a1[i] + 2 * a2[i] + 2 * a3[i] + a4[i]) / 6);
  // drift control: re-project onto the sphere and strip radial velocity
  let r = sub(px, f), rn = norm(r);
  if (!Number.isFinite(rn) || rn < 1e-6) { s.done = true; s.fallen = true; return; }
  const k = L / rn;
  s.p = [f[0] + r[0] * k, f[1] + r[1] * k, f[2] + r[2] * k];
  r = sub(s.p, f);
  const rh = [r[0] / L, r[1] / L, r[2] / L], vr = dot(vx, rh);
  s.v = [vx[0] - vr * rh[0], vx[1] - vr * rh[1], vx[2] - vr * rh[2]];
  s.t += DT; s.tStep += DT;
  s.energy += (tau[0] * tau[0] + tau[1] * tau[1]) / (TAU_MAX * TAU_MAX) * DT;
  if (!s.p.every(Number.isFinite) || !s.v.every(Number.isFinite)) { s.done = true; s.fallen = true; return; }
  if (s.p[2] < FALL_H) { s.done = true; s.fallen = true; }
  if (norm(s.v) > VMAX) { s.done = true; s.fallen = true; }
}

// Place the swing foot and take the collision loss.
function takeStep(s, dirx, diry) {
  const newSide = -s.side;
  const h = clamp(s.p[2], 0.05, L - 1e-4);
  const rad = Math.sqrt(Math.max(1e-6, L * L - h * h));
  let dx = clamp(dirx, -1, 1), dy = clamp(diry, -1, 1) + newSide * 0.75;
  const dn = Math.hypot(dx, dy) || 1;
  dx /= dn; dy /= dn;
  const f2 = [s.p[0] + rad * dx, s.p[1] + rad * dy, 0];
  const r2 = sub(s.p, f2), r2n = norm(r2);
  if (!Number.isFinite(r2n) || r2n < 1e-6) { s.done = true; s.fallen = true; return; }
  const rh = [r2[0] / r2n, r2[1] / r2n, r2[2] / r2n];
  const vr = dot(s.v, rh);
  s.lastImpulse = Math.abs(vr);
  s.peakImpulse = Math.max(s.peakImpulse, s.lastImpulse);
  if (s.lastImpulse > INJURY) { s.injured = true; s.done = true; return; }
  s.v = [s.v[0] - vr * rh[0], s.v[1] - vr * rh[1], s.v[2] - vr * rh[2]];
  s.f = f2; s.side = newSide; s.tStep = 0; s.steps++;
}

// ----- sensing
function encode(s) {
  const r = sub(s.p, s.f), vs = 1.0;
  return [
    tri(r[0] / (0.30 * L)), tri(r[1] / (0.30 * L)), // lean fore-aft, lateral
    tri(r[0] / (0.12 * L)), tri(r[1] / (0.12 * L)), // same, finer band
    tri(s.v[0] / vs), tri(s.v[1] / vs), tri(s.v[2] / vs),
    tri((s.v[0] - s.vTarget) / 0.35), // speed error
    tri((s.p[2] - 0.95 * L) / (0.08 * L)), // height error
    tri(s.side), // which foot is down
    tri((r[0] + s.v[0] * Math.sqrt(L / G)) / (0.30 * L)), // capture point fore-aft
    tri((r[1] + s.v[1] * Math.sqrt(L / G)) / (0.30 * L)), // capture point lateral
  ];
}

```

```

    tri(s.side * r[1] / (0.15 * L)), // lean toward the stance side
    1,
  ];
}

// ----- brain
function makeBrain(N, rng, decDensity) {
  const total = SENS + N, src = [], w = [];
  for (let i = 0; i < N; i++) {
    src.push([Math.floor(rng() * total), Math.floor(rng() * total), Math.floor(rng() * total)]);
    w.push([Math.floor(rng() * 3) - 1, Math.floor(rng() * 3) - 1, Math.floor(rng() * 3) - 1]);
  }
  const decoder = Array.from({ length: ACT }, () =>
    Array.from({ length: N }, () => (rng() < decDensity ? (rng() < 0.5 ? -1 : 1) : 0)));
  return sanitize({ N, src, w, decoder, bias: Array(N).fill(0),
    fireBias: Array.from({ length: N }, () => [0, 0, 0]), fireGain: Array(N).fill(0.9) });
}

function sanitize(b) {
  for (let i = 0; i < b.N; i++) {
    for (let k = 0; k < 3; k++) {
      if (![-1, 0, 1].includes(b.w[i][k])) b.w[i][k] = 0;
      b.src[i][k] = clamp(b.src[i][k] | 0, 0, SENS + b.N - 1);
      b.fireBias[i][k] = clamp(finite(b.fireBias[i][k]), -5, 5);
    }
    b.fireGain[i] = clamp(finite(b.fireGain[i], 0.9), 0.08, 3.5);
    b.bias[i] = clamp(finite(b.bias[i]), -2.5, 2.5);
  }
  b.dnz = [];
  for (let j = 0; j < ACT; j++) {
    let nz = 0;
    for (let i = 0; i < b.N; i++) { if (![-1, 0, 1].includes(b.decoder[j][i])) b.decoder[j][i] = 0; if
      (b.decoder[j][i]) nz++; }
    b.dnz[j] = nz;
  }
  return b;
}

const cloneBrain = b => sanitize({ N: b.N, src: b.src.map(r => r.slice()), w: b.w.map(r => r.slice()),
  decoder: b.decoder.map(r => r.slice()), bias: b.bias.slice(),
  fireBias: b.fireBias.map(r => r.slice()), fireGain: b.fireGain.slice() });

function forward(b, s, prev) {
  const f = encode(s), h = new Array(b.N);
  for (let i = 0; i < b.N; i++) {
    let z = b.bias[i];
    for (let k = 0; k < 3; k++) {
      const sc = b.src[i][k];
      z += b.w[i][k] * (sc < SENS ? f[sc] : prev[sc - SENS]);
    }
    // argmax over the three affine pieces; identical decision to the lab's softmax
    const d = b.fireGain[i] * clamp(z, -12, 12) / DECT;
    const r0 = b.fireBias[i][0] - d, r1 = b.fireBias[i][1], r2 = b.fireBias[i][2] + d;
    h[i] = (r0 > r1 && r0 >= r2) ? -1 : (r2 > r1 && r2 > r0) ? 1 : 0;
  }
  const out = new Array(ACT);
  for (let j = 0; j < ACT; j++) {
    let sum = 0;
    for (let i = 0; i < b.N; i++) sum += b.decoder[j][i] * h[i];
    out[j] = Math.tanh(sum / Math.max(3, (b.dnz[j] || b.N * 0.12) * 0.55));
  }
  return { h, out };
}

// ----- curriculum
// Cycled: the trainer keeps a frontier stage but always re-draws a share of cases from
// every earlier stage, so progress on walking cannot quietly destroy standing.
const STAGES = [
  { name: 'stand', vTarget: 0.00, T: 4.0, perturb: 0.0, pass: 0.75 },
  { name: 'sway', vTarget: 0.00, T: 5.0, perturb: 0.5, pass: 0.70 },
  { name: 'creep', vTarget: 0.25, T: 6.0, perturb: 0.0, pass: 0.60 },
  { name: 'walk slow', vTarget: 0.50, T: 8.0, perturb: 0.3, pass: 0.55 },
  { name: 'walk', vTarget: 0.80, T: 10.0, perturb: 0.6, pass: 0.50 },
];

function caseFor(stage, seed) {
  const r = PRNG(seed >>> 0), st = STAGES[stage];
  const s = freshState(st.vTarget, r);
  s.caseSeed = seed >>> 0; s.stage = stage; s.T = st.T; s.perturb = st.perturb;
  return s;
}

function buildSuite(frontier, seedBase, n, rng) {
  const out = [];
  for (let i = 0; i < n; i++) {
    // 55% at the frontier, the rest spread back over everything already learned
    const stage = (rng() < 0.55 || frontier === 0) ? frontier : Math.floor(rng() * frontier);
    out.push(caseFor(stage, (seedBase + i * 104729 + stage * 7919) >>> 0));
  }
}

```

```

    return out;
}

// ----- episode
// One control step. Shared by the evaluator and the viewer so what you watch is exactly
// what was scored.
function controlStep(brain, s, prev, policy) {
    const st = STAGES[s.stage];
    if (!s._pr) { s._pr = PRNG((s.caseSeed || 1) + 12345); s._nextKick = 1.0 + s._pr() * 1.5; }
    const pr = s._pr;
    let rew = 0, stepped = false;
    {
        let tau = [0, 0], trig = -1, dirx = 0, diry = 0;
        if (policy) {
            const a = policy(s);
            tau = [a[0] * TAU_MAX, a[1] * TAU_MAX]; trig = a[2]; dirx = a[3]; diry = a[4];
        } else {
            const o = forward(brain, s, prev); prev = o.h;
            tau = [o.out[0] * TAU_MAX, o.out[1] * TAU_MAX];
            trig = o.out[2]; dirx = o.out[3]; diry = o.out[4];
        }
        s.lastTau = tau; s.lastCmd = [dirx, diry];
        integrate(s, tau);
        if (s.done) return { prev, rew, stepped };
        const r = sub(s.p, s.f);
        const lean2 = (r[0] * r[0] + r[1] * r[1]) / (L * L);
        const vErr = s.v[0] - s.vTarget;
        rew += (1.0 + 2.2 * s.v[0] - 1.6 * vErr * vErr - 3.0 * lean2
            - 1.2 * (r[1] / L) * (r[1] / L) - 0.04 * (tau[0] * tau[0] + tau[1] * tau[1]) / (TAU_MAX * TAU_MAX)) *
            DT;
        const lean = Math.hypot(r[0], r[1]) / L;
        // The reflex step must also respect a swing time, or a leaning walker re-plants its
        // foot every timestep and skates without ever balancing on one leg.
        const canStep = s.tStep > MIN_STEP, canReflex = s.tStep > 0.5 * MIN_STEP;
        if ((trig > 0.25 && canStep) || (lean > FORCE_LEAN && canReflex)) {
            takeStep(s, dirx, diry);
            stepped = true;
            if (s.injured) { rew -= 6; return { prev, rew, stepped }; }
            rew -= 0.9 * s.lastImpulse * s.lastImpulse;
        }
        if (st.perturb > 0 && s.t >= s._nextKick) {
            const ang = pr() * Math.PI * 2, mag = st.perturb * (0.20 + pr() * 0.25);
            s.v[0] += Math.cos(ang) * mag; s.v[1] += Math.sin(ang) * mag;
            s._nextKick = s.t + 1.2 + pr() * 1.8;
        }
    }
    return { prev, rew, stepped };
}

function runEpisode(brain, base, policy) {
    const s = cloneState(base);
    let prev = new Array(brain ? brain.N : 0).fill(0);
    let rew = 0;
    const x0 = s.p[0];
    while (!s.done && s.t < s.T) {
        const r = controlStep(brain, s, prev, policy);
        prev = r.prev; rew += r.rew;
    }
    s.dist = s.p[0] - x0;
    const frac = s.t / s.T;
    return { dist: s.dist, time: s.t, steps: s.steps, rew, injured: s.injured, fallen: s.fallen,
        peakImpulse: s.peakImpulse, energy: s.energy, frac,
        // stage score in [0,1]: survive the clock, and cover the distance the target implies
        // Standing rungs score pure survival. An earlier version also rewarded staying
        // near the origin, which punished the stepping the later rungs depend on, so the
        // curriculum fought itself at its first step.
        score: s.vTarget > 0
            ? Math.max(0, Math.min(1, 0.4 * frac + 0.6 * clamp(s.dist / Math.max(0.2, s.vTarget * s.T), 0, 1)))
            : Math.max(0, Math.min(1, frac - 0.25 * (s.injured ? 1 : 0))) };
}

function evaluate(brain, suite, policy) {
    let rew = 0, dist = 0, time = 0, steps = 0, inj = 0, score = 0, peak = 0;
    const byStage = {};
    for (const c of suite) {
        const e = runEpisode(brain, c, policy);
        rew += e.rew; dist += e.dist; time += e.time; steps += e.steps;
        inj += e.injured ? 1 : 0; score += e.score; peak = Math.max(peak, e.peakImpulse);
        (byStage[c.stage] = byStage[c.stage] || []).push(e.score);
    }
    const n = Math.max(1, suite.length);
    const stageScore = {};
    for (const k of Object.keys(byStage)) stageScore[k] = byStage[k].reduce((a, b) => a + b, 0) / byStage[k].length;
    return { rew: rew / n, dist: dist / n, time: time / n, steps: steps / n,
        injRate: inj / n, score: score / n, peak, stageScore };
}

```

```

const fitness = (brain, suite) => {
  const e = evaluate(brain, suite);
  return { f: e.rew + 3.0 * e.dist + 1.2 * e.score - 4.0 * e.injRate, ...e };
};

// ----- reference controller
// Capture-point foot placement plus ankle regulation. Exists to prove the plant is
// walkable before asking evolution to walk it.
function referenceController(s) {
  const r = sub(s.p, s.f), tc = Math.sqrt(L / G);
  const Fx = 1.6 * (s.vTarget - s.v[0]) - 0.8 * r[0] / L;
  const Fy = -3.2 * (r[1] / L) - 1.4 * s.v[1];
  const tauX = clamp(-L * Fy, -TAU_MAX, TAU_MAX) / TAU_MAX;
  const tauY = clamp(L * Fx, -TAU_MAX, TAU_MAX) / TAU_MAX;
  const cpx = r[0] + s.v[0] * tc, cpy = r[1] + s.v[1] * tc;
  const wantStep = s.tStep > 0.34 || cpx > 0.16 * L || Math.abs(cpy) > 0.22 * L;
  const dirx = clamp((s.v[0] * tc + 0.35 * (s.v[0] - s.vTarget)) / (0.4 * L), -1, 1);
  const diry = clamp((s.v[1] * tc) / (0.4 * L), -1, 1);
  return [tauX, tauY, wantStep ? 1 : -1, dirx, diry];
}

// ----- evolution
function relevant(b) {
  const read = new Set();
  for (let j = 0; j < ACT; j++) for (let i = 0; i < b.N; i++) if (b.decoder[j][i]) read.add(i);
  const deep = new Set(read);
  for (const i of read) for (let k = 0; k < 3; k++) { const sc = b.src[i][k]; if (sc >= SENS) deep.add(sc - SENS); }
  return [...deep];
}

function mutate(parent, rng, opts) {
  const c = cloneBrain(parent), rel = relevant(c);
  const other = cur => { const o = [-1, 0, 1].filter(v => v !== cur); return o[Math.floor(rng() * o.length)]; };
  const target = () => (rel.length && rng() < 0.85 ? rel[Math.floor(rng() * rel.length)] : Math.floor(rng() * c.N));
  const source = () => (rng() < 0.5 ? Math.floor(rng() * SENS) : Math.floor(rng() * (SENS + c.N)));
  for (let z = 0; z < opts.mut; z++) {
    const i = target(), k = Math.floor(rng() * 3), u = rng();
    if (u < 0.24) c.w[i][k] = other(c.w[i][k]);
    else if (u < 0.46) c.src[i][k] = source();
    else if (u < 0.58) c.bias[i] = clamp(c.bias[i] + (rng() < 0.5 ? -1 : 1) * 0.25, -2.5, 2.5);
    else if (u < 0.72 && !opts.noThresholds) { const ci = Math.floor(rng() * 3); c.fireBias[i][ci] =
      clamp(c.fireBias[i][ci] + (rng() < 0.5 ? -1 : 1) * 0.3, -5, 5); }
    else if (u < 0.80 && !opts.noThresholds) c.fireGain[i] = clamp(c.fireGain[i] * (rng() < 0.5 ? 0.82 : 1.22),
      0.08, 3.5);
    else { const j = Math.floor(rng() * ACT), ni = rng() < 0.85 && rel.length ? rel[Math.floor(rng() * rel.length)]
      : Math.floor(rng() * c.N); c.decoder[j][ni] = other(c.decoder[j][ni]); }
  }
  return sanitize(c);
}

function evolve(opts) {
  const rng = PRNG(opts.seed);
  let net = makeBrain(opts.neurons, rng, opts.decDensity);
  let frontier = 0, gen = 0, epoch = 0;
  let suite = buildSuite(frontier, opts.seed, opts.cases, rng);
  const t0 = Date.now();
  let best = null;
  const log = [];
  while ((Date.now() - t0) / 1000 < opts.seconds) {
    if (gen % opts.rotate === 0) { epoch++; suite = buildSuite(frontier, opts.seed + epoch * 2654435761, opts.cases,
      rng); }
    let wF = fitness(net, suite), winner = net;
    for (let i = 0; i < opts.cands; i++) {
      const c = mutate(net, rng, opts), f = fitness(c, suite);
      if (f.f > wF.f) { winner = c; wF = f; }
    }
    net = winner; gen++;
    if (gen % 8 === 0) {
      // promotion is judged on a fresh draw so it cannot be an artifact of one suite
      const probe = buildSuite(frontier, opts.seed + 777 + gen, Math.max(8, opts.cases), rng);
      const e = evaluate(net, probe);
      const fs = e.stageScore[frontier];
      if (fs !== undefined && fs >= STAGES[frontier].pass && frontier < STAGES.length - 1) {
        frontier++;
        log.push(`gen ${gen}: promoted to stage ${frontier} (${STAGES[frontier].name}) - ${STAGES[frontier - 1].name} scored ${fs.toFixed(2)}`);
      }
      if (!best || e.score > best.score) best = { brain: cloneBrain(net), score: e.score, gen, frontier, e };
    }
  }
  return { net, best, frontier, gen, log, secs: (Date.now() - t0) / 1000 };
}

// ----- exports / cli
const IS_NODE = typeof process !== 'undefined' && !(process.versions && process.versions.node);
const WALKER_API = { L, G, M, DT, TAU_MAX, MIN_STEP, INJURY, FALL_H, ACT, SENS, STAGES,
  PRNG, clamp, tri, sub, dot, norm, freshState, cloneState, accel, integrate, takeStep,
  encode, makeBrain, sanitize, cloneBrain, forward, caseFor, buildSuite, controlStep,

```

```

    runEpisode, evaluate, fitness, referenceController, relevant, mutate, evolve };
if (typeof module !== 'undefined' && module.exports) module.exports = WALKER_API;
if (typeof window !== 'undefined') window.WALKER = WALKER_API;

if (IS_NODE && typeof require !== 'undefined' && require.main === module) {
const argv = {};
for (let i = 2; i < process.argv.length; i += 2) argv[process.argv[i].replace(/^--/, '')] = process.argv[i + 1];
const opts = {
  mode: argv.mode || 'selftest', seed: +(argv.seed || 1000), neurons: +(argv.neurons || 24),
  cases: +(argv.cases || 10), cands: +(argv.cands || 8), mut: +(argv.mut || 3),
  rotate: +(argv.rotate || 20), seconds: +(argv.seconds || 60),
  decDensity: +(argv.decDensity || 0.5), noThresholds: argv.noThresholds === 'on',
};

function report(tag, e) {
  console.log(` ${tag.padEnd(22)} dist ${e.dist.toFixed(2)}m time ${e.time.toFixed(2)}s ` +
    `steps ${e.steps.toFixed(1)} score ${e.score.toFixed(3)} injured ${e.injRate * 100}.toFixed(0)}% ` +
    `peak impulse ${e.peak.toFixed(2)} `);
}

if (opts.mode === 'selftest') {
  console.log('PLANT SELF-TEST');
  const rng = PRNG(1);
  // 1. constraint: the leg must stay rigid
  let s = freshState(0, rng); let worst = 0;
  for (let i = 0; i < 400 && !s.done; i++) { integrate(s, [0.1 * TAU_MAX, -0.05 * TAU_MAX]); worst = Math.max(worst,
    Math.abs(norm(sub(s.p, s.f)) - L)); }
  console.log(` leg length error over 400 steps: ${worst.toExponential(2)} m (L = ${L})`);
  // 2. energy: no torque, no steps, should conserve
  s = freshState(0, PRNG(7)); s.p = [0.12, 0.05, Math.sqrt(L * L - 0.0169 - 0.0025)]; s.v = [0, 0, 0];
  const E0 = 0.5 * M * dot(s.v, s.v) + M * G * s.p[2];
  for (let i = 0; i < 300 && !s.done; i++) integrate(s, [0, 0]);
  const E1 = 0.5 * M * dot(s.v, s.v) + M * G * s.p[2];
  console.log(` energy drift, torque-free fall: ${((E1 - E0) / E0 * 100).toFixed(4)}%`);
  // 3. it must actually fall over when unactuated
  s = freshState(0, PRNG(3)); s.p = [0.1, 0, Math.sqrt(L * L - 0.01)];
  let tf = 0; while (!s.done && s.t < 5) { integrate(s, [0, 0]); tf = s.t; }
  console.log(` unactuated fall: ${s.fallen ? 'fell' : 'still up'} at t = ${tf.toFixed(2)}s`);
  // 4. a collision must lose energy
  s = freshState(0.5, PRNG(9)); s.v = [0.6, 0, 0];
  const before = 0.5 * dot(s.v, s.v);
  takeStep(s, 0.4, 0);
  console.log(` collision: impulse ${s.lastImpulse.toFixed(3)} m/s, kinetic energy ${before.toFixed(3)} -> ${(0.5 *
    dot(s.v, s.v)).toFixed(3)} `);
  console.log(` injury cap ${INJURY} m/s, fall height ${FALL_H.toFixed(2)}m, tau_max ${TAU_MAX.toFixed(3)} N*m`);
} else if (opts.mode === 'reference') {
  console.log('REFERENCE CONTROLLER (hand-tuned, not evolved) - is the plant walkable?');
  const rng = PRNG(opts.seed);
  for (let st = 0; st < STAGES.length; st++) {
    const suite = Array.from({ length: 12 }, (_, i) => caseFor(st, (opts.seed + i * 104729) >>> 0));
    report(`stage ${st} ${STAGES[st].name}`, evaluate(null, suite, referenceController));
  }
  void rng;
} else if (opts.mode === 'evolve' || opts.mode === 'ablate') {
  console.log(`EVOLVING: N=${opts.neurons}, ${opts.seconds}s budget, ternary weights and activations` +
    (opts.noThresholds ? ', THRESHOLD MUTATIONS DISABLED' : ''));
  const r = evolve(opts);
  console.log(` ${r.gen} generations in ${r.secs.toFixed(0)}s, reached stage ${r.frontier}
    (${STAGES[r.frontier].name})`);
  for (const l of r.log) console.log(' ' + l);
  const rng = PRNG(opts.seed + 999);
  const brain = (r.best || { brain: r.net }).brain;
  for (let st = 0; st <= r.frontier; st++) {
    const suite = Array.from({ length: 12 }, (_, i) => caseFor(st, (opts.seed + 555 + i * 15485863) >>> 0));
    report(`stage ${st} ${STAGES[st].name}`, evaluate(brain, suite));
  }
  void rng;
  if (opts.mode === 'ablate') {
    const suite = Array.from({ length: 12 }, (_, i) => caseFor(r.frontier, (opts.seed + 555 + i * 15485863) >>> 0));
    console.log('ABLATIONS on the best brain, at its frontier stage');
    report('baseline', evaluate(brain, suite));
    const noRec = cloneBrain(brain);
    for (let i = 0; i < noRec.N; i++) for (let k = 0; k < 3; k++) if (noRec.src[i][k] >= SENS) noRec.w[i][k] = 0;
    report('recurrence cut', evaluate(sanitize(noRec), suite));
    const noThr = cloneBrain(brain);
    for (let i = 0; i < noThr.N; i++) { noThr.bias[i] = 0; noThr.fireBias[i] = [0, 0, 0]; noThr.fireGain[i] = 0.9; }
    report('thresholds reset', evaluate(sanitize(noThr), suite));
    let rec = 0, live = 0;
    for (let i = 0; i < brain.N; i++) for (let k = 0; k < 3; k++) { if (brain.w[i][k] !== 0) { live++; if
      (brain.src[i][k] >= SENS) rec++; } }
    console.log(` live slots ${live}/${brain.N * 3}, of which recurrent ${rec}; decoder fan-in
      ${brain.dnz.join('/')}`);
  }
}
}
}

```

J.3 walker/viewer.html — biped viewer: in-page training, 3D rendering, ablation switches.

```

type="text/html">
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Protoneuron 3D Walker</title>
<style>
:root{color:scheme:dark;--bg:#06101b;--panel:#0f1b2c;--panel2:#091522;--line:#29415f;--text:#eaf3ff;--muted:#9fb1c8;
--cyan:#53d9ff;--green:#55e58b;--gold:#ffd166;--red:#ff6f7d;--violet:#b9a0ff}
*{box-sizing:border-box}body{margin:0;background:radial-gradient(circle at top,#112540,#06101b
62%);font-family:Segoe UI,Arial,sans-serif;color:var(--text)}
.wrap{max-width:1500px;margin:auto;padding:16px}h1{margin:0;font-size:26px}h3{margin:4px 0 10px;font-size:15px}
.sub{color:var(--muted);line-height:1.5;margin:5px 0 14px}
.grid{display:grid;grid-template-columns:1.25fr
.75fr;gap:14px}@media(max-width:1080px){.grid{grid-template-columns:1fr}}
.card{background:rgba(15,27,44,.97);border:1px solid var(--line);border-radius:14px;padding:14px;box-shadow:0 12px
30px #0005}
.controls{display:flex;flex-wrap:wrap;gap:7px;margin:10px 0}
button,input,select{border:1px solid var(--line);border-radius:8px;padding:9px
11px;background:#142844;color:var(--text);font-size:13px}
button{cursor:pointer;font-weight:700}button.primary{background:var(--cyan);color:#04121b}button.good{background:var
(--green);color:#062014}button.bad{background:var(--red);color:#25040a}
button.disabled{opacity:.4;cursor:not-allowed}input{width:84px}
label{font-size:12px;color:var(--muted);display:inline-flex;align-items:center;gap:5px}
.small{font-size:12px;color:var(--muted)}.tiny{font-size:11px;color:var(--muted)}
.metrics{display:grid;grid-template-columns:repeat(7,1fr);gap:7px;margin:10px 0}
@media(max-width:950px){.metrics{grid-template-columns:repeat(4,1fr)}}
.m{background:var(--panel2);border:1px solid var(--line);border-radius:9px;padding:8px}
.m span{display:block;color:var(--muted);font-size:11px}.m b{font-size:17px}
canvas{display:block;width:100%;height:auto;background:#081321;border:1px solid var(--line);border-radius:10px}
pre{background:#07101d;border:1px solid
var(--line);border-radius:9px;padding:10px;max-height:190px;overflow:auto;white-space:pre-wrap;font:11.5px
Consolas,monospace;margin:0}
textarea{width:100%;min-height:70px;background:#07101d;color:var(--text);border:1px solid
var(--line);border-radius:9px;padding:9px;font:11px Consolas,monospace}
.status{padding:9px 10px;border-radius:9px;background:#081523;border:1px solid
var(--line);min-height:38px;font-size:13px}
table{width:100%;border-collapse:collapse;font-size:12px}
th{text-align:left;color:var(--muted);font-weight:600;padding:5px 8px;border-bottom:1px solid var(--line)}
td{padding:5px 8px;border-bottom:1px solid #16273c}
.bar{height:8px;border-radius:999px;background:#0b1a2b;border:1px solid var(--line);overflow:hidden;margin:6px 0}
.bar i{display:block;height:100%;background:linear-gradient(90deg,var(--cyan),var(--green));width:0;transition:width
.4s}
.ok{color:var(--green)}.warn{color:var(--gold)}.bad{color:var(--red)}.dim{color:var(--muted)}
.pill{display:inline-block;border:1px solid var(--line);border-radius:999px;padding:2px
8px;font-size:11px;margin:2px}
</style>
</head>
<body><div class="wrap">
<h1>Protoneuron 3D Walker</h1>
<div class="sub">A point-mass torso on rigid massless legs in 3D, driven by the same ternary protoneuron controller
as the pendulum lab: 3 incoming slots per neuron, weights and activations in <b>&minus;1, 0, +1</b>, and nothing
continuous except each neuron's bias, thresholds and gain. Ankle torque is the only energy source and the
landing collision is the only sink, so a steady gait has to be a real limit cycle. Train it here and watch it
walk.</div>
<div class="grid">
<div class="card">
<canvas id="view" width="900" height="470"></canvas>
<div class="controls">
<button class="primary" id="train">Train</button>
<button id="pause">Pause</button>
<button class="good" id="watch">Watch</button>
<button id="report">Score all stages</button>
<button class="bad" id="reset">Reset</button>
</div>
<div class="controls">
<label>Neurons <input id="neurons" type="number" min="8" max="256" step="4" value="24"></label>
<label>Seed <input id="seed" type="number" value="1000"></label>
<label>Cases <input id="cases" type="number" min="4" max="32" value="8"></label>
<label>Candidates <input id="cands" type="number" min="2" max="16" value="8"></label>
<label>Mutations <input id="mut" type="number" min="1" max="12" value="3"></label>
</div>
<div class="controls">
<label>Watch stage <select id="stage"></select></label>
<label>Policy <select id="policy"><option value="evolved">evolved brain</option><option
value="reference">hand-tuned reference</option></select></label>
<label>Ablation <select id="ablate">
<option value="none">none</option>
<option value="rec">cut all recurrence</option>
<option value="thr">reset all thresholds</option></select></label>
</div>
<div class="metrics">
<div class="m"><span>Mode</span><b id="mMode">idle</b></div>

```

```

<div class="m"><span>Generation</span><b id="mGen">0</b></div>
<div class="m"><span>Curriculum</span><b id="mStage">stand</b></div>
<div class="m"><span>Distance</span><b id="mDist">0.00 m</b></div>
<div class="m"><span>Speed</span><b id="mSpeed">0.00</b></div>
<div class="m"><span>Steps</span><b id="mSteps">0</b></div>
<div class="m"><span>Peak impulse</span><b id="mImp">0.00</b></div>
</div>
<div class="status" id="status">Press Train. Promotion through the five curriculum rungs usually takes a couple of
  minutes on one core; then press Watch.</div>
</div>

<div class="card">
  <h3>Curriculum</h3>
  <div class="bar"><i id="stagebar"></i></div>
  <div id="stages"></div>
  <h3 style="margin-top:14px">Live connectome</h3>
  <canvas id="net" width="420" height="260"></canvas>
  <div class="tiny" style="margin-top:6px">Green node = +1 firing • red = &minus;1 • gray = 0. Edges are live
    recurrent connections; green = +1 weight, red = &minus;1.</div>
  <h3 style="margin-top:14px">Log</h3>
  <pre id="log">ready</pre>
</div>
</div>

<div class="card" style="margin-top:14px">
  <h3>Stage report</h3>
  <div class="tiny" style="margin-bottom:8px">Twelve fresh cases per rung, scored with the current ablation applied.
    Run it with ablation set to <b>none</b>, then to <b>cut all recurrence</b>, and compare – on this task the
    difference is the whole result.</div>
  <table><thead><tr><th>rung</th><th>distance</th><th>time</th><th>steps</th><th>score</th><th>injured</th><th>peak
    impulse</th></tr></thead><tbody id="rep"><tr><td colspan="7" class="dim">not scored
    yet</td></tr></tbody></table>
  <h3 style="margin-top:14px">Save / load champion</h3>
  <div class="controls"><button id="save">Write to box</button><button id="load">Load from box</button><button
    id="download">Download</button></div>
  <textarea id="box" spellcheck="false" placeholder="Champion JSON appears here."></textarea>
  <div class="status" id="io" style="margin-top:8px">Nothing saved this session.</div>
</div>
</div>

<script src="walker3d.js"></script>
<script>
const W = window.WALKER;
const $ = id => document.getElementById(id);
const setText = (id, v) => { const e = $(id); if (e) e.textContent = v; else console.warn('missing #' + id); };
const setHTML = (id, v) => { const e = $(id); if (e) e.innerHTML = v; else console.warn('missing #' + id); };
const vctx = $('view').getContext('2d'), nctx = $('net').getContext('2d');

let T = null; // trainer state
let mode = 'idle';
let ws = null, wprev = [], prints = [], trail = [], firing = [];
let trainTimer = 0, logLines = [];
let brainForWatch = null;

function say(m) { logLines.push(m); logLines = logLines.slice(-120); setText('log', logLines.join('\n')); const e =
  $('log'); if (e) e.scrollTop = e.scrollHeight; }
const opts = () => ({
  seed: +$('seed').value || 1000, neurons: Math.max(8, +$('neurons').value || 24),
  cases: Math.max(4, +$('cases').value || 8), cands: Math.max(2, +$('cands').value || 8),
  mut: Math.max(1, +$('mut').value || 3), rotate: 20, decDensity: 0.5,
});

// ---- ablations, applied to a copy so the trained brain is never damaged ----
function applyAblation(b) {
  const kind = $('ablate').value;
  if (!b || kind === 'none') return b;
  const c = W.cloneBrain(b);
  if (kind === 'rec') { for (let i = 0; i < c.N; i++) for (let k = 0; k < 3; k++) if (c.src[i][k] >= W.SENS)
    c.w[i][k] = 0; }
  if (kind === 'thr') { for (let i = 0; i < c.N; i++) { c.bias[i] = 0; c.fireBias[i] = [0, 0, 0]; c.fireGain[i] =
    0.9; } }
  return W.sanitize(c);
}

// ---- trainer ----
function initTrainer() {
  const o = opts(), rng = W.PRNG(o.seed);
  T = { o, rng, net: W.makeBrain(o.neurons, rng, o.decDensity), frontier: 0, gen: 0, epoch: 0, best: null };
  T.suite = W.buildSuite(0, o.seed, o.cases, rng);
  say(`fresh brain: ${o.neurons} protoneurons, ${o.neurons * 3} incoming slots, seed ${o.seed}`);
}
function oneGeneration() {
  if (!T) initTrainer();
  const o = T.o;
  if (T.gen % o.rotate === 0) { T.epoch++; T.suite = W.buildSuite(T.frontier, o.seed + T.epoch * 2654435761,

```

```

    o.cases, T.rng); }
let wF = W.fitness(T.net, T.suite), winner = T.net;
for (let i = 0; i < o.cands; i++) {
  const c = W.mutate(T.net, T.rng, o), f = W.fitness(c, T.suite);
  if (f.f > wF.f) { winner = c; wF = f; }
}
T.net = winner; T.gen++;
if (T.gen % 8 === 0) {
  const probe = W.buildSuite(T.frontier, o.seed + 777 + T.gen, Math.max(8, o.cases), T.rng);
  const e = W.evaluate(T.net, probe);
  const fs = e.stageScore[T.frontier];
  if (fs !== undefined && fs >= W.STAGES[T.frontier].pass && T.frontier < W.STAGES.length - 1) {
    T.frontier++;
    say(`gen ${T.gen}: promoted to ${W.STAGES[T.frontier].name} (${W.STAGES[T.frontier - 1].name} scored
      ${fs.toFixed(2)})`);
    $('stage').value = String(T.frontier);
  }
  if (!T.best || e.score > T.best.score) T.best = { brain: W.cloneBrain(T.net), score: e.score, gen: T.gen };
  say(`gen ${T.gen}: ${W.STAGES[T.frontier].name} score ${e.score.toFixed(3)} dist ${e.dist.toFixed(2)}m steps
    ${e.steps.toFixed(1)} injured ${e.injRate * 100}.toFixed(0)}%`);
}
drawStages(); updateMetrics();
}
function trainLoop() {
  if (mode !== 'train') return;
  try { oneGeneration(); } catch (e) { say('training error: ' + e.message); mode = 'idle'; return; }
  trainTimer = setTimeout(trainLoop, 0);
}

// ---- watching ----
function startWatch() {
  const src = $('policy').value === 'reference' ? null : ((T && (T.best ? T.best.brain : T.net)) || null);
  if ($('policy').value === 'evolved' && !src) { setText('status', 'Train first, or switch Policy to the hand-tuned
    reference.');
```

```

const P = q => project(q, cam, tgt, W2, H2, f);
// ground grid
c.lineWidth = 1;
const x0 = Math.floor(px - 3), x1 = px + 11;
for (let gx = x0; gx <= x1; gx += 0.5) {
  const a = P([gx, -2.4, 0]), b = P([gx, 2.4, 0]);
  if (a && b) { c.strokeStyle = (Math.abs(gx % 1) < 1e-6) ? '#20364f' : '#14243a'; c.beginPath(); c.moveTo(a[0],
    a[1]); c.lineTo(b[0], b[1]); c.stroke(); }
}
for (let gy = -2.4; gy <= 2.4; gy += 0.4) {
  const a = P([x0, gy, 0]), b = P([x1, gy, 0]);
  if (a && b) { c.strokeStyle = Math.abs(gy) < 1e-6 ? '#2c4a6b' : '#14243a'; c.beginPath(); c.moveTo(a[0], a[1]);
    c.lineTo(b[0], b[1]); c.stroke(); }
}
if (!ws) { c.fillStyle = '#9fb1c8'; c.font = '15px Segoe UI'; c.fillText('Press Train, then Watch.', 24, 32);
  return; }
// footprints
for (const fp of prints) {
  const a = P(fp.p); if (!a) continue;
  c.fillStyle = fp.side > 0 ? '#2e6f8e' : '#8e5a2e';
  c.beginPath(); c.ellipse(a[0], a[1], Math.max(2, 46 / a[2]), Math.max(1.2, 22 / a[2]), 0, 0, Math.PI * 2);
  c.fill();
}
// CoM trail
c.strokeStyle = '#2a4a6b'; c.lineWidth = 1.5; c.beginPath();
let started = false;
for (const q of trail) { const a = P(q); if (!a) { started = false; continue; } if (!started) { c.moveTo(a[0],
  a[1]); started = true; } else c.lineTo(a[0], a[1]); }
c.stroke();
// shadow under the CoM
const sh = P([ws.p[0], ws.p[1], 0]);
if (sh) { c.fillStyle = '#0d1b2b'; c.beginPath(); c.ellipse(sh[0], sh[1], Math.max(2, 40 / sh[2]), Math.max(1, 18
  / sh[2]), 0, 0, Math.PI * 2); c.fill(); }
// stance leg and torso
const fo = P(ws.f), co = P(ws.p);
if (fo && co) {
  c.strokeStyle = ws.side > 0 ? '#53d9ff' : '#ffd166'; c.lineWidth = Math.max(2.5, 26 / co[2]);
  c.beginPath(); c.moveTo(fo[0], fo[1]); c.lineTo(co[0], co[1]); c.stroke();
  // swing-leg preview toward the commanded placement
  if (ws.lastCmd) {
    const h = Math.min(W.L - 1e-3, Math.max(0.05, ws.p[2]));
    const rad = Math.sqrt(Math.max(1e-6, W.L * W.L - h * h));
    let dx = Math.max(-1, Math.min(1, ws.lastCmd[0])), dy = Math.max(-1, Math.min(1, ws.lastCmd[1])) + (-ws.side)
      * 0.75;
    const dn = Math.hypot(dx, dy) || 1;
    const nf = P([ws.p[0] + rad * dx / dn, ws.p[1] + rad * dy / dn, 0]);
    if (nf) { c.strokeStyle = '#3d5a77'; c.lineWidth = Math.max(1.5, 14 / co[2]); c.setLineDash([4, 5]);
      c.beginPath(); c.moveTo(co[0], co[1]); c.lineTo(nf[0], nf[1]); c.stroke(); c.setLineDash([]); }
  }
  const rr = Math.max(4, 120 / co[2]);
  const grd = c.createRadialGradient(co[0] - rr * .3, co[1] - rr * .3, rr * .2, co[0], co[1], rr);
  grd.addColorStop(0, '#eaf3ff'); grd.addColorStop(1, ws.injured ? '#ff6f7d' : '#7fb6dd');
  c.fillStyle = grd; c.beginPath(); c.arc(co[0], co[1], rr, 0, Math.PI * 2); c.fill();
}
c.fillStyle = '#eaf3ff'; c.font = '14px Segoe UI';
c.fillText(`t ${ws.t.toFixed(2)}s x ${ws.p[0].toFixed(2)}m v ${ws.v[0].toFixed(2)} m/s target
  ${ws.vTarget.toFixed(2)}`, 18, 26);
c.fillStyle = '#9fb1c8'; c.font = '12px Segoe UI';
c.fillText(`stance ${ws.side > 0 ? 'right' : 'left'} lean ${Math.hypot(ws.p[0] - ws.f[0], ws.p[1] - ws.f[1]) /
  W.L}.toFixed(2)} impulse ${ws.lastImpulse.toFixed(2)}`, 18, 46);
if (ws.injured) { c.fillStyle = '#ff6f7d'; c.font = '15px Segoe UI'; c.fillText('INJURED', Wd - 90, 26); }
else if (ws.fallen) { c.fillStyle = '#ffd166'; c.font = '15px Segoe UI'; c.fillText('FELL', Wd - 62, 26); }
}
function drawNet() {
  const b = brainForWatch || (T && (T.best ? T.best.brain : T.net));
  const c = nctx, Wd = $('net').width, Ht = $('net').height;
  c.clearRect(0, 0, Wd, Ht);
  if (!b) { c.fillStyle = '#9fb1c8'; c.font = '12px Segoe UI'; c.fillText('no brain yet', 14, 22); return; }
  const cx = Wd / 2, cy = Ht / 2, rad = Math.min(Wd, Ht) * 0.38, pts = [];
  for (let i = 0; i < b.N; i++) { const a = (i / b.N) * Math.PI * 2 - Math.PI / 2; pts.push([cx + Math.cos(a) * rad,
    cy + Math.sin(a) * rad]); }
  for (let i = 0; i < b.N; i++) for (let k = 0; k < 3; k++) {
    const s = b.src[i][k], w = b.w[i][k];
    if (s < W.SENS || w === 0) continue;
    const a = pts[s - W.SENS], d = pts[i], on = firing[s - W.SENS] !== 0 && firing.length;
    c.strokeStyle = w > 0 ? '#55e58b' : '#ff6f7d';
    c.globalAlpha = on ? 0.8 : 0.22; c.lineWidth = on ? 1.6 : 1;
    c.beginPath(); c.moveTo(a[0], a[1]); c.lineTo(d[0], d[1]); c.stroke();
  }
  c.globalAlpha = 1;
  for (let i = 0; i < b.N; i++) {
    const v = firing.length ? firing[i] : 0;
    c.fillStyle = v > 0 ? '#55e58b' : v < 0 ? '#ff6f7d' : '#52647c';
    c.beginPath(); c.arc(pts[i][0], pts[i][1], Math.max(3, 60 / b.N + 2), 0, Math.PI * 2); c.fill();
  }
}

```

```

c.fillStyle = '#9fb1c8'; c.font = '11px Segoe UI';
let rec = 0, live = 0;
for (let i = 0; i < b.N; i++) for (let k = 0; k < 3; k++) if (b.w[i][k] !== 0) { live++; if (b.src[i][k] >=
  W.SENS) rec++; }
c.fillText(`N=${b.N} · ${live}/${b.N * 3} live slots · ${rec} recurrent`, 12, Ht - 10);
}

function drawStages() {
  const fr = T ? T.frontier : 0;
  setHTML('stages', W.STAGES.map((s, i) => {
    const state = i < fr ? 'ok' : i === fr ? 'warn' : 'dim';
    const tag = i < fr ? 'passed' : i === fr ? 'training' : 'locked';
    return `<div style="display:flex;justify-content:space-between;padding:4px 0;border-bottom:1px solid #16273c">
      <span><b>${i}</b> </span> <span>${s.name}</span>
      <span class="small">target ${s.vTarget.toFixed(2)} m/s · ${s.T}s · kick &times; ${s.perturb} </span>
      <span class="${state}">${tag}</span></div>`;
  })).join('');
  const e = $('stagebar'); if (e) e.style.width = (100 * fr / (W.STAGES.length - 1)) + '%';
}

function updateMetrics() {
  setText('mMode', mode);
  setText('mGen', T ? T.gen : 0);
  setText('mStage', W.STAGES[T ? T.frontier : 0].name);
  setText('mDist', (ws ? ws.p[0] : 0).toFixed(2) + ' m');
  setText('mSpeed', (ws ? ws.v[0] : 0).toFixed(2));
  setText('mSteps', ws ? ws.steps : 0);
  setText('mImp', (ws ? ws.peakImpulse : 0).toFixed(2));
  $('watch').disabled = !(T || $('policy').value === 'reference');
}

function scoreAll() {
  const src = $('policy').value === 'reference' ? null : (T && (T.best ? T.best.brain : T.net));
  if (['reference', 'evolved'].includes($('policy').value) && !src) { setText('status', 'Train first, or score the hand-tuned reference.');
```

```

        document.createElement('a');
        a.href = u; a.download = 'walker-champion.json'; document.body.appendChild(a); a.click(); a.remove();
        setTimeout(() => URL.revokeObjectURL(u), 2000); setText('io', 'Download started.');
```

J.4 quad4.js — the 3D quadruped: 6-DOF torso, two-segment legs, penalty contact with stick-slip friction, cycled objectives, gait descriptors, diversity archive.

```

'use strict';
// A very simple little dog: 3D quadruped, two-segment legs, driven by the same ternary
// protoneuron controller as the pendulum lab and the biped walker.
//
// node quad4.js --mode selftest
// node quad4.js --mode reference
// node quad4.js --mode evolve --seconds 240 --neurons 32
// node quad4.js --mode ablate --seconds 240 --neurons 32
//
// PLANT: rigid torso with full 6-DOF dynamics (rotation carried as a matrix, so there is
// no Euler singularity), four massless two-segment legs with hip swing, hip abduction and
// a knee. Feet touch the ground through a penalty contact: a normal spring-damper plus a
// stick-slip Coulomb friction anchor. Ground reaction is the only thing that can push the
// dog forward, so it has to press a foot down and sweep it back – there is no cheat where
// a leg can pull the body along through the air.
//
// The controller gets no clock. A quadruped gait is a four-phase rhythm, so any rhythm has
// to come out of the recurrent population itself.

const M = 3.0, G = 9.81; // little dog, 3 kg
const BL = 0.30, BW = 0.16, BH = 0.09; // torso length, width, height
const IX = M / 12 * (BW * BW + BH * BH), IY = M / 12 * (BL * BL + BH * BH), IZ = M / 12 * (BL * BL + BW * BW);
const L1 = 0.090, L2 = 0.100; // upper and lower segment
const HIPX = 0.130, HIPY = 0.060, HIPZ = -0.035;
const DT = 0.002;
const KN = 1100, DN = 18; // contact normal spring-damper
const KT = 900, DTC = 14, MU = 0.85; // friction, stick-slip
const RATE = 5.0; // joint servo rate limit, rad/s. Higher and a
// rate-limited leg drives the foot into the ground
// at metres per second and the damper spikes.

const K0 = 0.62; // nominal knee flexion
const H0 = 0.205; // nominal torso height
const FALL_H = 0.105, MAX_TILT = 0.70, VMAX = 3.0;
const FMAX = 6.0 * M * G; // per-foot force above this counts as injury
const ACT = 12; // 4 legs x (swing, abduction, knee)
const SENS = 18;
const DECT = 0.22;
// leg order: 0 front-left, 1 front-right, 2 rear-left, 3 rear-right
const HIP = [[HIPX, HIPY, HIPZ], [HIPX, -HIPY, HIPZ], [-HIPX, HIPY, HIPZ], [-HIPX, -HIPY, HIPZ]];
const SY = [1, -1, 1, -1]; // +1 left, -1 right; abduction is outward

const clamp = (x, a, b) => Math.max(a, Math.min(b, x));
const tri = x => (x > 0.33 ? 1 : x < -0.33 ? -1 : 0);
const fin = (x, f = 0) => (Number.isFinite(x) ? x : f);
function PRNG(seed) {
    let s = (seed >>> 0) || 1;
    return () => { s = (s + 0x6D2B79F5) >>> 0; let t = s; t = Math.imul(t ^ (t >>> 15), t | 1);
        t ^= t + Math.imul(t ^ (t >>> 7), t | 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
}
const dot3 = (a, b) => a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
const cross = (a, b) => [a[1] * b[2] - a[2] * b[1], a[2] * b[0] - a[0] * b[2], a[0] * b[1] - a[1] * b[0]];
const mul = (R, u) => [R[0] * u[0] + R[1] * u[1] + R[2] * u[2],
    R[3] * u[0] + R[4] * u[1] + R[5] * u[2],
    R[6] * u[0] + R[7] * u[1] + R[8] * u[2]];
const mult = (R, u) => [R[0] * u[0] + R[3] * u[1] + R[6] * u[2],
    R[1] * u[0] + R[4] * u[1] + R[7] * u[2],
    R[2] * u[0] + R[5] * u[1] + R[8] * u[2]];

// ----- leg kinematics
// Foot position in body frame. Abduction is outward for both sides; the knee bends the
// shank backward, like an elbow.
function footBody(leg, th, ph, kn) {
    const s = Math.sin(SY[leg] * ph), c = Math.cos(SY[leg] * ph);
    const d1 = [c * Math.sin(th), s, -c * Math.cos(th)];
    const a2 = th - kn;
    const d2 = [c * Math.sin(a2), s, -c * Math.cos(a2)];
    const h = HIP[leg];
```

```

    return [h[0] + L1 * d1[0] + L2 * d2[0], h[1] + L1 * d1[1] + L2 * d2[1], h[2] + L1 * d1[2] + L2 * d2[2]];
}
// hip, knee and foot in world space, for drawing
function legPoints(s, leg) {
    const h = HIP[leg], sg = Math.sin(SY[leg] * s.ph[leg]), cg = Math.cos(SY[leg] * s.ph[leg]);
    const th = s.th[leg], a2 = th - s.kn[leg];
    const knee = [h[0] + L1 * cg * Math.sin(th), h[1] + L1 * sg, h[2] - L1 * cg * Math.cos(th)];
    const foot = [knee[0] + L2 * cg * Math.sin(a2), knee[1] + L2 * sg, knee[2] - L2 * cg * Math.cos(a2)];
    const W = b => { const w = mul(s.R, b); return [s.p[0] + w[0], s.p[1] + w[1], s.p[2] + w[2]]; };
    return [W(h), W(knee), W(foot)];
}
const footWorld = (s, leg) => {
    const b = footBody(leg, s.th[leg], s.ph[leg], s.kn[leg]);
    const w = mul(s.R, b);
    return [s.p[0] + w[0], s.p[1] + w[1], s.p[2] + w[2]];
};

// ----- state
function freshState(vTarget, r) {
    return {
        p: [0, 0, H0 + (r() - 0.5) * 0.004], v: [0.1 * vTarget, 0, 0],
        R: [1, 0, 0, 0, 1, 0, 0, 0, 1], w: [(r() - 0.5) * 0.04, (r() - 0.5) * 0.04, (r() - 0.5) * 0.04],
        th: [0, 0, 0, 0].map(() => (r() - 0.5) * 0.04),
        ph: [0, 0, 0, 0].map(() => 0),
        kn: [0, 0, 0, 0].map(() => K0 + (r() - 0.5) * 0.04),
        anchor: [null, null, null, null], contact: [0, 0, 0, 0], fprev: [null, null, null, null],
        t: 0, done: false, fallen: false, injured: false, peakF: 0, steps: 0, energy: 0,
        vTarget, slip: 0, cmd: new Array(ACT).fill(0),
    };
}
function cloneState(s) {
    return { ...s, p: s.p.slice(), v: s.v.slice(), R: s.R.slice(), w: s.w.slice(),
        th: s.th.slice(), ph: s.ph.slice(), kn: s.kn.slice(), contact: s.contact.slice(),
        anchor: s.anchor.map(a => (a ? a.slice() : null)), fprev: s.fprev.map(a => (a ? a.slice() : null)),
        cmd: s.cmd.slice(), _pr: null, _nextKick: 0 };
}
// Orthonormalise after integrating the rotation, so the frame cannot drift into a shear.
function fixR(R) {
    let x = [R[0], R[3], R[6]], y = [R[1], R[4], R[7]];
    let n = Math.hypot(x[0], x[1], x[2]) || 1; x = x.map(v => v / n);
    const d = dot3(x, y); y = [y[0] - d * x[0], y[1] - d * x[1], y[2] - d * x[2]];
    n = Math.hypot(y[0], y[1], y[2]) || 1; y = y.map(v => v / n);
    const z = cross(x, y);
    return [x[0], y[0], z[0], x[1], y[1], z[1], x[2], y[2], z[2]];
}

// ----- one physics step
function physics(s, targets) {
    // servos move toward the commanded angles at a bounded rate
    const lim = RATE * DT;
    for (let i = 0; i < 4; i++) {
        s.th[i] += clamp(targets[3 * i] - s.th[i], -lim, lim);
        s.ph[i] += clamp(targets[3 * i + 1] - s.ph[i], -lim, lim);
        s.kn[i] += clamp(targets[3 * i + 2] - s.kn[i], -lim, lim);
        s.energy += (Math.abs(targets[3 * i] - s.th[i]) + Math.abs(targets[3 * i + 2] - s.kn[i])) * DT;
    }
    let F = [0, 0, -M * G], Tq = [0, 0, 0];
    for (let i = 0; i < 4; i++) {
        const q = footWorld(s, i);
        const fp = s.fprev[i];
        const vel = fp ? [(q[0] - fp[0]) / DT, (q[1] - fp[1]) / DT, (q[2] - fp[2]) / DT] : [0, 0, 0];
        s.fprev[i] = q.slice();
        if (q[2] < 0) {
            const pen = -q[2];
            const Fn = Math.max(0, KN * pen - DN * vel[2]);
            s.peakF = Math.max(s.peakF, Fn);
            if (Fn > FMAX) { s.injured = true; s.done = true; return; }
            if (!s.anchor[i]) s.anchor[i] = [q[0], q[1]];
            let dx = q[0] - s.anchor[i][0], dy = q[1] - s.anchor[i][1];
            const dm = Math.hypot(dx, dy), limF = MU * Fn;
            if (KT * dm > limF && dm > 1e-9) { // slip: drag the anchor
                const keep = limF / (KT * dm);
                s.anchor[i][0] += dx * (1 - keep); s.anchor[i][1] += dy * (1 - keep);
                dx *= keep; dy *= keep; s.slip += (1 - keep) * dm;
            }
            const Ft = [clamp(-KT * dx - DTC * vel[0], -limF, limF), clamp(-KT * dy - DTC * vel[1], -limF, limF)];
            const Fi = [Ft[0], Ft[1], Fn];
            F = [F[0] + Fi[0], F[1] + Fi[1], F[2] + Fi[2]];
            const rr = [q[0] - s.p[0], q[1] - s.p[1], q[2] - s.p[2]], tq = cross(rr, Fi);
            Tq = [Tq[0] + tq[0], Tq[1] + tq[1], Tq[2] + tq[2]];
            s.contact[i] = 1;
        } else { s.anchor[i] = null; s.contact[i] = 0; }
    }
}
// linear
const a = [F[0] / M, F[1] / M, F[2] / M];

```

```

for (let k = 0; k < 3; k++) { s.v[k] += a[k] * DT; s.p[k] += s.v[k] * DT; }
// angular, in the body frame
const Tb = mult(s.R, Tq, I = [IX, IY, IZ]);
const Iw = [I[0] * s.w[0], I[1] * s.w[1], I[2] * s.w[2]], gy = cross(s.w, Iw);
for (let k = 0; k < 3; k++) s.w[k] += ((Tb[k] - gy[k]) / I[k]) * DT;
const om = s.w, th = Math.hypot(om[0], om[1], om[2]) * DT;
if (th > 1e-9) { // Rodrigues, then re-orthonormalise
  const ax = [om[0] * DT / th, om[1] * DT / th, om[2] * DT / th];
  const ct = Math.cos(th), st = Math.sin(th), vt = 1 - ct;
  const Rr = [ct + ax[0] * ax[0] * vt, ax[0] * ax[1] * vt - ax[2] * st, ax[0] * ax[2] * vt + ax[1] * st,
    ax[1] * ax[0] * vt + ax[2] * st, ct + ax[1] * ax[1] * vt, ax[1] * ax[2] * vt - ax[0] * st,
    ax[2] * ax[0] * vt - ax[1] * st, ax[2] * ax[1] * vt + ax[0] * st, ct + ax[2] * ax[2] * vt];
  const R = s.R, out = new Array(9);
  for (let i2 = 0; i2 < 3; i2++) for (let j = 0; j < 3; j++) {
    out[i2 * 3 + j] = R[i2 * 3] * Rr[j] + R[i2 * 3 + 1] * Rr[3 + j] + R[i2 * 3 + 2] * Rr[6 + j];
  }
  s.R = fixR(out);
}
s.t += DT;
if (!s.p.every(Number.isFinite) || !s.v.every(Number.isFinite) || !s.R.every(Number.isFinite)) {
  s.done = true; s.fallen = true; return;
}
const up = [s.R[2], s.R[5], s.R[8]]; // body z in world
const tilt = Math.acos(clamp(up[2], -1, 1));
s.tilt = tilt;
if (s.p[2] < FALL_H || tilt > MAX_TILT || Math.hypot(s.v[0], s.v[1], s.v[2]) > VMAX) { s.done = true; s.fallen = true; }
}

// ----- sensing
function encode(s) {
  const up = mult(s.R, [0, 0, 1]); // gravity direction in body frame
  const vb = mult(s.R, s.v); // velocity in body frame
  return [
    tri(up[0] / 0.25), tri(up[1] / 0.25), // pitch, roll
    tri(s.w[0] / 2.0), tri(s.w[1] / 2.0), tri(s.w[2] / 2.0),
    tri(vb[0] / 0.5), tri(vb[1] / 0.35), tri(s.v[2] / 0.35),
    tri((s.p[2] - H0) / 0.03),
    tri((vb[0] - s.vTarget) / 0.25),
    s.contact[0] ? 1 : -1, s.contact[1] ? 1 : -1, s.contact[2] ? 1 : -1, s.contact[3] ? 1 : -1,
    tri((s.th[0] - s.th[3]) / 0.3), // diagonal pair skew, front-left vs rear-right
    tri((s.th[1] - s.th[2]) / 0.3), // the other diagonal
    tri((s.kn[0] + s.kn[3] - s.kn[1] - s.kn[2]) / 0.5), // which diagonal is loaded
    1,
  ];
}
// map ternary votes to joint targets
function targetsFrom(out) {
  const t = new Array(ACT);
  for (let i = 0; i < 4; i++) {
    t[3 * i] = 0.55 * out[3 * i];
    t[3 * i + 1] = 0.26 * out[3 * i + 1];
    t[3 * i + 2] = clamp(K0 + 0.52 * out[3 * i + 2], 0.06, 1.25);
  }
  return t;
}

// ----- brain (unchanged model)
function makeBrain(N, rng, decDensity) {
  const total = SENS + N, src = [], w = [];
  for (let i = 0; i < N; i++) {
    src.push([Math.floor(rng() * total), Math.floor(rng() * total), Math.floor(rng() * total)]);
    w.push([Math.floor(rng() * 3) - 1, Math.floor(rng() * 3) - 1, Math.floor(rng() * 3) - 1]);
  }
  const decoder = Array.from({ length: ACT }, () =>
    Array.from({ length: N }, () => (rng() < decDensity ? (rng() < 0.5 ? -1 : 1) : 0)));
  return sanitize({ N, src, w, decoder, bias: Array(N).fill(0),
    fireBias: Array.from({ length: N }, () => [0, 0, 0]), fireGain: Array(N).fill(0.9) });
}
function sanitize(b) {
  for (let i = 0; i < b.N; i++) {
    for (let k = 0; k < 3; k++) {
      if (![-1, 0, 1].includes(b.w[i][k])) b.w[i][k] = 0;
      b.src[i][k] = clamp(b.src[i][k] | 0, 0, SENS + b.N - 1);
      b.fireBias[i][k] = clamp(fin(b.fireBias[i][k]), -5, 5);
    }
    b.fireGain[i] = clamp(fin(b.fireGain[i], 0.9), 0.08, 3.5);
    b.bias[i] = clamp(fin(b.bias[i]), -2.5, 2.5);
  }
  b.dnz = [];
  for (let j = 0; j < ACT; j++) {
    let nz = 0;
    for (let i = 0; i < b.N; i++) { if (![-1, 0, 1].includes(b.decoder[j][i])) b.decoder[j][i] = 0; if
      (b.decoder[j][i]) nz++; }
    b.dnz[j] = nz;
  }
}

```

```

    }
    return b;
}
const cloneBrain = b => sanitize({ N: b.N, src: b.src.map(r => r.slice()), w: b.w.map(r => r.slice()),
  decoder: b.decoder.map(r => r.slice()), bias: b.bias.slice(),
  fireBias: b.fireBias.map(r => r.slice()), fireGain: b.fireGain.slice() });
function forward(b, s, prev) {
  const f = encode(s), h = new Array(b.N);
  for (let i = 0; i < b.N; i++) {
    let z = b.bias[i];
    for (let k = 0; k < 3; k++) { const sc = b.src[i][k]; z += b.w[i][k] * (sc < SENS ? f[sc] : prev[sc - SENS]); }
    const d = b.fireGain[i] * clamp(z, -12, 12) / DECT;
    const r0 = b.fireBias[i][0] - d, r1 = b.fireBias[i][1], r2 = b.fireBias[i][2] + d;
    h[i] = (r0 > r1 && r0 >= r2) ? -1 : (r2 > r1 && r2 > r0) ? 1 : 0;
  }
  const out = new Array(ACT);
  for (let j = 0; j < ACT; j++) {
    let sum = 0;
    for (let i = 0; i < b.N; i++) sum += b.decoder[j][i] * h[i];
    out[j] = Math.tanh(sum / Math.max(3, (b.dnz[j] || b.N * 0.12) * 0.55));
  }
  return { h, out };
}

// ----- curriculum (cycled)
const STAGES = [
  { name: 'stand', vTarget: 0.00, T: 3.0, perturb: 0.0, pass: 0.80 },
  { name: 'settle', vTarget: 0.00, T: 4.0, perturb: 0.4, pass: 0.72 },
  { name: 'creep', vTarget: 0.15, T: 5.0, perturb: 0.0, pass: 0.60 },
  { name: 'walk', vTarget: 0.35, T: 6.0, perturb: 0.3, pass: 0.55 },
  { name: 'trot', vTarget: 0.60, T: 7.0, perturb: 0.5, pass: 0.50 },
];
function caseFor(stage, seed) {
  const r = PRNG(seed >>> 0), st = STAGES[stage];
  const s = freshState(st.vTarget, r);
  s.caseSeed = seed >>> 0; s.stage = stage; s.T = st.T; s.perturb = st.perturb;
  return s;
}
function buildSuite(frontier, seedBase, n, rng) {
  const out = [];
  for (let i = 0; i < n; i++) {
    const stage = (rng() < 0.55 || frontier === 0) ? frontier : Math.floor(rng() * frontier);
    out.push(caseFor(stage, (seedBase + i * 104729 + stage * 7919) >>> 0));
  }
  return out;
}

// ----- episode
function controlStep(brain, s, prev, policy) {
  const st = STAGES[s.stage];
  if (!s._pr) { s._pr = PRNG((s.caseSeed || 1) + 4242); s._nextKick = 0.8 + s._pr() * 1.4; }
  const pr = s._pr;
  let rew = 0;
  let out;
  if (policy) out = policy(s);
  else { const o = forward(brain, s, prev); prev = o.h; out = o.out; }
  s.cmd = out.slice();
  const targets = targetsFrom(out);
  physics(s, targets);
  if (s.done) return { prev, rew };
  const up = mult(s.R, [0, 0, 1]), vb = mult(s.R, s.v);
  const tiltPen = up[0] * up[0] + up[1] * up[1];
  const vErr = vb[0] - s.vTarget;
  const nContact = s.contact[0] + s.contact[1] + s.contact[2] + s.contact[3];
  rew += (1.0 + 2.5 * s.v[0] - 2.0 * vErr * vErr - 4.0 * tiltPen
    - 2.0 * (s.p[1] * s.p[1]) - 1.5 * Math.pow(s.p[2] - H0, 2) * 100 / 100
    - 0.25 * (nContact === 0 ? 1 : 0) // all four feet airborne is not walking
    - 0.02 * (s.w[0] * s.w[0] + s.w[1] * s.w[1] + s.w[2] * s.w[2])) * DT;
  if (st.perturb > 0 && s.t >= s._nextKick) {
    const ang = pr() * Math.PI * 2, mag = st.perturb * (0.12 + pr() * 0.18);
    s.v[0] += Math.cos(ang) * mag; s.v[1] += Math.sin(ang) * mag;
    s._nextKick = s.t + 1.0 + pr() * 1.5;
  }
  return { prev, rew };
}
function runEpisode(brain, base, policy) {
  const s = cloneState(base);
  let prev = new Array(brain ? brain.N : 0).fill(0), rew = 0;
  const x0 = s.p[0];
  // gait descriptors, accumulated as the episode runs
  let n = 0, feetSum = 0, trans = 0, diagSame = 0, latSame = 0, tiltSum = 0;
  let last = s.contact.slice();
  while (!s.done && s.t < s.T) {
    const r = controlStep(brain, s, prev, policy); prev = r.prev; rew += r.rew;
    const c = s.contact;

```

```

    feetSum += (c[0] + c[1] + c[2] + c[3]) / 4;
    for (let k = 0; k < 4; k++) if (c[k] !== last[k]) trans++;
    last = c.slice();
    diagSame += ((c[0] === c[3] ? 1 : 0) + (c[1] === c[2] ? 1 : 0)) / 2;
    latSame += ((c[0] === c[1] ? 1 : 0) + (c[2] === c[3] ? 1 : 0)) / 2;
    tiltSum += s.tilt || 0;
    n++;
  }
  const dist = s.p[0] - x0, frac = s.t / s.T, nn = Math.max(1, n);
  const desc = {
    duty: clamp(feetSum / nn, 0, 1), // how much of the time feet are down
    rhythm: clamp(trans / Math.max(0.2, s.t) / 12, 0, 1), // contact changes per second
    diag: clamp(0.5 + 0.5 * (diagSame - latSame) / nn, 0, 1), // diagonal vs lateral coupling
  };
  return { dist, time: s.t, rew, injured: s.injured, fallen: s.fallen, peakF: s.peakF,
    slip: s.slip, drift: Math.abs(s.p[1]), frac, desc, tilt: tiltSum / nn,
    score: s.vTarget > 0
      ? Math.max(0, Math.min(1, 0.4 * frac + 0.6 * clamp(dist / Math.max(0.05, s.vTarget * s.T), 0, 1)))
      : Math.max(0, Math.min(1, frac - 0.3 * (s.injured ? 1 : 0))) };
}

function evaluate(brain, suite, policy) {
  let rew = 0, dist = 0, time = 0, inj = 0, score = 0, peak = 0, drift = 0, tilt = 0;
  let duty = 0, rhythm = 0, diag = 0, peakSum = 0;
  const byStage = {};
  for (const c of suite) {
    const e = runEpisode(brain, c, policy);
    rew += e.rew; dist += e.dist; time += e.time; inj += e.injured ? 1 : 0;
    score += e.score; peak = Math.max(peak, e.peakF); peakSum += e.peakF;
    drift += e.drift; tilt += e.tilt;
    duty += e.desc.duty; rhythm += e.desc.rhythm; diag += e.desc.diag;
    (byStage[c.stage] = byStage[c.stage] || []).push(e.score);
  }
  const n = Math.max(1, suite.length), stageScore = {};
  for (const k of Object.keys(byStage)) stageScore[k] = byStage[k].reduce((a, b) => a + b, 0) / byStage[k].length;
  const injRate = inj / n;
  // safety in [0,1]-ish: light on the feet, level, and never injured
  const safety = clamp(1 - peakSum / n / FMAX, 0, 1) - 2.0 * injRate - 1.2 * (tilt / n);
  return { rew: rew / n, dist: dist / n, time: time / n, injRate,
    score: score / n, peak, meanPeak: peakSum / n, drift: drift / n, tilt: tilt / n, safety,
    desc: { duty: duty / n, rhythm: rhythm / n, diag: diag / n }, stageScore };
}

// Two objectives, cycled. Selecting on distance alone buys speed with hard landings;
// selecting on safety alone converges on standing still. Alternating keeps both alive, and
// archive admission uses a score that needs both, so neither phase can throw the other away.
const OBJECTIVES = [
  { name: 'safe walking', wDist: 1.0, wSafe: 3.5 },
  { name: 'distance', wDist: 5.0, wSafe: 0.8 },
];
const robustScore = e => 2.0 * e.dist + 1.0 * e.score + 1.5 * clamp(1 - e.meanPeak / FMAX, 0, 1) - 3.0 * e.injRate;
function fitness(brain, suite, objIdx) {
  const e = evaluate(brain, suite);
  const w = OBJECTIVES[objIdx || 0] % OBJECTIVES.length;
  return { f: e.rew + 4.0 * w.wDist * e.dist + 2.0 * w.wSafe * e.safety
    + 1.5 * e.score - 0.8 * e.drift, robust: robustScore(e), ...e };
}

// ----- diversity archive
// One elite per gait cell. Descriptors never span the full 0..1 range, so the axes track
// the range actually observed and rebin when it widens – rebinning is free because every
// elite carries its descriptor.
const AXES = ['duty', 'rhythm', 'diag'];
const cellKey = c => c.join('|');
let axisRange = { duty: [0.45, 0.95], rhythm: [0.0, 0.35], diag: [0.35, 0.75] };
let BINS = 4;
function resetAxes() { axisRange = { duty: [0.45, 0.95], rhythm: [0.0, 0.35], diag: [0.35, 0.75] }; }
function noteDesc(d) {
  let widened = false;
  for (const k of AXES) {
    const v = clamp(fin(d[k]), 0, 1), r = axisRange[k];
    if (v < r[0] - 1e-9) { r[0] = v; widened = true; }
    if (v > r[1] + 1e-9) { r[1] = v; widened = true; }
  }
  return widened;
}
function binOf(d) {
  return AXES.map(k => {
    const [lo, hi] = axisRange[k], span = Math.max(1e-6, hi - lo);
    return Math.min(BINS - 1, Math.max(0, Math.floor((clamp(fin(d[k]), lo, hi) - lo) / span * BINS * 0.999999)));
  });
}
function genomeDistance(a, b) {
  if (a.N !== b.N) return 1;
  let diff = 0, tot = 0;
  for (let i = 0; i < a.N; i++) for (let k = 0; k < 3; k++) {
    tot += 2; if (a.src[i][k] !== b.src[i][k]) diff++; if (a.w[i][k] !== b.w[i][k]) diff++;
  }
}

```

```

    }
    for (let j = 0; j < ACT; j++) for (let i = 0; i < a.N; i++) { tot++; if (a.decoder[j][i] !== b.decoder[j][i])
      diff++; }
    return diff / Math.max(1, tot);
  }
  function rebinArchive(arc) {
    const all = [...arc.values()]; arc.clear();
    for (const e of all) {
      e.cell = binOf(e.desc);
      const k = cellKey(e.cell), cur = arc.get(k);
      if (!cur || e.robust > cur.robust) arc.set(k, e);
    }
  }
  function tryArchive(arc, brain, ev, gen, serial) {
    if (noteDesc(ev.desc)) rebinArchive(arc);
    const cell = binOf(ev.desc), key = cellKey(cell), cur = arc.get(key);
    const entry = { id: serial, gen, cell, brain: cloneBrain(brain), desc: { ...ev.desc },
      robust: robustScore(ev), dist: ev.dist, score: ev.score, safety: ev.safety,
      meanPeak: ev.meanPeak, injRate: ev.injRate };
    if (!cur || entry.robust > cur.robust + 1e-9) { arc.set(key, entry); return cur ? 'improved' : 'new'; }
    return false;
  }
  // Best first, skipping anything structurally close to a brain already chosen, so the pool
  // holds different dogs rather than one lineage several times over.
  function derivePool(arc, poolSize, minDist) {
    const all = [...arc.values()].sort((a, b) => b.robust - a.robust), picked = [];
    for (const e of all) {
      if (picked.length >= poolSize) break;
      if (minDist > 0 && picked.some(p => genomeDistance(p.brain, e.brain) < minDist)) continue;
      picked.push(e);
    }
    for (const e of all) { if (picked.length >= poolSize) break; if (!picked.includes(e)) picked.push(e); }
    return picked;
  }
  const coverage = arc => ({ cells: arc.size, total: Math.pow(BINS, 3), pct: 100 * arc.size / Math.pow(BINS, 3) });
  function serializeArchive(arc, pool, meta) {
    return JSON.stringify({ v: 1, kind: 'protoneuron-quad4-archive', bins: BINS, axes: axisRange,
      meta: meta || {},
      entries: [...arc.values()].map(e => ({ id: e.id, gen: e.gen, cell: e.cell, desc: e.desc,
        robust: e.robust, dist: e.dist, score: e.score, safety: e.safety, meanPeak: e.meanPeak,
        injRate: e.injRate, brain: { N: e.brain.N, src: e.brain.src, w: e.brain.w,
        decoder: e.brain.decoder, bias: e.brain.bias, fireBias: e.brain.fireBias, fireGain: e.brain.fireGain } })),
      poolIds: (pool || []).map(e => e.id) });
  }
  function deserializeArchive(txt) {
    const d = JSON.parse(txt);
    if (!d || d.kind !== 'protoneuron-quad4-archive') throw new Error('not a quadruped archive');
    BINS = d.bins || BINS;
    resetAxes();
    if (d.axes) for (const k of AXES) { const r = d.axes[k]; if (Array.isArray(r) && r.length === 2) axisRange[k] =
      [clamp(r[0], 0, 1), clamp(r[1], 0, 1)]; }
    for (const e of d.entries || []) noteDesc(e.desc);
    const arc = new Map();
    for (const e of d.entries || []) {
      const entry = { ...e, brain: sanitize({ N: e.brain.N, src: e.brain.src, w: e.brain.w,
        decoder: e.brain.decoder, bias: e.brain.bias, fireBias: e.brain.fireBias, fireGain: e.brain.fireGain }) };
      entry.cell = binOf(entry.desc);
      const k = cellKey(entry.cell), cur = arc.get(k);
      if (!cur || entry.robust > cur.robust) arc.set(k, entry);
    }
    return { archive: arc, meta: d.meta || {} };
  }
}

// ----- reference gait
// Hand-coded trot with an explicit clock. Exists only to prove the plant is walkable
// before asking a clockless network to walk it.
function refWith(g) {
  return s => {
    const off = [0, 0.5, 0.25, 0.75], out = new Array(ACT).fill(0);
    const up = multT(s.R, [0, 0, 1]), fr = [1, 1, -1, -1], lr = [1, -1, 1, -1];
    for (let i = 0; i < 4; i++) {
      const psi = ((s.t / g.per) + off[i]) % 1;
      let th, kn = K0;
      if (psi < g.duty) { const u = psi / g.duty; th = g.amp * (1 - 2 * u); }
      else { const u = (psi - g.duty) / (1 - g.duty); th = -g.amp + 2 * g.amp * u; kn += g.lift * Math.sin(Math.PI *
        u); }
      kn += g.kh * (s.p[2] - H0) + g.kp * up[0] * fr[i] + g.kr * up[1] * lr[i];
      out[3 * i] = clamp(th / 0.55, -1, 1);
      out[3 * i + 1] = 0;
      out[3 * i + 2] = clamp((clamp(kn, 0.1, 1.2) - K0) / 0.52, -1, 1);
    }
    return out;
  };
}
const REF_GAIT = { amp: 0.14, lift: 0.30, per: 0.50, duty: 0.80, kh: -4, kp: 0, kr: 2 };

```

```

const referenceController = refWith(REF_GAIT);

// ----- evolution
function relevant(b) {
  const read = new Set();
  for (let j = 0; j < ACT; j++) for (let i = 0; i < b.N; i++) if (b.decoder[j][i]) read.add(i);
  const deep = new Set(read);
  for (const i of read) for (let k = 0; k < 3; k++) { const sc = b.src[i][k]; if (sc >= SENS) deep.add(sc - SENS); }
  return [...deep];
}
function mutate(parent, rng, o) {
  const c = cloneBrain(parent), rel = relevant(c);
  const other = cur => { const a = [-1, 0, 1].filter(v => v !== cur); return a[Math.floor(rng() * a.length)]; };
  const target = () => (rel.length && rng() < 0.85 ? rel[Math.floor(rng() * rel.length)] : Math.floor(rng() * c.N));
  const source = () => (rng() < 0.5 ? Math.floor(rng() * SENS) : Math.floor(rng() * (SENS + c.N)));
  for (let z = 0; z < o.mut; z++) {
    const i = target(), k = Math.floor(rng() * 3), u = rng();
    if (u < 0.24) c.w[i][k] = other(c.w[i][k]);
    else if (u < 0.46) c.src[i][k] = source();
    else if (u < 0.58) c.bias[i] = clamp(c.bias[i] + (rng() < 0.5 ? -1 : 1) * 0.25, -2.5, 2.5);
    else if (u < 0.72 && !o.noThresholds) { const ci = Math.floor(rng() * 3); c.fireBias[i][ci] =
      clamp(c.fireBias[i][ci] + (rng() < 0.5 ? -1 : 1) * 0.3, -5, 5); }
    else if (u < 0.80 && !o.noThresholds) c.fireGain[i] = clamp(c.fireGain[i] * (rng() < 0.5 ? 0.82 : 1.22), 0.08,
      3.5);
    else { const j = Math.floor(rng() * ACT), ni = (rng() < 0.85 && rel.length) ? rel[Math.floor(rng() *
      rel.length)] : Math.floor(rng() * c.N); c.decoder[j][ni] = other(c.decoder[j][ni]); }
  }
  return sanitize(c);
}
function evolve(o) {
  const rng = PRNG(o.seed);
  BINS = o.bins || 4;
  resetAxes();
  let net = makeBrain(o.neurons, rng, o.decDensity);
  let frontier = 0, gen = 0, epoch = 0, serial = 0, objIdx = 0;
  const archive = new Map();
  let pool = [];
  let suite = buildSuite(frontier, o.seed, o.cases, rng);
  const t0 = Date.now(), log = [];
  while ((Date.now() - t0) / 1000 < o.seconds) {
    if (gen % o.rotate === 0) { epoch++; suite = buildSuite(frontier, o.seed + epoch * 2654435761, o.cases, rng); }
    // cycle the objective: safe walking, then distance, then back
    const prevObj = objIdx;
    objIdx = Math.floor(gen / Math.max(1, o.objCycle)) % OBJECTIVES.length;
    if (objIdx !== prevObj) log.push(`gen ${gen}: objective now "${OBJECTIVES[objIdx].name}"`);
    let wF = fitness(net, suite, objIdx), winner = net, winnerF = wF;
    for (let i = 0; i < o.cands; i++) {
      // parents come from the current brain or, sometimes, a diverse archive elite
      let base = net;
      if (pool.length && rng() * 100 < (o.revisitPct || 30)) {
        base = (rng() < 0.5 ? pool[Math.floor(Math.pow(rng(), 1.8) * pool.length)]
          : [...archive.values()][Math.floor(rng() * archive.size)]).brain;
      }
      const c = mutate(base, rng, o), f = fitness(c, suite, objIdx);
      if (f > wF.f) { winner = c; wF = f; winnerF = f; }
    }
    net = winner; gen++;
    // every candidate that reaches here is archived on its gait, not just the winner
    if (gen % 4 === 0) {
      tryArchive(archive, net, winnerF, gen, ++serial);
      pool = derivePool(archive, o.poolSize || 8, (o.minDist || 4) / 100);
    }
    if (gen % 8 === 0) {
      const probe = buildSuite(frontier, o.seed + 777 + gen, Math.max(8, o.cases), rng);
      const e = evaluate(net, probe), fs = e.stageScore[frontier];
      tryArchive(archive, net, e, gen, ++serial);
      pool = derivePool(archive, o.poolSize || 8, (o.minDist || 4) / 100);
      if (fs !== undefined && fs >= STAGES[frontier].pass && frontier < STAGES.length - 1) {
        frontier++;
        log.push(`gen ${gen}: promoted to ${STAGES[frontier].name} (${STAGES[frontier - 1].name} scored
          ${fs.toFixed(2)})`);
      }
    }
  }
  const best = pool.length ? pool[0] : null;
  return { net, best, brain: best ? best.brain : net, frontier, gen, log, archive, pool,
    secs: (Date.now() - t0) / 1000, objIdx };
}

// ----- exports / cli
const IS_NODE = typeof process !== 'undefined' && !(process.versions && process.versions.node);
const QUAD_API = { M, G, DT, L1, L2, HIP, SY, H0, K0, ACT, SENS, FALL_H, FMAX, STAGES, MAX_TILT,
  PRNG, clamp, freshState, cloneState, physics, footBody, footWorld, encode, targetsFrom,
  makeBrain, sanitize, cloneBrain, forward, caseFor, buildSuite, controlStep, runEpisode,
  evaluate, fitness, robustScore, OBJECTIVES, referenceController, refWith, REF_GAIT,

```

```

    relevant, mutate, evolve, mul, mult, legPoints, genomeDistance, derivePool, tryArchive,
    coverage, binOf, cellKey, resetAxes, noteDesc, serializeArchive, deserializeArchive,
    get BINS() { return BINS; }, setBins(b) { BINS = b; }, get axisRange() { return axisRange; }, AXES };
if (typeof module !== 'undefined' && module.exports) module.exports = QUAD_API;
if (typeof window !== 'undefined') window.QUAD = QUAD_API;

if (IS_NODE && typeof require !== 'undefined' && require.main === module) {
  const argv = {};
  for (let i = 2; i < process.argv.length; i += 2) argv[process.argv[i].replace(/^--/, '')] = process.argv[i + 1];
  const o = { mode: argv.mode || 'selftest', seed: +(argv.seed || 1000), neurons: +(argv.neurons || 32),
    cases: +(argv.cases || 8), cands: +(argv.cands || 8), mut: +(argv.mut || 3), rotate: +(argv.rotate || 20),
    objCycle: +(argv.objCycle || 15), bins: +(argv.bins || 4), poolSize: +(argv.poolSize || 8),
    minDist: +(argv.minDist || 4), revisitPct: +(argv.revisitPct || 30),
    seconds: +(argv.seconds || 120), decDensity: +(argv.decDensity || 0.4), noThresholds: argv.noThresholds === 'on'
  };
  const rep = (tag, e) => console.log(` ${tag.padEnd(20)} dist ${e.dist.toFixed(3)}m time ${e.time.toFixed(2)}s ` +
    `score ${e.score.toFixed(3)} injured ${e.injRate * 100}.toFixed(0)}% drift ${e.drift.toFixed(3)}m peakF
    ${e.peak.toFixed(1)}N`);

  if (o.mode === 'selftest') {
    console.log('PLANT SELF-TEST');
    const nom = footBody(0, 0, 0, K0);
    console.log(` nominal foot in body frame: [${nom.map(v => v.toFixed(3)).join(', ')}] (hip z ${HIPZ})`);
    console.log(` so nominal torso height ${(-nom[2] + 0).toFixed(3)} + ground = ${H0} configured`);
    // hold the nominal pose: it should stand
    let s = freshState(0, PRNG(5));
    const hold = new Array(ACT).fill(0);
    for (let i = 0; i < 4; i++) { hold[3 * i] = 0; hold[3 * i + 1] = 0; hold[3 * i + 2] = 0; }
    const tg = targetsFrom(hold);
    for (let i = 0; i < 1500 && !s.done; i++) physics(s, tg);
    console.log(` standing 3s on the nominal pose: ${s.done ? 'FELL at t=' + s.t.toFixed(2)} : 'still up', ` +
      `height ${s.p[2].toFixed(4)}m, tilt ${s.tilt || 0}.toFixed(4)}rad, contacts ${s.contact.join('')}, peakF
      ${s.peakF.toFixed(1)}N`);
    console.log(` static load check: mg = ${M * G}.toFixed(2)}N over 4 feet = ${M * G / 4}.toFixed(2)}N each`);
    // limp: knees fully flexed, it should collapse
    s = freshState(0, PRNG(6));
    const limp = targetsFrom(new Array(ACT).fill(0).map((_, k) => (k % 3 === 2 ? 1 : 0)));
    let tf = 0; while (!s.done && s.t < 3) { physics(s, limp); tf = s.t; }
    console.log(` knees flexed: ${s.fallen ? 'collapsed' : 'held'} at t=${tf.toFixed(2)}s, height
      ${s.p[2].toFixed(3)}m`);
    // push it and see the friction anchor hold
    s = freshState(0, PRNG(7)); s.v[0] = 0.4;
    for (let i = 0; i < 500 && !s.done; i++) physics(s, tg);
    console.log(` pushed at 0.4 m/s with feet planted: settled to ${s.v[0].toFixed(4)} m/s, slid
      ${s.p[0].toFixed(4)}m`);
  } else if (o.mode === 'reference') {
    console.log('REFERENCE TROT (hand-coded clock) - is the plant walkable?');
    for (let st = 0; st < STAGES.length; st++) {
      const suite = Array.from({ length: 10 }, (_, i) => caseFor(st, (o.seed + i * 104729) >>> 0));
      rep(`stage ${st} ${STAGES[st].name}`, evaluate(null, suite, referenceController));
    }
  } else {
    console.log(`EVOLVING: N=${o.neurons}, ${o.seconds}s, 12 ternary actuators, no clock` + (o.noThresholds ? ',
      THRESHOLDS FROZEN' : ''));
    const r = evolve(o);
    console.log(` ${r.gen} generations in ${r.secs.toFixed(0)}s, reached stage ${r.frontier}
      (${STAGES[r.frontier].name})`);
    for (const l of r.log) console.log(' ' + l);
    const cov = coverage(r.archive);
    console.log(` archive: ${cov.cells} of ${cov.total} gait cells (${cov.pct.toFixed(0)}%), pool of
      ${r.pool.length}`);
    for (const e of r.pool.slice(0, 6)) {
      console.log(` #${e.id} cell ${e.cell.join('/')}`); robust ${e.robust.toFixed(2)} dist ${e.dist.toFixed(3)}m
        +
        `duty ${e.desc.duty.toFixed(2)} rhythm ${e.desc.rhythm.toFixed(2)} diag ${e.desc.diag.toFixed(2)} ` +
        `peak ${e.meanPeak.toFixed(0)}N inj ${e.injRate * 100}.toFixed(0)}%`);
    }
    if (r.pool.length > 1) {
      let mn = 1;
      for (let i = 0; i < r.pool.length; i++) for (let j = i + 1; j < r.pool.length; j++) mn = Math.min(mn,
        genomeDistance(r.pool[i].brain, r.pool[j].brain));
      console.log(` pool genome spread: min ${mn * 100}.toFixed(1)}%`);
    }
    const brain = r.brain;
    for (let st = 0; st <= r.frontier; st++) {
      const suite = Array.from({ length: 10 }, (_, i) => caseFor(st, (o.seed + 555 + i * 15485863) >>> 0));
      rep(`stage ${st} ${STAGES[st].name}`, evaluate(brain, suite));
    }
    if (o.mode === 'ablate') {
      const suite = Array.from({ length: 10 }, (_, i) => caseFor(r.frontier, (o.seed + 555 + i * 15485863) >>> 0));
      console.log('ABLATIONS at the frontier stage');
      rep('baseline', evaluate(brain, suite));
      const nr = cloneBrain(brain);
      for (let i = 0; i < nr.N; i++) for (let k = 0; k < 3; k++) if (nr.src[i][k] >= SENS) nr.w[i][k] = 0;
      rep('recurrence cut', evaluate(sanitize(nr), suite));
    }
  }
}

```

```

const nt = cloneBrain(brain);
for (let i = 0; i < nt.N; i++) { nt.bias[i] = 0; nt.fireBias[i] = [0, 0, 0]; nt.fireGain[i] = 0.9; }
rep('thresholds reset', evaluate(sanitize(nt), suite));
let live = 0, rec = 0;
for (let i = 0; i < brain.N; i++) for (let k = 0; k < 3; k++) if (brain.w[i][k] !== 0) { live++; if
  (brain.src[i][k] >= SENS) rec++; }
console.log('live slots ${live}/${brain.N * 3}, recurrent ${rec}; decoder fan-in ${brain.dnz.join('/')}`);
}
}
}

```

J.5 quad/viewer.html — quadruped viewer, including the gait diagram.

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Protoneuron Quadruped</title>
<style>
:root{color-scheme:dark;--bg:#06101b;--panel:#0f1b2c;--panel2:#091522;--line:#29415f;--text:#eaf3ff;--muted:#9fb1c8;
--cyan:#53d9ff;--green:#55e58b;--gold:#ffd166;--red:#ff6f7d;--violet:#b9a0ff}
*{box-sizing:border-box}body{margin:0;background:radial-gradient(circle at top,#112540,#06101b
62%);font-family:Segoe UI,Arial,sans-serif;color:var(--text)}
.wrap{max-width:1500px;margin:auto;padding:16px}h1{margin:0;font-size:26px}h3{margin:4px 0 10px;font-size:15px}
.sub{color:var(--muted);line-height:1.5;margin:5px 0 14px}
.grid{display:grid;grid-template-columns:1.25fr
.75fr;gap:14px}@media(max-width:1080px){.grid{grid-template-columns:1fr}}
.card{background:rgba(15,27,44,.97);border:1px solid var(--line);border-radius:14px;padding:14px;box-shadow:0 12px
30px #0005}
.controls{display:flex;flex-wrap:wrap;gap:7px;margin:10px 0}
button,input,select{border:1px solid var(--line);border-radius:8px;padding:9px
11px;background:#142844;color:var(--text);font-size:13px}
button{cursor:pointer;font-weight:700}button.primary{background:var(--cyan);color:#04121b}button.good{background:var
(--green);color:#062014}button.bad{background:var(--red);color:#25040a}
button.disabled{opacity:.4;cursor:not-allowed}input{width:84px}
label{font-size:12px;color:var(--muted);display:inline-flex;align-items:center;gap:5px}
small{font-size:12px;color:var(--muted)}.tiny{font-size:11px;color:var(--muted)}
.metrics{display:grid;grid-template-columns:repeat(7,1fr);gap:7px;margin:10px 0}
@media(max-width:950px){.metrics{grid-template-columns:repeat(4,1fr)}}
.m{background:var(--panel2);border:1px solid var(--line);border-radius:9px;padding:8px}
.m span{display:block;color:var(--muted);font-size:11px}.m b{font-size:17px}
canvas{display:block;width:100%;height:auto;background:#081321;border:1px solid var(--line);border-radius:10px}
pre{background:#07101d;border:1px solid
var(--line);border-radius:9px;padding:10px;max-height:150px;overflow:auto;white-space:pre-wrap;font:11.5px
Consolas,monospace;margin:0}
textarea{width:100%;min-height:60px;background:#07101d;color:var(--text);border:1px solid
var(--line);border-radius:9px;padding:9px;font:11px Consolas,monospace}
.status{padding:9px 10px;border-radius:9px;background:#081523;border:1px solid
var(--line);min-height:38px;font-size:13px}
table{width:100%;border-collapse:collapse;font-size:12px}
th{text-align:left;color:var(--muted);font-weight:600;padding:5px 8px;border-bottom:1px solid var(--line)}
td{padding:5px 8px;border-bottom:1px solid #16273c}
.bar{height:8px;border-radius:999px;background:#0b1a2b;border:1px solid var(--line);overflow:hidden;margin:6px 0}
.bar i{display:block;height:100%;background:linear-gradient(90deg,var(--cyan),var(--green));width:0;transition:width
.4s}
.ok{color:var(--green)}.warn{color:var(--gold)}.bad{color:var(--red)}.dim{color:var(--muted)}
</style>
</head>
<body><div class="wrap">
<h1>Protoneuron Quadruped</h1>
<div class="sub">A very simple little dog: 3 kg rigid torso with full 6-DOF dynamics, four massless two-segment legs
with hip swing, abduction and a knee, and feet that touch the ground through a spring-damper with stick-slip
friction. Ground reaction is the only thing that can push it forward. The controller is the same ternary
protoneuron population as the pendulum and biped &mdash; 3 slots per neuron, weights and activations in
&minus;1/0/+1 &mdash; and it gets <b>no clock</b>, so any gait rhythm has to come out of its own
recurrence.</div>

<div class="grid">
<div class="card">
<canvas id="view" width="900" height="430"></canvas>
<div class="controls">
<button class="primary" id="train">Train</button>
<button id="pause">Pause</button>
<button class="good" id="watch">Watch</button>
<button id="report">Score all stages</button>
<button class="bad" id="reset">Reset</button>
</div>
<div class="controls">
<label>Neurons <input id="neurons" type="number" min="8" max="256" step="4" value="32"></label>
<label>Seed <input id="seed" type="number" value="1000"></label>
<label>Cases <input id="cases" type="number" min="4" max="24" value="6"></label>
<label>Candidates <input id="cands" type="number" min="2" max="16" value="8"></label>
<label>Mutations <input id="mut" type="number" min="1" max="12" value="3"></label>
</div>

```

```

<div class="controls">
  <label>Watch stage <select id="stage"></select></label>
  <label>Policy <select id="policy"><option value="evolved">evolved brain</option><option
    value="reference">hand-tuned crawl</option></select></label>
  <label>Ablation <select id="ablate">
    <option value="none">none</option>
    <option value="rec">cut all recurrence</option>
    <option value="thr">reset all thresholds</option></select></label>
</div>
<div class="metrics">
  <div class="m"><span>Mode</span><b id="mMode">idle</b></div>
  <div class="m"><span>Generation</span><b id="mGen">0</b></div>
  <div class="m"><span>Curriculum</span><b id="mStage">stand</b></div>
  <div class="m"><span>Distance</span><b id="mDist">0.000 m</b></div>
  <div class="m"><span>Speed</span><b id="mSpeed">0.00</b></div>
  <div class="m"><span>Tilt</span><b id="mTilt">0.00</b></div>
  <div class="m"><span>Peak force</span><b id="mF">0 N</b></div>
</div>
<h3>Gait diagram &mdash; which feet are on the ground</h3>
<canvas id="gait" width="900" height="130"></canvas>
<div class="tiny" style="margin-top:6px">Each row is one foot, filled where it is loaded. A walking dog shows a
  repeating pattern with three feet down at a time; a shuffle shows all four down almost always; noise means no
  rhythm at all. This is the clearest test of whether the population found a four-phase cycle.</div>
<div class="status" id="status" style="margin-top:10px">Press Train. Standing comes in a minute or so; forward
  motion takes much longer.</div>
</div>

<div class="card">
  <h3>Curriculum</h3>
  <div class="bar"><i id="stagebar"></i></div>
  <div id="stages"></div>
  <h3 style="margin-top:14px">Live connectome</h3>
  <canvas id="net" width="420" height="250"></canvas>
  <div class="tiny" style="margin-top:6px">Green = +1 firing, red = &minus;1, gray = 0. Edges are live recurrent
    connections.</div>
  <h3 style="margin-top:14px">Log</h3>
  <pre id="log">ready</pre>
</div>
</div>

<div class="card" style="margin-top:14px">
  <h3>Stage report</h3>
  <div class="tiny" style="margin-bottom:8px">Ten fresh cases per rung with the current ablation applied to a copy.
    Score it with ablation <b>none</b>, then <b>cut all recurrence</b>: on this task recurrence is worth about 95%
    of the performance, and you can watch it collapse.</div>
  <table><thead><tr><th>rung</th><th>distance</th><th>time</th><th>score</th><th>injured</th><th>drift</th><th>peak
    force</th></tr></thead><tbody id="rep"><tr><td colspan="7" class="dim">not scored yet</td></tr></tbody></table>
  <h3 style="margin-top:14px">Save / load champion</h3>
  <div class="controls"><button id="save">Write to box</button><button id="load">Load from box</button><button
    id="download">Download</button></div>
  <textarea id="box" spellcheck="false" placeholder="Champion JSON appears here."></textarea>
  <div class="status" id="io" style="margin-top:8px">Nothing saved this session.</div>
</div>
</div>

<script src="quad4.js"></script>
<script>
const Q = window.QUAD;
const $ = id => document.getElementById(id);
const setText = (id, v) => { const e = $(id); if (e) e.textContent = v; else console.warn('missing #' + id); };
const setHTML = (id, v) => { const e = $(id); if (e) e.innerHTML = v; else console.warn('missing #' + id); };
const vctx = $('view').getContext('2d'), gctx = $('gait').getContext('2d'), nctx = $('net').getContext('2d');

let T = null, mode = 'idle', ws = null, wprev = [], firing = [];
let prints = [], trail = [], gaitHist = [], trainTimer = 0, logLines = [], brainForWatch = null;

function say(m) { logLines.push(m); logLines = logLines.slice(-120); setText('log', logLines.join('\n')); const e =
  $('log'); if (e) e.scrollTop = e.scrollHeight; }
const opts = () => ({ seed: $('seed').value || 1000, neurons: Math.max(8, $('neurons').value || 32),
  cases: Math.max(4, $('cases').value || 6), cands: Math.max(2, $('cands').value || 8),
  mut: Math.max(1, $('mut').value || 3), rotate: 20, decDensity: 0.4 });

function applyAblation(b) {
  const kind = $('ablate').value;
  if (!b || kind === 'none') return b;
  const c = Q.cloneBrain(b);
  if (kind === 'rec') { for (let i = 0; i < c.N; i++) for (let k = 0; k < 3; k++) if (c.src[i][k] >= Q.SENS)
    c.w[i][k] = 0; }
  if (kind === 'thr') { for (let i = 0; i < c.N; i++) { c.bias[i] = 0; c.fireBias[i] = [0, 0, 0]; c.fireGain[i] =
    0.9; } }
  return Q.sanitize(c);
}

function initTrainer() {
  const o = opts(), rng = Q.PRNG(o.seed);
  T = { o, rng, net: Q.makeBrain(o.neurons, rng, o.decDensity), frontier: 0, gen: 0, epoch: 0, best: null };
}

```

```

T.suite = Q.buildSuite(0, o.seed, o.cases, rng);
say(`fresh brain: ${o.neurons} protoneurons, ${o.neurons * 3} slots, 12 ternary actuators, seed ${o.seed}`);
}
function oneGeneration() {
  if (!T) initTrainer();
  const o = T.o;
  if (T.gen % o.rotate === 0) { T.epoch++; T.suite = Q.buildSuite(T.frontier, o.seed + T.epoch * 2654435761,
    o.cases, T.rng); }
  let wF = Q.fitness(T.net, T.suite), winner = T.net;
  for (let i = 0; i < o.cands; i++) {
    const c = Q.mutate(T.net, T.rng, o), f = Q.fitness(c, T.suite);
    if (f.f > wF.f) { winner = c; wF = f; }
  }
  T.net = winner; T.gen++;
  if (T.gen % 8 === 0) {
    const probe = Q.buildSuite(T.frontier, o.seed + 777 + T.gen, Math.max(8, o.cases), T.rng);
    const e = Q.evaluate(T.net, probe), fs = e.stageScore[T.frontier];
    if (fs !== undefined && fs >= Q.STAGES[T.frontier].pass && T.frontier < Q.STAGES.length - 1) {
      T.frontier++; say(`gen ${T.gen}: promoted to ${Q.STAGES[T.frontier].name} (${Q.STAGES[T.frontier] - 1}.name)
        scored ${fs.toFixed(2)})`);
      $('stage').value = String(T.frontier);
    }
    if (!T.best || e.score > T.best.score) T.best = { brain: Q.cloneBrain(T.net), score: e.score, gen: T.gen };
    say(`gen ${T.gen}: ${Q.STAGES[T.frontier].name} score ${e.score.toFixed(3)} dist ${e.dist.toFixed(3)}m injured
      ${e.injRate * 100}.toFixed(0)}%`);
  }
  drawStages(); updateMetrics();
}
function trainLoop() {
  if (mode !== 'train') return;
  try { oneGeneration(); } catch (e) { say('training error: ' + e.message); mode = 'idle'; return; }
  trainTimer = setTimeout(trainLoop, 0);
}
function startWatch() {
  const ref = $('policy').value === 'reference';
  const src = ref ? null : (T && (T.best ? T.best.brain : T.net));
  if (!ref && !src) { setText('status', 'Train first, or switch Policy to the hand-tuned crawl.');
```

```

  return; }
  brainForWatch = src ? applyAblation(src) : null;
  const stage = $('stage').value || 0;
  ws = Q.cloneState(Q.caseFor(stage, (opts).seed + 555 + Math.floor(Math.random() * 1e6)) >>> 0));
  wprev = new Array(brainForWatch ? brainForWatch.N : 0).fill(0);
  prints = []; trail = []; gaitHist = []; firing = [];
  mode = 'watch';
  const ab = $('ablate').value;
  setText('status', `Watching ${ref ? 'the hand-tuned crawl' : 'the evolved brain'} on "${Q.STAGES[stage].name}"`
    + (ab === 'none' || ref ? '' : ab === 'rec' ? ' with every recurrent connection cut' : ' with all thresholds
      reset') + '.');
}
function stepWatch() {
  if (!ws || ws.done) return;
  for (let i = 0; i < 8 && !ws.done && ws.t < ws.T; i++) {
    const before = ws.contact.slice();
    const r = Q.controlStep(brainForWatch, ws, wprev, brainForWatch ? null : Q.referenceController);
    wprev = r.prev; if (brainForWatch) firing = r.prev;
    for (let k = 0; k < 4; k++) if (!before[k] && ws.contact[k]) prints.push({ p: Q.legPoints(ws, k)[2].slice(),
      leg: k });
  }
  gaitHist.push(ws.contact.slice());
  if (gaitHist.length > 420) gaitHist.shift();
  if (prints.length > 120) prints = prints.slice(-120);
  trail.push([ws.p[0], ws.p[1], ws.p[2]]);
  if (trail.length > 360) trail.shift();
  if (ws.done || ws.t >= ws.T) {
    setText('status', ws.injured ? `Injured: a foot exceeded ${Q.FMAX.toFixed(0)}N at ${ws.t.toFixed(2)}s.`
      : ws.fallen ? `Fell at ${ws.t.toFixed(2)}s after ${ws.p[0].toFixed(3)}m.`
      : `Completed ${ws.T}s: ${ws.p[0].toFixed(3)}m, peak force ${ws.peakF.toFixed(0)}N, no injury.`);
  }
}
// ---- 3D ----
function project(q, cam, tgt, w2, H2, f) {
  const fw = [tgt[0] - cam[0], tgt[1] - cam[1], tgt[2] - cam[2]];
  let n = Math.hypot(fw[0], fw[1], fw[2]) || 1; fw[0] /= n; fw[1] /= n; fw[2] /= n;
  let rt = [fw[1], -fw[0], 0]; n = Math.hypot(rt[0], rt[1], rt[2]) || 1; rt = [rt[0] / n, rt[1] / n, 0];
  const up = [rt[1] * fw[2] - rt[2] * fw[1], rt[2] * fw[0] - rt[0] * fw[2], rt[0] * fw[1] - rt[1] * fw[0]];
  const d = [q[0] - cam[0], q[1] - cam[1], q[2] - cam[2]];
  const ze = d[0] * fw[0] + d[1] * fw[1] + d[2] * fw[2];
  if (ze < 0.04) return null;
  return [w2 + f * (d[0] * rt[0] + d[1] * rt[1] + d[2] * rt[2]) / ze,
    H2 - f * (d[0] * up[0] + d[1] * up[1] + d[2] * up[2]) / ze, ze];
}
function drawView() {
  const c = vctx, Wd = $('view').width, Ht = $('view').height;
  c.clearRect(0, 0, Wd, Ht);

```

```

const px = ws ? ws.p[0] : 0;
const cam = [px - 0.72, -0.60, 0.36], tgt = [px + 0.12, 0, 0.11];
const f = 0.62 * Wd, W2 = Wd / 2, H2 = Ht / 2 + 40;
const P = q => project(q, cam, tgt, W2, H2, f);
c.lineWidth = 1;
for (let gx = Math.floor((px - 0.6) * 10) / 10; gx <= px + 1.6; gx += 0.1) {
  const a = P([gx, -0.55, 0]), b = P([gx, 0.55, 0]);
  if (a && b) { c.strokeStyle = '#16273c'; c.beginPath(); c.moveTo(a[0], a[1]); c.lineTo(b[0], b[1]); c.stroke(); }
}
for (let gy = -0.5; gy <= 0.5; gy += 0.1) {
  const a = P([px - 0.6, gy, 0]), b = P([px + 1.6, gy, 0]);
  if (a && b) { c.strokeStyle = Math.abs(gy) < 1e-6 ? '#2c4a6b' : '#14243a'; c.beginPath(); c.moveTo(a[0], a[1]); c.lineTo(b[0], b[1]); c.stroke(); }
}
if (!ws) { c.fillStyle = '#9fb1c8'; c.font = '15px Segoe UI'; c.fillText('Press Train, then Watch.', 24, 32); return; }
for (const fp of prints) {
  const a = P(fp.p); if (!a) continue;
  c.fillStyle = fp.leg < 2 ? '#2e5f7e' : '#7e5a2e';
  c.beginPath(); c.arc(a[0], a[1], Math.max(1.5, 0.012 * f / a[2]), 0, Math.PI * 2); c.fill();
}
c.strokeStyle = '#2a4a6b'; c.lineWidth = 1.5; c.beginPath();
let st = false;
for (const q of trail) { const a = P(q); if (!a) { st = false; continue; } if (!st) { c.moveTo(a[0], a[1]); st = true; } else c.lineTo(a[0], a[1]); }
c.stroke();
// torso box
const hl = 0.15, hw = 0.08, hh = 0.045;
const corners = [[hl, hw, hh], [hl, -hw, hh], [-hl, -hw, hh], [-hl, hw, hh],
  [hl, hw, -hh], [hl, -hw, -hh], [-hl, -hw, -hh], [-hl, hw, -hh]];
const wc = corners.map(b => { const w = Q.mul(ws.R, b); return P([ws.p[0] + w[0], ws.p[1] + w[1], ws.p[2] + w[2]]); });
const edges = [[0, 1], [1, 2], [2, 3], [3, 0], [4, 5], [5, 6], [6, 7], [7, 4], [0, 4], [1, 5], [2, 6], [3, 7]];
if (wc[0] && wc[1] && wc[2] && wc[3]) {
  c.fillStyle = ws.injured ? 'rgba(255,111,125,.28)' : 'rgba(127,182,221,.22)';
  c.beginPath(); c.moveTo(wc[0][0], wc[0][1]);
  for (const i of [1, 2, 3]) c.lineTo(wc[i][0], wc[i][1]);
  c.closePath(); c.fill();
}
c.strokeStyle = ws.injured ? '#ff6f7d' : '#7fb6dd'; c.lineWidth = 2;
for (const [a, b] of edges) if (wc[a] && wc[b]) { c.beginPath(); c.moveTo(wc[a][0], wc[a][1]); c.lineTo(wc[b][0], wc[b][1]); c.stroke(); }
// legs: hip to knee to foot, foot filled when loaded
for (let i = 0; i < 4; i++) {
  const pts = Q.legPoints(ws, i).map(P);
  if (!pts[0] || !pts[1] || !pts[2]) continue;
  c.strokeStyle = i < 2 ? '#53d9ff' : '#ffd166';
  c.lineWidth = Math.max(1.6, 0.009 * f / pts[1][2]);
  c.beginPath(); c.moveTo(pts[0][0], pts[0][1]); c.lineTo(pts[1][0], pts[1][1]); c.lineTo(pts[2][0], pts[2][1]); c.stroke();
  c.fillStyle = '#20364f'; c.beginPath(); c.arc(pts[1][0], pts[1][1], Math.max(1.5, 0.006 * f / pts[1][2]), 0, Math.PI * 2); c.fill();
  c.fillStyle = ws.contact[i] ? '#55e58b' : '#43566d';
  c.beginPath(); c.arc(pts[2][0], pts[2][1], Math.max(2, 0.009 * f / pts[2][2]), 0, Math.PI * 2); c.fill();
}
c.fillStyle = '#eaf3ff'; c.font = '14px Segoe UI';
c.fillText(`t ${ws.t.toFixed(2)}s x ${ws.p[0].toFixed(3)}m v ${ws.v[0].toFixed(3)} m/s target
  ${ws.vTarget.toFixed(2)}`, 18, 26);
c.fillStyle = '#9fb1c8'; c.font = '12px Segoe UI';
c.fillText(`height ${ws.p[2].toFixed(3)}m tilt ${ws.tilt || 0}.toFixed(3)}rad feet down
  ${ws.contact.join('')} peak ${ws.peakF.toFixed(0)}N`, 18, 46);
if (ws.injured) { c.fillStyle = '#ff6f7d'; c.font = '15px Segoe UI'; c.fillText('INJURED', Wd - 92, 26); }
else if (ws.fallen) { c.fillStyle = '#ffd166'; c.font = '15px Segoe UI'; c.fillText('FELL', Wd - 62, 26); }
}
function drawGait() {
  const c = gctx, Wd = $('gait').width, Ht = $('gait').height;
  c.clearRect(0, 0, Wd, Ht);
  const names = ['front left', 'front right', 'rear left', 'rear right'];
  const rowH = (Ht - 16) / 4, x0 = 86;
  c.font = '11px Segoe UI';
  for (let i = 0; i < 4; i++) {
    const y = 8 + i * rowH;
    c.fillStyle = '#9fb1c8'; c.fillText(names[i], 8, y + rowH * 0.62);
    c.strokeStyle = '#16273c'; c.beginPath(); c.moveTo(x0, y + rowH - 2); c.lineTo(Wd - 8, y + rowH - 2); c.stroke();
    c.fillStyle = i < 2 ? '#53d9ff' : '#ffd166';
    const w = (Wd - 8 - x0) / 420;
    for (let k = 0; k < gaitHist.length; k++) if (gaitHist[k][i]) c.fillRect(x0 + k * w, y + 2, Math.max(1, w), rowH - 6);
  }
  if (!gaitHist.length) { c.fillStyle = '#9fb1c8'; c.fillText('press Watch to record a gait', x0 + 10, Ht / 2); }
}
function drawNet() {
  const b = brainForWatch || (T && (T.best ? T.best.brain : T.net));

```

```

const c = nctx, Wd = $('net').width, Ht = $('net').height;
c.clearRect(0, 0, Wd, Ht);
if (!b) { c.fillStyle = '#9fb1c8'; c.font = '12px Segoe UI'; c.fillText('no brain yet', 14, 22); return; }
const cx = Wd / 2, cy = Ht / 2, rad = Math.min(Wd, Ht) * 0.37, pts = [];
for (let i = 0; i < b.N; i++) { const a = (i / b.N) * Math.PI * 2 - Math.PI / 2; pts.push([cx + Math.cos(a) * rad,
  cy + Math.sin(a) * rad]); }
for (let i = 0; i < b.N; i++) for (let k = 0; k < 3; k++) {
  const s = b.src[i][k], w = b.w[i][k];
  if (s < Q.SENS || w === 0) continue;
  const a = pts[s - Q.SENS], d = pts[i], on = firing.length && firing[s - Q.SENS] !== 0;
  c.strokeStyle = w > 0 ? '#55e58b' : '#ff6f7d'; c.globalAlpha = on ? 0.8 : 0.2; c.lineWidth = on ? 1.5 : 1;
  c.beginPath(); c.moveTo(a[0], a[1]); c.lineTo(d[0], d[1]); c.stroke();
}
c.globalAlpha = 1;
for (let i = 0; i < b.N; i++) {
  const v = firing.length ? firing[i] : 0;
  c.fillStyle = v > 0 ? '#55e58b' : v < 0 ? '#ff6f7d' : '#52647c';
  c.beginPath(); c.arc(pts[i][0], pts[i][1], Math.max(3, 60 / b.N + 2), 0, Math.PI * 2); c.fill();
}
let live = 0, rec = 0;
for (let i = 0; i < b.N; i++) for (let k = 0; k < 3; k++) if (b.w[i][k] !== 0) { live++; if (b.src[i][k] >=
  Q.SENS) rec++; }
c.fillStyle = '#9fb1c8'; c.font = '11px Segoe UI';
c.fillText(`N=${b.N} · ${live}/${b.N * 3} live · ${rec} recurrent`, 12, Ht - 8);
}
function drawStages() {
  const fr = T ? T.frontier : 0;
  setHTML('stages', Q.STAGES.map((s, i) => {
    const st = i < fr ? 'ok' : i === fr ? 'warn' : 'dim', tag = i < fr ? 'passed' : i === fr ? 'training' :
      'locked';
    return `<div style="display:flex;justify-content:space-between;padding:4px 0;border-bottom:1px solid #16273c">
      <span><b>${i}</b> </span><span class="small">${s.vTarget.toFixed(2)} m/s · ${s.T}s · kick
        &times;${s.perturb}</span>
      <span class="${st}">${tag}</span></div>`;
  })).join('');
  const e = $('stagebar'); if (e) e.style.width = (100 * fr / (Q.STAGES.length - 1)) + '%';
}
function updateMetrics() {
  setText('mMode', mode); setText('mGen', T ? T.gen : 0);
  setText('mStage', Q.STAGES[T ? T.frontier : 0].name);
  setText('mDist', (ws ? ws.p[0] : 0).toFixed(3) + ' m');
  setText('mSpeed', (ws ? ws.v[0] : 0).toFixed(3));
  setText('mTilt', (ws && ws.tilt ? ws.tilt : 0).toFixed(3));
  setText('mF', (ws ? ws.peakF : 0).toFixed(0) + ' N');
  const w = $('watch'); if (w) w.disabled = !(T || $('policy').value === 'reference');
}
function scoreAll() {
  const ref = $('policy').value === 'reference';
  const src = ref ? null : (T && (T.best ? T.best.brain : T.net));
  if (!ref && !src) { setText('status', 'Train first, or score the hand-tuned crawl.');
```

```

        fireGain: d.fireGain });
    if (!T) initTrainer();
    T.net = b; T.best = { brain: Q.cloneBrain(b), score: 0, gen: d.gen || 0 };
    T.frontier = Math.min(Q.STAGES.length - 1, d.frontier || 0);
    $('#neurons').value = b.N; $('#stage').value = String(T.frontier);
    drawStages(); updateMetrics(); setText('io', `Loaded a ${b.N}-neuron champion. Press Watch.`);
  } catch (e) { setText('io', 'Could not load: ' + e.message); }
};
$('#download').onclick = () => {
  if (!$('box').value) $('#save').onclick();
  if (!$('box').value) return;
  try {
    const bl = new Blob(['box').value], { type: 'application/json' }); u = URL.createObjectURL(bl), a =
      document.createElement('a');
    a.href = u; a.download = 'quad-champion.json'; document.body.appendChild(a); a.click(); a.remove();
    setTimeout(() => URL.revokeObjectURL(u), 2000); setText('io', 'Download started.');
```

```

  } catch (e) { setText('io', 'Download blocked – copy the box contents instead.');
```

J.6 sweep/engine.js — loads the pendulum lab headlessly under Node and instruments it with compute counters.

```

// Loads the lab's simulator into node with a stubbed DOM, and instruments it with
// counters so compute can be measured in hardware-independent units.
const fs = require('fs');

const ANCHOR = " const plan=perturbSchedule(s.caseSeed||1);\n while(!s.done&&MAXT){\n";
const PATCH = " const plan=perturbSchedule(s.caseSeed||1);__C.ep++;__C.steps++;\n while(!s.done&&MAXT){__C.steps++;\n";

function loadEngine(htmlPath) {
  if (!fs.existsSync(htmlPath)) {
    throw new Error('lab HTML not found at ' + htmlPath +
      ' - pass --html with the full path to protoneuron-champion-lab-v12.html');
  }
  const html = fs.readFileSync(htmlPath, 'utf8');
  const m = html.match(/<script>([\s\S]*)</script>/);
  if (!m) throw new Error('no <script> block found in ' + htmlPath);
  let js = m[1];

  const hits = js.split(ANCHOR).length - 1;
  if (hits !== 1) {
    throw new Error(
      `instrumentation anchor matched ${hits} times, expected 1. The engine source ` +
      `changed; fix ANCHOR in engine.js before trusting any compute numbers.`);
  }
  js = js.replace(ANCHOR, PATCH);

  // default input values straight out of the markup, so the stub starts where the UI does
  const vals = {};
  for (const mm of html.matchAll(/<input id="([A-Za-z0-9_]+)"[^>]*value="([^\"]*)"/g)) vals[mm[1]] = mm[2];

  const ctx = new Proxy({}, { get: (t, k) => (k === 'canvas' ? { width: 900, height: 420 } : () => {}) });
  const els = {};
  const el = (id) => {
    if (els[id]) return els[id];
    els[id] = {
      id, value: vals[id] ?? '', textContent: '', innerHTML: '', style: {}, disabled: false,
      firstChild: null, getContext: () => ctx, setAttribute() {}, getAttribute() {},
      appendChild() {}, removeChild() {}, select() {},
    };
    return els[id];
  };

  const sandbox = {
    __C: { ep: 0, steps: 0 },
    document: {
      getElementById: el,
      createElementNS: () => ({ setAttribute() {}, textContent: '' }),
      createElement: () => ({ style: {}, click() {}, remove() {}, setAttribute() {}, select() {} }),
      body: { appendChild() {}, removeChild() {} },
      execCommand: () => false,
    },
    window: {},
    performance: { now: () => Date.now() },
    requestAnimationFrame: () => {},
    setTimeout: () => 0,
  };

```

```

clearTimeout: () => {},
navigator: { clipboard: { writeText: async () => {} } },
localStorage: (() => { const s = {}; return { getItem: k => s[k] ?? null, setItem: (k, v) => { s[k] = v; },
  removeItem: k => { delete s[k]; } }; })(),
URL: { createObjectURL: () => 'blob:x', revokeObjectURL() {} },
Blob: class { constructor(a) { this.a = a; } },
};
for (const k of Object.keys(sandbox)) global[k] = sandbox[k];

let api;
eval(js + `
;api = {
  els,
  counters: __C,
  get CFG(){return CFG}, get net(){return net}, get champions(){return champions},
  get archive(){return archive}, get generation(){return generation},
  get validationPool(){return validationPool}, get holdoutPool(){return holdoutPool},
  get trainingPool(){return trainingPool}, get numericalFaults(){return numericalFaults},
  set mode(v){mode=v}, set paused(v){paused=v}, set running(v){running=v},
  readCfg, buildSuites, startTrain, trainGeneration, scoreSuite, scoreSet, coverage, hofDiversity,
  packBrain, unpackBrain, serializeAll,
};`);
return api;
}

module.exports = { loadEngine };

```

J.7 sweep/trial.js — one trial: criterion ladder, perturbation stress ladder, champion genome.

```

// Runs one (neurons, seed) trial and prints one JSONL record to stdout.
//
// node trial.js --html ../protoneuron-champion-lab-v12.html --N 128 --seed 47017
//
// Compute is recorded in four currencies at every criterion crossing, because none of
// them is neutral across neuron counts: fixed generations favours large N, fixed wall
// clock favours small N, and episodes ignore that a step costs O(N).
const { loadEngine } = require('./engine.js');

// The lab HTML may sit beside these scripts (zip layout) or one level up (checkout
// layout), so resolve rather than assume.
function findLab(explicit) {
  if (explicit) return explicit;
  const fs = require('fs'), path = require('path');
  const name = 'protoneuron-champion-lab-v12.html';
  for (const d of [__dirname, path.join(__dirname, '..')]) {
    const p = path.join(d, name);
    if (fs.existsSync(p)) return p;
  }
  for (const d of [__dirname, path.join(__dirname, '..')]) {
    try {
      const hit = fs.readdirSync(d).filter(f => /protoneuron.*lab.*\.html$/i.test(f)).sort()[0];
      if (hit) return path.join(d, hit);
    } catch {}
  }
  return path.join(__dirname, name);
}

const argv = {};
for (let i = 2; i < process.argv.length; i += 2) argv[process.argv[i].replace(/^--/, '')] = process.argv[i + 1];

const HTML = findLab(argv.html);
const N = +(argv.N || 128);
const SEED = +(argv.seed || 47017);
const BUDGET = +(argv.budget || 1e10); // neuron-updates, the fairest cross-N currency
const WALL = +(argv.wall || 3600); // seconds, safety cap only
const FANIN = argv.fanin || 'fixed'; // fixed = ~15 voters per actuator at every N
const FIXEDFAN = +(argv.fixedfan || 15);
const MUTSCALE = argv.mutscale || 'on'; // on = mutations scale with genome size
const PROGRESS = +(argv.progress || 0); // seconds between stderr progress lines, 0 = off

// survival fractions on the 48-case validation suite, plus "perfect" = the robust ceiling
const LADDER = [0.10, 0.25, 0.50, 0.75, 0.90, 1.00];
// Perturbation headroom. Every evaluation already kicks the pendulum at x1, so a solve is
// already perturbation-tolerant; this asks how much harder it can be kicked before failing.
const STRESS = [1, 1.5, 2, 2.5, 3, 4];
// Robust-score rungs matter because survival is an all-or-nothing wall: a night of compute
// that never reaches survival >= 10% would otherwise produce an entirely censored table.
const ROBRUNGS = [2, 4, 6, 9, 11];
const A = loadEngine(HTML);

// tested defaults, then the sweep's own overrides
const cfg = {
  seed: SEED, neurons: N, torque: 22, focus: 85, sensorBias: 50, rotate: 25,
  trainCases: 12, validationTrials: 48, valEvery: 3, poolSize: 8, bins: 5,

```

```

    minDist: 4, novelty: 0, candidates: 8, strongEvery: 20,
    decDensity: FANIN === 'fixed' ? Math.min(100, Math.max(1, Math.round(100 * FIXEDFAN / N))) : 12,
    mutations: MUTSCALE === 'on' ? Math.max(2, Math.round(N / 32)) : 4,
  };
  for (const [k, v] of Object.entries(cfg)) { if (A.els[k]) A.els[k].value = String(v); }
  A.readCfg();
  A.startTrain(false);

  // The lab clamps neuron count internally. If it clamped, the record would claim an N the
  // simulator never ran, so fail loudly instead of writing a mislabelled row.
  if (A.net.N !== N) {
    process.stderr.write(`requested N=${N} but the engine built N=${A.net.N}; refusing to ` +
      `write a mislabelled record. Widen the clamp in the lab HTML.\n`);
    process.exit(3);
  }

  const t0 = Date.now();
  let lastProgress = 0;
  const cross = {}; // criterion -> compute snapshot at first crossing
  let bestSurv = 0, bestRobust = 0, perfectAt = null, holdoutAtPerfect = null;

  const snapshot = () => ({
    gen: A.generation,
    episodes: A.counters.ep,
    steps: A.counters.steps,
    neuronUpdates: A.counters.steps * N,
    wallSec: +(Date.now() - t0) / 1000).toFixed(2),
  });

  while (true) {
    A.mode = 'train'; A.paused = false; A.running = true;
    A.trainGeneration();

    for (const c of A.champions) {
      if (c.validation.survival > bestSurv) bestSurv = c.validation.survival;
      if (c.validation.robust > bestRobust) bestRobust = c.validation.robust;
    }
    for (const lvl of LADDER) {
      const key = 'surv' + Math.round(lvl * 100);
      if (!cross[key] && bestSurv >= lvl - 1e-9) cross[key] = snapshot();
    }
    for (const lvl of ROBRUNGS) {
      const key = 'rob' + lvl;
      if (!cross[key] && bestRobust >= lvl - 1e-9) cross[key] = snapshot();
    }
    if (!cross.perfect && bestRobust >= 12.999) {
      cross.perfect = snapshot();
      perfectAt = A.generation;
      const top = A.champions[0];
      const h = A.scoreSuite(top.brain, A.holdoutPool);
      holdoutAtPerfect = { mean: +h.mean.toFixed(3), worst: +h.worst.toFixed(3), survival: +h.survival.toFixed(3),
        robust: +h.robust.toFixed(3) };
    }

    const s = snapshot();
    if (PROGRESS && Date.now() - lastProgress > PROGRESS * 1000) {
      lastProgress = Date.now();
      process.stderr.write(JSON.stringify({ progress: true, N, seed: SEED, gen: s.gen,
        neuronUpdates: s.neuronUpdates, wallSec: s.wallSec,
        bestRobust: +bestRobust.toFixed(3), bestSurv: +bestSurv.toFixed(3) }) + '\n');
    }
    if (perfectAt) break; // hardest criterion met, nothing left to time
    if (s.neuronUpdates >= BUDGET) break;
    if (s.wallSec >= WALL) break;
  }

  // ---- perturbation stress on the holdout suite, plus the genome for the viewer ----
  let stress = null, breakMul = null, champion = null;
  if (A.champions.length) {
    const top = A.champions[0];
    const orig = A.els.perturbMul.value;
    stress = {};
    for (const mul of STRESS) {
      A.els.perturbMul.value = String(mul);
      A.readCfg();
      const s = A.scoreSuite(top.brain, A.holdoutPool);
      stress[mul] = { mean: +s.mean.toFixed(3), worst: +s.worst.toFixed(3),
        survival: +s.survival.toFixed(3), robust: +s.robust.toFixed(3) };
      if (s.survival >= 0.9) breakMul = mul; // last multiplier still holding 90%
    }
    A.els.perturbMul.value = orig;
    A.readCfg();
    champion = { id: top.id, solvedTrial: !!perfectAt, generation: top.generation, brain: A.packBrain(top.brain),
      validation: { robust: +top.validation.robust.toFixed(4),
        mean: +top.validation.mean.toFixed(3),

```

```

        median: +top.validation.median.toFixed(3),
        worst: +top.validation.worst.toFixed(3),
        survival: +top.validation.survival.toFixed(3),
        total: +top.validation.total.toFixed(3),
        desc: top.validation.desc } });
    }

    const fin = snapshot();
    process.stdout.write(JSON.stringify({
      N, seed: SEED, fanin: FANIN, mutscale: MUTSCALE,
      decDensity: cfg.decDensity, mutations: cfg.mutations,
      fanInActual: A.net.dnz ? A.net.dnz.slice() : null,
      solved: !!perfectAt, bestSurv: +bestSurv.toFixed(4), bestRobust: +bestRobust.toFixed(4),
      cross, final: fin, holdoutAtPerfect, stress, breakMul, champion,
      archiveCells: A.archive.size, poolSize: A.champions.length,
      faults: A.numericalFaults,
      budget: BUDGET, wallCap: WALL,
    }) + '\n');

```

J.8 sweep/analyze.js — compute-to-half-solved with percentile bootstrap.

```

// Compute-to-half-solved, per neuron count.
//
// node analyze.js --in results.jsonl [--unit neuronUpdates] [--crit perfect]
//
// The half-solved point is the compute at which the 50th percentile seed crosses the
// criterion. It is the right statistic for this data because it is identifiable under
// right-censoring: as long as more than half the seeds solve, the median is exact even
// though the slowest seeds never finished. Means are not identifiable here, and means
// over solvers only are survivor-biased.
const fs = require('fs');
const path = require('path');

const argv = {};
for (let i = 2; i < process.argv.length; i += 2) argv[process.argv[i].replace(/^--/, '')] = process.argv[i + 1];
const IN = argv.in || path.join(__dirname, 'results.jsonl');
const CRIT = argv.crit || 'perfect';
const UNITS = ['gen', 'episodes', 'neuronUpdates', 'wallSec'];
const B = 4000;

const rows = fs.readFileSync(IN, 'utf8').split('\n').filter(l => l.trim()).map(l => JSON.parse(l));
if (!rows.length) { console.error('no records in ' + IN); process.exit(1); }

const groups = new Map();
for (const r of rows) {
  const g = `N=${r.N} fanin=${r.fanin} mutscale=${r.mutscale}`;
  if (!groups.has(g)) groups.set(g, []);
  groups.get(g).push(r);
}

// smallest compute at which >= half the seeds have crossed; Infinity if they never do
function halfSolved(vals) {
  const s = vals.slice().sort((a, b) => a - b);
  return s[Math.ceil(s.length / 2) - 1];
}

function boot(vals) {
  const out = [];
  let censored = 0;
  for (let b = 0; b < B; b++) {
    const r = Array.from({ length: vals.length }, () => vals[Math.random() * vals.length | 0]);
    const h = halfSolved(r);
    if (!isFinite(h)) censored++; else out.push(h);
  }
  out.sort((a, b) => a - b);
  return {
    lo: out.length ? out[Math.floor(out.length * 0.05)] : Infinity,
    hi: out.length ? out[Math.floor(out.length * 0.95)] : Infinity,
    censoredFrac: censored / B,
  };
}

const fmt = (v, u) => !isFinite(v) ? 'not reached' :
  u === 'neuronUpdates' ? (v / 1e9).toFixed(2) + 'e9' :
  u === 'wallSec' ? v.toFixed(0) + 's' :
  u === 'episodes' ? (v / 1000).toFixed(1) + 'k' :
  String(Math.round(v));

console.log(`criterion: ${CRIT === 'perfect' ? 'robust' : 'surv'} >= 12.999 (every validation case held to the episode cap)` :
  CRIT`);
console.log(`half-solved = compute at which the median seed crosses it, 90% bootstrap interval over seeds\n`);

const RUNGS = ['rob2', 'rob4', 'rob6', 'rob9', 'rob11', 'perfect', 'surv25', 'surv50', 'surv90', 'surv100'];
const namesAll = [...groups.keys()].sort((x, y) => (+x.match(/N=(\d+)/)[1]) - (+y.match(/N=(\d+)/)[1]));
console.log(`LADDER — half-solved neuron-updates at each rung (rob = robust score, surv = validation survival)`);
console.log(String('group').padEnd(34) + RUNGS.map(k => k.padEnd(12)).join(''));

```

```

for (const g of namesAll) {
  const rs = groups.get(g);
  const cells = RUNGS.map(k => {
    const vals = rs.map(r => r.cross[k] ? r.cross[k].neuronUpdates : Infinity);
    return fmt(halfSolved(vals), 'neuronUpdates').padEnd(12);
  });
  console.log(String(g).padEnd(34) + cells.join(''));
}
console.log('');

const names = [...groups.keys()].sort((a, b) => (+a.match(/N=(\d+)/)[1]) - (+b.match(/N=(\d+)/)[1]));
const table = [];
for (const g of names) {
  const rs = groups.get(g);
  const n = rs.length;
  const nSolved = rs.filter(r => r.cross[CRIT]).length;
  const row = { g, n, rate: nSolved / n };
  for (const u of UNITS) {
    const vals = rs.map(r => r.cross[CRIT] ? r.cross[CRIT][u] : Infinity);
    row[u] = halfSolved(vals);
    if (u === 'neuronUpdates') row.ci = boot(vals);
  }
  // total compute actually spent, censored trials included – the real price of the arm
  row.totalNNU = rs.reduce((a, r) => a + r.final.neuronUpdates, 0);
  row.totalCpuSec = rs.reduce((a, r) => a + r.final.wallSec, 0);
  row.voters = rs.reduce((a, r) => a + (r.fanInActual ? r.fanInActual.reduce((x, y) => x + y, 0) / 3 : 0), 0) / n;
  const hs = rs.filter(r => r.holdoutAtPerfect);
  row.holdout = hs.length ? (hs.reduce((a, r) => a + r.holdoutAtPerfect.survival, 0) / hs.length) : null;
  row.cells = rs.reduce((a, r) => a + r.archiveCells, 0) / n;
  table.push(row);
}

const pad = (s, w) => String(s).padEnd(w);
console.log(pad('group', 34) + pad('seeds', 7) + pad('solve rate', 12) + pad('gens', 13) + pad('episodes', 13) +
  pad('neuron-updates', 17) + pad('wall', 13) + pad('holdout', 10) + pad('voters', 9) + pad('total spent', 14) +
  'cpu-h');
for (const r of table) {
  console.log(pad(r.g, 34) + pad(r.n, 7) + pad((r.rate * 100).toFixed(0) + '%', 12) +
    pad(fmt(r.gen, 'gen'), 13) + pad(fmt(r.episodes, 'episodes'), 13) +
    pad(fmt(r.neuronUpdates, 'neuronUpdates'), 17) + pad(fmt(r.wallSec, 'wallSec'), 13) +
    pad(r.holdout == null ? '-' : (r.holdout * 100).toFixed(0) + '%', 10) +
    pad(r.voters.toFixed(1), 9) + pad((r.totalNNU / 1e9).toFixed(2) + 'e9', 14) +
    (r.totalCpuSec / 3600).toFixed(2));
}

console.log('\n90% bootstrap interval on half-solved neuron-updates:');
for (const r of table) {
  const c = r.ci;
  const note = c.censoredFrac > 0.1 ? ` (${(c.censoredFrac * 100).toFixed(0)}% of resamples never reach half solved
    – treat as a lower bound)` : '';
  console.log(` ${pad(r.g, 34)} ${fmt(c.lo, 'neuronUpdates')} .. ${fmt(c.hi, 'neuronUpdates')}${note}`);
}

const best = table.filter(r => isFinite(r.neuronUpdates)).sort((a, b) => a.neuronUpdates - b.neuronUpdates)[0];
if (best) {
  console.log(`\ncheapest to half solved by neuron-updates: ${best.g}`);
  const overlap = table.filter(r => r !== best && isFinite(r.neuronUpdates) && r.ci.lo <= best.ci.hi);
  if (overlap.length) console.log(`intervals overlap with: ${overlap.map(r => r.g).join(', ')} – not separated at
    this seed count`);
} else {
  console.log('\nno group reached half solved inside the budget. Raise --budget or lower the criterion with --crit
    surv90.');
}

const gNU = table.reduce((a, r) => a + r.totalNNU, 0), gS = table.reduce((a, r) => a + r.totalCpuSec, 0);
const gN = table.reduce((a, r) => a + r.n, 0);
console.log(`\nsweep total: ${gN} trials, ${gNU / 1e9}.toFixed(2)}e9 neuron-updates, ` +
  `${(gS / 3600).toFixed(2)} cpu-hours (summed across workers, not elapsed time)`);
console.log(`\nraking flips with the unit: fixed generations favours large N, wall clock favours small N.`);
console.log('Report all four columns rather than picking the one that makes a point.');
```

J.9 sweep/run.js — parallel sweep coordinator with resume.

```

// Overnight sweep coordinator. Resumable: rerun the same command after a crash or a
// Ctrl-C and it skips trials already present in the output file.
//
// node run.js --Ns 64,128,256 --seeds 30 --budget 4e9 --workers 8
const { spawn } = require('child_process');
const fs = require('fs');
const path = require('path');
const os = require('os');

// The lab HTML may sit beside these scripts (zip layout) or one level up (checkout
// layout), so resolve rather than assume.
function findLab(explicit) {
```

```

    if (explicit) return explicit;
    const fs = require('fs'), path = require('path');
    const name = 'protoneuron-champion-lab-v12.html';
    for (const d of [__dirname, path.join(__dirname, '..')]) {
        const p = path.join(d, name);
        if (fs.existsSync(p)) return p;
    }
    for (const d of [__dirname, path.join(__dirname, '..')]) {
        try {
            const hit = fs.readdirSync(d).filter(f => /protoneuron.*lab.*\.html$/i.test(f)).sort()[0];
            if (hit) return path.join(d, hit);
        } catch (_) {}
    }
    return path.join(__dirname, name);
}

const argv = {};
for (let i = 2; i < process.argv.length; i += 2) argv[process.argv[i].replace(/^--/, '')] = process.argv[i + 1];

const HTML = findLab(argv.html);
const NS = (argv.Ns || '64,128,256').split(',').map(Number);
const SEEDS = +(argv.seeds || 30);
const SEED0 = +(argv.seed0 || 1000);
const BUDGET = argv.budget || '1e10';
const WALL = argv.wall || '3600';
const WORKERS = +(argv.workers || Math.max(1, os.cpus().length - 1));
const OUT = argv.out || path.join(__dirname, 'results.jsonl');
const FANIN = argv.fanin || 'fixed';
const MUTSCALE = argv.mutscale || 'on';

const key = (t) => `${t.N}|${t.seed}|${FANIN}|${MUTSCALE}`;
const done = new Set();
if (fs.existsSync(OUT)) {
    for (const line of fs.readFileSync(OUT, 'utf8').split('\n')) {
        if (!line.trim()) continue;
        try { const r = JSON.parse(line); done.add(`${r.N}|${r.seed}|${r.fanin}|${r.mutscale}`); } catch (_) {}
    }
}
const queue = [];
for (const N of NS) for (let i = 0; i < SEEDS; i++) {
    const t = { N, seed: SEED0 + i * 7919 };
    if (!done.has(key(t))) queue.push(t);
}
const TOTAL = queue.length;
const total = NS.length * SEEDS;
console.log(`${total} trials total, ${done.size} already in ${path.basename(OUT)}, ${queue.length} to run on
${WORKERS} workers`);
if (!queue.length) { console.log('nothing to do'); process.exit(0); }

let started = 0, finished = 0, solved = 0;
const startedAt = Date.now();

function launch() {
    const t = queue.shift();
    if (!t) return false;
    started++;
    const child = spawn(process.execPath, [
        path.join(__dirname, 'trial.js'),
        '--html', HTML, '--N', String(t.N), '--seed', String(t.seed),
        '--budget', BUDGET, '--wall', WALL, '--fanin', FANIN, '--mutscale', MUTSCALE,
    ], { stdio: ['ignore', 'pipe', 'pipe'] });

    let out = '', err = '';
    child.stdout.on('data', d => { out += d; });
    child.stderr.on('data', d => { err += d; });
    child.on('close', (code) => {
        finished++;
        if (code === 0 && out.trim()) {
            fs.appendFileSync(OUT, out.trim() + '\n'); // one line, O_APPEND, safe across workers
            try {
                const r = JSON.parse(out);
                if (r.solved) solved++;
                const c = r.solved ? r.cross.perfect : r.final;
                console.log(`[${finished}/${TOTAL}] N=${r.N} seed=${r.seed} ` +
                    `${r.solved ? 'SOLVED' : 'censored'} surv=${(r.bestSurv * 100).toFixed(0)}% ` +
                    `gen=${c.gen} nu=${(c.neuronUpdates / 1e9).toFixed(2)}e9 ${c.wallSec}s`);
            } catch (_) { console.log(`[${finished}/${TOTAL}] N=${t.N} seed=${t.seed} unparseable output`); }
        } else {
            console.error(`[${finished}/${TOTAL}] N=${t.N} seed=${t.seed} FAILED code=${code} ${err.split('\n')[0]}`);
        }
    });
    if (!launch() && finished === started) {
        const mins = ((Date.now() - startedAt) / 60000).toFixed(1);
        console.log(`\ndone: ${finished} trials, ${solved} solved, ${mins} min. Now: node analyze.js --in
${path.basename(OUT)}`);
    }
}

```

```

    });
    return true;
}
for (let w = 0; w < WORKERS; w++) if (!launch()) break;

```

J.10 sweep/app.py — Flask dashboard: worker pool, live statistics, champion viewer endpoints.

```

"""Flask control panel for the compute-to-half-solved sweep.

    pip install -r requirements.txt
    python app.py
    open http://127.0.0.1:5001

The compute still happens in node (the simulator is JavaScript). This process owns the
queue, a worker pool of `node trial.js` subprocesses, and the live statistics. Results
append to results.jsonl in the same format the CLI produces, so `node analyze.js` and
this dashboard always read the same file.
"""
import json
import math
import os
import random
import shutil
import subprocess
import sys
import threading
import time
from collections import defaultdict
from pathlib import Path

from flask import Flask, jsonify, request, send_from_directory

HERE = Path(__file__).resolve().parent
LAB_NAME = "protoneuron-champion-lab-v12.html"

def find_lab_html():
    """Locate the lab HTML. It may sit beside app.py (the zip layout) or one level up
    (a checkout where the sweep is a subfolder), so try both and then glob."""
    for root in (HERE, HERE.parent):
        p = root / LAB_NAME
        if p.exists():
            return p
    for root in (HERE, HERE.parent):
        hits = sorted(root.glob("protoneuron*lab*.html"))
        if hits:
            return hits[0]
    return HERE / LAB_NAME # missing; the error message will name this path

DEFAULT_HTML = find_lab_html()
RESULTS = HERE / "results.jsonl"
LADDER = ["rob2", "rob4", "rob6", "rob9", "rob11", "perfect",
          "surv25", "surv50", "surv90", "surv100"]
UNITS = ["gen", "episodes", "neuronUpdates", "wallSec"]

app = Flask(__name__, static_folder=str(HERE / "static"), static_url_path="")
class Sweep:
    """Owns the trial queue and the subprocess pool. One instance per process."""

    def __init__(self):
        self.lock = threading.Lock()
        self.reset()

    def reset(self):
        self.state = "idle" # idle | running | stopping | done
        self.cfg = {}
        self.queue = []
        self.running = {} # (N, seed) -> live progress dict
        self.procs = {} # (N, seed) -> Popen
        self.log = []
        self.total = 0
        self.finished = 0
        self.skipped = 0
        self.started_at = None
        self.stop_flag = False
        self.error = None

    # ---- logging -----
    def say(self, msg):
        line = {"t": time.strftime("%H:%M:%S"), "msg": msg}
        with self.lock:
            self.log.append(line)
            del self.log[:-400]

```

```

# ---- lifecycle -----
def start(self, cfg):
    with self.lock:
        if self.state == "running":
            return False, "a sweep is already running"
        html = Path(cfg["html"]).expanduser()
        if not html.exists():
            return False, (f"lab HTML not found at {html} – put the path to "
                           f"{LAB_NAME} in the HTML field")
        if not shutil.which("node"):
            return False, "node is not on PATH; the simulator needs it"

    self.reset()
    self.cfg = cfg
    self.state = "running"
    self.started_at = time.time()

    done = set()
    if cfg["resume"] and RESULTS.exists():
        for line in RESULTS.read_text().splitlines():
            if not line.strip():
                continue
            try:
                r = json.loads(line)
                done.add((r["N"], r["seed"], r["fanin"], r["mutscale"]))
            except Exception:
                pass

    trials = []
    for n in cfg["Ns"]:
        for i in range(cfg["seeds"]):
            seed = cfg["seed0"] + i * 7919
            if (n, seed, cfg["fanin"], cfg["mutscale"]) in done:
                self.skipped += 1
            else:
                trials.append((n, seed))
    self.queue = trials
    self.total = len(trials)
    self.say(f"queued {self.total} trials across N={cfg['Ns']} "
            f"({self.skipped} already in results.jsonl) on {cfg['workers']} workers")
    if not trials:
        self.state = "done"
        self.say("nothing to do – every trial is already in results.jsonl")
        return True, "nothing to run"

    for _ in range(min(cfg["workers"], len(trials))):
        threading.Thread(target=self._worker, daemon=True).start()
    return True, f"started {self.total} trials"

def stop(self):
    with self.lock:
        if self.state != "running":
            return False
        self.state = "stopping"
        self.stop_flag = True
        self.queue = []
        procs = list(self.procs.values())
        for p in procs:
            try:
                p.terminate()
            except Exception:
                pass
        self.say("stop requested – terminating in-flight trials")
        return True

# ---- the pool -----
def _worker(self):
    while True:
        with self.lock:
            if self.stop_flag or not self.queue:
                break
            n, seed = self.queue.pop(0)
            self.running[(n, seed)] = {"N": n, "seed": seed, "gen": 0,
                                       "neuronUpdates": 0, "wallSec": 0,
                                       "bestRobust": 0, "bestSurv": 0}
            self._run_trial(n, seed)

        with self.lock:
            if not self.running and not self.queue and self.state in ("running", "stopping"):
                self.state = "done"
                mins = (time.time() - self.started_at) / 60
                self.say(f"sweep {'stopped' if self.stop_flag else 'complete'}: "
                        f"{self.finished} trials in {mins:.1f} min")

def _run_trial(self, n, seed):

```

```

c = self.cfg
cmd = [
    "node", str(HERE / "trial.js"),
    "--html", c["html"], "--N", str(n), "--seed", str(seed),
    "--budget", str(c["budget"]), "--wall", str(c["wall"]),
    "--fanin", c["fanin"], "--mutscale", c["mutscale"],
    "--fixedfan", str(c["fixedfan"]), "--progress", "3",
]
try:
    p = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
        text=True, bufsize=1)
except Exception as exc:
    # node missing mid-run
    self.say(f"N={n} seed={seed} failed to launch: {exc}")
    with self.lock:
        self.running.pop((n, seed), None)
        self.finished += 1
    return

with self.lock:
    self.procs[(n, seed)] = p

def pump_stderr():
    for line in p.stderr:
        line = line.strip()
        if not line:
            continue
        try:
            d = json.loads(line)
            if d.get("progress"):
                with self.lock:
                    if (n, seed) in self.running:
                        self.running[(n, seed)].update(d)
                continue
        except Exception:
            pass
        self.say(f"N={n} seed={seed}: {line[:200]}")

t = threading.Thread(target=pump_stderr, daemon=True)
t.start()
out = p.stdout.read()
p.wait()
t.join(timeout=2)

with self.lock:
    self.procs.pop((n, seed), None)
    self.running.pop((n, seed), None)
    self.finished += 1

if p.returncode == 0 and out.strip():
    with self.lock:
        with open(RESULTS, "a") as fh:
            # one line, O_APPEND
            fh.write(out.strip() + "\n")
    try:
        r = json.loads(out)
        mark = r["cross"].get("perfect") or r["final"]
        self.say(f"N={r['N']} seed={r['seed']} "
            f"{'SOLVED' if r['solved'] else 'censored'} "
            f"surv={r['bestSurv'] * 100:.0f}% robust={r['bestRobust']:.2f} "
            f"gen={mark['gen']} nu={mark['neuronUpdates'] / 1e9:.2f}e9 "
            f"{mark['wallSec']:.0f}s")
    except Exception:
        self.say(f"N={n} seed={seed} produced unparseable output")
elif self.stop_flag:
    self.say(f"N={n} seed={seed} terminated")
else:
    self.say(f"N={n} seed={seed} FAILED rc={p.returncode}")

SWEEP = Sweep()

# ---- statistics -----
def half_solved(values):
    """Smallest compute at which at least half the seeds have crossed.

    Identifiable under right-censoring: censored seeds enter as infinity and the
    order statistic is still exact provided more than half of them crossed.
    """
    s = sorted(values)
    return s[math.ceil(len(s) / 2) - 1] if s else float("inf")

def bootstrap(values, b=4000):
    out, censored = [], 0
    for _ in range(b):

```

```

        sample = [random.choice(values) for _ in values]
        h = half_solved(sample)
        if math.isinf(h):
            censored += 1
        else:
            out.append(h)
    out.sort()
    if not out:
        return {"lo": None, "hi": None, "censoredFrac": 1.0}
    return {"lo": out[int(len(out) * 0.05)],
            "hi": out[int(len(out) * 0.95)],
            "censoredFrac": censored / b}

def load_results():
    rows = []
    if RESULTS.exists():
        for line in RESULTS.read_text().splitlines():
            if line.strip():
                try:
                    rows.append(json.loads(line))
                except Exception:
                    pass
    return rows

STRESS_MULS = ["1", "1.5", "2", "2.5", "3", "4"]

def median(vals):
    s = sorted(vals)
    return s[len(s) // 2] if s else None

def best_champion(n):
    """Highest-scoring champion for an arm, cheapest solve breaking ties."""
    rows = [r for r in load_results() if r["N"] == n and r.get("champion")]
    if not rows:
        return None
    rows.sort(key=lambda r: (-r["champion"]["validation"]["robust"], r["final"]["neuronUpdates"]))
    return rows[0]

def lab_payload(row):
    """A v12 payload the lab itself can import, so the pendulum and connectome are
    rendered by the lab's own tested viewer rather than a second implementation."""
    c = row["champion"]
    return {
        "v": 12, "kind": "champion-evolution", "generation": c["generation"], "serial": c["id"],
        "cfg": {"seed": row["seed"], "N": row["N"], "laneDeg": 77, "velLane": 10,
                "torque": 22, "maxT": 10, "survS": 9, "bins": 5, "valTrials": 48},
        "axes": {"effort": [0.15, 0.55], "activity": [0.35, 0.80], "topShare": [0.19, 0.48]},
        "champions": [{"id": c["id"], "generation": c["generation"],
                       "validation": c["validation"], "brain": c["brain"]}],
        "archive": [],
    }

def analysis(crit="perfect"):
    rows = load_results()
    groups = defaultdict(list)
    for r in rows:
        groups[(r["N"], r["fanin"], r["mutscale"])].append(r)

    table = []
    for (n, fanin, mutscale), rs in sorted(groups.items()):
        vals = {u: [(r["cross"][crit][u] if r["cross"].get(crit) else float("inf"))
                    for r in rs] for u in UNITS}
        ci = bootstrap(vals["neuronUpdates"])
        holds = [r["holdoutAtPerfect"]["survival"] for r in rs if r.get("holdoutAtPerfect")]
        total_nu = sum(r["final"]["neuronUpdates"] for r in rs)
        total_cpu = sum(r["final"]["wallSec"] for r in rs)
        voters = sum((sum(r["fanInActual"]) / 3) if r.get("fanInActual") else 0 for r in rs) / len(rs)
        st = [r["stress"] for r in rs if r.get("stress")]
        stress = {m: median([s[m]["survival"] for s in st if m in s]) for m in STRESS_MULS} if st else {}
        breaks = [r["breakMul"] for r in rs if r.get("breakMul")]
        ladder = {}
        for rung in LADDER:
            v = [(r["cross"][rung]["neuronUpdates"] if r["cross"].get(rung) else float("inf"))
                 for r in rs]
            h = half_solved(v)
            ladder[rung] = None if math.isinf(h) else h
        table.append({
            "N": n, "fanin": fanin, "mutscale": mutscale, "seeds": len(rs),
            "solveRate": sum(1 for r in rs if r["cross"].get(crit)) / len(rs),

```

```

        "half": {u: (None if math.isinf(half_solved(vals[u])) else half_solved(vals[u]))
                  for u in UNITS},
        "ci": ci, "ladder": ladder,
        "holdout": (sum(holds) / len(holds)) if holds else None,
        "cells": sum(r["archiveCells"] for r in rs) / len(rs),
        "decDensity": rs[0]["decDensity"], "mutations": rs[0]["mutations"],
        "totalNU": total_nu, "totalCpuSec": total_cpu, "voters": voters,
        "stress": stress, "breakMul": median(breaks), "hasChampion": bool(best_champion(n)),
    })
    finite = [t for t in table if t["half"]["neuronUpdates"]]
    best = min(finite, key=lambda t: t["half"]["neuronUpdates"])[0] if finite else None
    return {"crit": crit, "ladder": LADDER, "stressMuls": STRESS_MULS, "table": table, "best": best,
            "trials": len(rows),
            "totalNU": sum(t["totalNU"] for t in table),
            "totalCpuSec": sum(t["totalCpuSec"] for t in table),
            "totalTrials": sum(t["seeds"] for t in table)}

# ---- api -----
@app.get("/")
def index():
    return send_from_directory(app.static_folder, "index.html")

@app.get("/api/defaults")
def defaults():
    return jsonify({
        "html": str(DEFAULT_HTML), "Ns": [64, 128, 256], "seeds": 30, "seed0": 1000,
        "budget": 1e10, "wall": 3600, "workers": max(1, (os.cpu_count() or 2) - 1),
        "fanin": "fixed", "fixedfan": 15, "mutscale": "on", "resume": True,
        "cpus": os.cpu_count() or 2, "node": bool(shutil.which("node")),
        "resultsExist": RESULTS.exists(),
        "htmlFound": DEFAULT_HTML.exists(),
    })

@app.post("/api/start")
def api_start():
    b = request.get_json(force=True) or {}
    try:
        cfg = {
            "html": str(b.get("html") or DEFAULT_HTML),
            "Ns": [int(x) for x in b.get("Ns", [64, 128, 256]) if int(x) >= 8],
            "seeds": max(1, int(b.get("seeds", 30))),
            "seed0": int(b.get("seed0", 1000)),
            "budget": float(b.get("budget", 1e10)),
            "wall": float(b.get("wall", 3600)),
            "workers": max(1, int(b.get("workers", 4))),
            "fanin": "fixed" if b.get("fanin", "fixed") == "fixed" else "fraction",
            "fixedfan": max(1, int(b.get("fixedfan", 15))),
            "mutscale": "on" if b.get("mutscale", "on") == "on" else "off",
            "resume": bool(b.get("resume", True)),
        }
    except (TypeError, ValueError) as exc:
        return jsonify({"ok": False, "error": f"bad config: {exc}"}, 400)
    if not cfg["Ns"]:
        return jsonify({"ok": False, "error": "give at least one neuron count of 8 or more"}, 400)
    ok, msg = SWEEP.start(cfg)
    return jsonify({"ok": ok, "msg": msg if ok else "error": msg}), (200 if ok else 409)

@app.post("/api/stop")
def api_stop():
    return jsonify({"ok": SWEEP.stop()})

@app.get("/api/status")
def api_status():
    with SWEEP.lock:
        elapsed = (time.time() - SWEEP.started_at) if SWEEP.started_at else 0
        done, total = SWEEP.finished, SWEEP.total
        eta = (elapsed / done) * (total - done) if done and total > done else None
        return jsonify({
            "state": SWEEP.state, "total": total, "finished": done,
            "skipped": SWEEP.skipped, "queued": len(SWEEP.queue),
            "running": list(SWEEP.running.values()),
            "elapsed": elapsed, "eta": eta,
            "log": SWEEP.log[-120:], "cfg": SWEEP.cfg,
        })

@app.get("/api/analysis")
def api_analysis():
    crit = request.args.get("crit", "perfect")
    if crit not in LADDER:
        return jsonify({"error": "unknown criterion"}, 400)
    return jsonify(analysis(crit))

```

```

@app.get("/api/champion")
def api_champion():
    try:
        n = int(request.args.get("N", 0))
    except ValueError:
        return jsonify({"error": "N must be an integer"}), 400
    row = best_champion(n)
    if not row:
        return jsonify({"error": f"no saved champion for N={n}"}), 404
    return jsonify({"N": n, "seed": row["seed"], "solved": row["champion"].get("solvedTrial"),
                  "stress": row.get("stress"), "breakMul": row.get("breakMul"),
                  "payload": lab_payload(row)})

@app.get("/api/lab")
def api_lab():
    """Serve the lab with this arm's champion injected, so one click shows the pendulum
    balancing and the connectome firing. Falls back to a filled paste box if the lab's
    internals ever move."""
    try:
        n = int(request.args.get("N", 0))
    except ValueError:
        return "N must be an integer", 400
    row = best_champion(n)
    if not row:
        return f"No saved champion for N={n}. Run a sweep with this build first.", 404
    if not DEFAULT_HTML.exists():
        return f"lab HTML not found at {DEFAULT_HTML}", 404
    html = DEFAULT_HTML.read_text()
    inject = ""
<script>
setTimeout(function () {
    var P = __PAYLOAD__;
    try {
        document.getElementById('ioBox').value = JSON.stringify(P);
        importBox();
        loadTopChampion();
        mode = 'train'; paused = true; running = false;
        testCurrent();
        setStatus('Champion N=__N__ loaded from the sweep. Frozen test running \u2014 '
            + 'perturbations follow the validation protocol. Press Perturb now to kick it harder. ');
    } catch (e) {
        var b = document.getElementById('ioBox');
        if (b) b.value = JSON.stringify(P);
        alert('Could not auto-load (' + e.message + '). The payload is in the Save and load box \u2014 '
            + 'press Load from box, then Load top champion, then Test paused current. ');
    }
}, 250);
</script>
"""
    inject = inject.replace("__PAYLOAD__", json.dumps(lab_payload(row))).replace("__N__", str(n))
    return html.replace("</body>", inject + "</body>"), 200, {"Content-Type": "text/html; charset=utf-8"}

@app.post("/api/clear")
def api_clear():
    if SWEEP.state == "running":
        return jsonify({"ok": False, "error": "stop the sweep first"}), 400
    if RESULTS.exists():
        RESULTS.rename(RESULTS.with_suffix(f".{int(time.time())}.bak"))
    return jsonify({"ok": True})

if __name__ == "__main__":
    PORT = int(os.environ.get("PORT", 5001))
    print(f"lab html : {DEFAULT_HTML} {'found' if DEFAULT_HTML.exists() else 'NOT FOUND - set it in the HTML field'}")
    print(f"node : {'found' if shutil.which('node') else 'NOT FOUND - required'}")
    print(f"results : {RESULTS}")
    print(f"dashboard : http://127.0.0.1:{PORT}")
    app.run(host="127.0.0.1", port=PORT, threaded=True, debug=False)

```

J.11 sweep/test_gui.js — headless DOM test for the dashboard.

```

// Runs the dashboard's JS against a stubbed DOM with canned API responses, and fails if
// the render path touches an element the HTML does not define. A missing #spend div once
// threw mid-render and left the page showing "no results yet" with a full results file;
// this test exists so that cannot ship again.
//
// node test_gui.js
const fs = require('fs');
const path = require('path');

const html = fs.readFileSync(path.join(__dirname, 'static', 'index.html'), 'utf8');

```

```

const js = html.match(</script>([\s\S]*)</script>)[1];
const declared = new Set([...html.matchAll(/id="([A-Za-z0-9_]+)"/g)].map(m => m[1]));

// A browser gives a <select> its first option's value and an <input> its value attribute.
// Without that the unit formatter sees an empty string and renders raw numbers.
const initial = {};
for (const m of html.matchAll(<select id="([A-Za-z0-9_]+)"[^>]*\s*<option value="(["]*)"/>)) initial[m[1]] = m[2];
for (const m of html.matchAll(<input id="([A-Za-z0-9_]+)"[^>]*value="(["]*)"/>)) initial[m[1]] = m[2];

const els = {};
const missing = new Set();
global.document = {
  getElementById(id) {
    if (!declared.has(id)) { missing.add(id); return null; } // browser behaviour
    return els[id] || (els[id] = {
      id, value: initial[id] ?? '', contentType: '', innerHTML: '', checked: true,
      style: {}, disabled: false, scrollTop: 0, scrollHeight: 0,
    });
  },
};

const ANALYSIS = {
  crit: 'perfect',
  ladder: ['rob2', 'rob4', 'rob6', 'rob9', 'rob11', 'perfect', 'surv25', 'surv50', 'surv90', 'surv100'],
  best: 16, trials: 10, totalTrials: 10, totalNU: 7.7e9, totalCpuSec: 1260,
  table: [
    { N: 16, fanin: 'fixed', mutscale: 'on', seeds: 5, solveRate: 1.0,
      half: { gen: 300, episodes: 40000, neuronUpdates: 0.36e9, wallSec: 58 },
      ci: { lo: 0.3e9, hi: 0.45e9, censoredFrac: 0.0 },
      ladder: { rob2: 5e7, rob4: 8e7, rob6: 1e8, rob9: 1.2e8, rob11: 2e8, perfect: 3.6e8,
        surv25: 1e8, surv50: 1.2e8, surv90: 2e8, surv100: 3.6e8 },
      holdout: 0.97, cells: 19.2, decDensity: 94, mutations: 2,
      totalNU: 1.69e9, totalCpuSec: 280, voters: 14.7 },
    { N: 64, fanin: 'fixed', mutscale: 'on', seeds: 5, solveRate: 0.4,
      half: { gen: null, episodes: null, neuronUpdates: null, wallSec: null },
      ci: { lo: null, hi: null, censoredFrac: 0.62 },
      ladder: { rob2: 8e7, rob4: 1.4e8, rob6: 1.7e8, rob9: 2e8, rob11: 3.3e8, perfect: null,
        surv25: 1.7e8, surv50: 2e8, surv90: 3.3e8, surv100: null },
      holdout: null, cells: 19.7, decDensity: 23, mutations: 2,
      totalNU: 3.88e9, totalCpuSec: 620, voters: 14.7 },
  ],
};

const STATUS = {
  state: 'running', total: 10, finished: 4, skipped: 0, queued: 3,
  running: [{ N: 16, seed: 1000, gen: 120, neuronUpdates: 1.4e8, wallSec: 23, bestRobust: 8.2, bestSurv: 0.4 }],
  elapsed: 240, eta: 360, log: [{ t: '12:00:00', msg: 'N=16 seed=1000 SOLVED' }], cfg: {},
};

const DEFAULTS = { html: '/x/protoneuron-champion-lab-v12.html', Ns: [16, 32], seeds: 13,
  seed0: 1000, budget: 3e9, wall: 1200, workers: 7, fanin: 'fixed', fixedfan: 15,
  mutscale: 'on', resume: true, cpus: 8, node: true, resultsExist: true, htmlFound: true };

const calls = [];
global.fetch = async (url) => {
  calls.push(url);
  const body = url.includes('defaults') ? DEFAULTS
    : url.includes('status') ? STATUS
    : url.includes('analysis') ? ANALYSIS
    : { ok: true };
  return { json: async () => body };
};
global.setInterval = () => 0;
global.console.warn = (m) => { missing.add(String(m).replace('missing #', '')); };

eval(js);

(async () => {
  await new Promise(r => setTimeout(r, 120));

  const fails = [];
  if (missing.size) fails.push('render touched elements not in the HTML: ' + [...missing].join(', '));

  const tbl = els.tbl ? els.tbl.innerHTML : '';
  if (!tbl.includes('N=16')) fails.push('results table did not render a row for N=16');
  if (!tbl.includes('0.36e9')) fails.push('half-solved value missing from the table');
  if (!tbl.includes('1.69e9')) fails.push('total spent column missing from the table');
  if (!tbl.includes('not reached')) fails.push('censored group not shown as not reached');

  const spend = els.spend ? els.spend.innerHTML : '';
  if (!spend.includes('cpu-hours')) fails.push('sweep total line did not render');

  const ladder = els.ladderbody ? els.ladderbody.innerHTML : '';
  if (!ladder.includes('N=16')) fails.push('ladder table did not render');

  const chart = els.chart ? els.chart.innerHTML : '';

```

```

if (!chart.includes('N=16')) fails.push('chart did not render');
if (!chart.includes('not reached')) fails.push('chart did not mark the censored group');

const live = els.live ? els.live.innerHTML : '';
if (!live.includes('seed') && !live.includes('1000')) fails.push('in-flight table did not render');
const msg = els.msg ? els.msg.innerHTML + els.msg.textContent : '';
if (msg.includes('render failed')) fails.push('render threw: ' + msg);

if (fails.length) {
  console.error('FAIL');
  for (const f of fails) console.error(' - ' + f);
  process.exit(1);
}
console.log('PASS results table, spend line, ladder, chart and in-flight table all rendered');
console.log(' ' + calls.length + ' API calls, no missing elements');
})();

```

J.12 quad/test_viewer.js — headless DOM test for the quadruped viewer.

```

// Runs the quadruped viewer.html against a stubbed DOM and canvas, so a missing element
// or a broken render path fails here instead of showing a blank page.
//
// node test_viewer.js
const fs = require('fs');
const path = require('path');

const html = fs.readFileSync(path.join(__dirname, 'viewer.html'), 'utf8');
const js = html.match(/<script>([\s\S]*)</script>/)[1];
const declared = new Set([...html.matchAll(/id="([A-Za-z0-9_+)]"/g)].map(m => m[1]));
const initial = {};
for (const m of html.matchAll(/<select id="([A-Za-z0-9_+)]"[\^>]*>\s*<option value="([^"]*)" /g)) initial[m[1]] = m[2];
for (const m of html.matchAll(/<input id="([A-Za-z0-9_+)]"[\^>]*value="([^"]*)" /g)) initial[m[1]] = m[2];

const missing = new Set();
const els = {};
const ops = { fill: 0, stroke: 0, text: 0 };
function ctx() {
  return new Proxy({}, {
    get(t, k) {
      if (k === 'createRadialGradient') return () => ({ addColorStop() {} });
      if (k === 'fill' || k === 'fillText') return () => { ops[k === 'fill' ? 'fill' : 'text']++; };
      if (k === 'stroke') return () => { ops.stroke++; };
      if (k === 'canvas') return { width: 900, height: 470 };
      if (typeof k === 'string' && /^(globalAlpha|linewidth|fillStyle|strokeStyle|font)$/.test(k)) return '';
      return () => {};
    },
    set() { return true; },
  });
}

global.document = {
  getElementById(id) {
    if (!declared.has(id)) { missing.add(id); return null; }
    if (!els[id]) {
      const sizes = { view: [900, 430], gait: [900, 130], net: [420, 250] };
      els[id] = { id, value: initial[id] ?? '', textContent: '', innerHTML: '', style: {}, disabled: false,
        scrollTop: 0, scrollHeight: 0, width: (sizes[id] || [0, 0])[0], height: (sizes[id] || [0, 0])[1],
        getContext: () => ctx(), onclick: null, onchange: null };
    }
    return els[id];
  },
  createElement: () => ({ style: {}, click() {}, remove() {}, setAttribute() {} }),
  body: { appendChild() {}, removeChild() {} },
};

global.window = {};
global.requestAnimationFrame = () => 0;
global.setTimeout = (fn) => { void fn; return 0; }; // never auto-loop the trainer
global.clearTimeout = () => {};
global.Blob = class { constructor(a) { this.a = a; } };
global.URL = { createObjectURL: () => 'blob:x', revokeObjectURL() {} };
global.console.warn = (m) => { missing.add(String(m).replace('missing #', '')); };

// the viewer expects walker3d.js to have populated window.WALKER
require('./quad4.js');
global.window.QUAD = require('./quad4.js');

eval(js);

// the registry is lazy, so fetch through the accessor to force creation
const el = id => global.document.getElementById(id);

const fails = [];
if (missing.size) fails.push('touched elements not in the HTML: ' + [...missing].join(', '));
if (!els.stages || !el('stages').innerHTML.includes('walk')) fails.push('curriculum panel did not render');

```

```

if (!els.stage || !el('stage').innerHTML.includes('option')) fails.push('stage selector not populated');

// train a handful of generations through the real handler
el('cases').value = '4'; el('cands').value = '2'; el('neurons').value = '12';
el('reset').onclick();
for (let i = 0; i < 10; i++) el('train').onclick(); // setTimeout is a no-op, so one gen each
if (!els.mGen || +el('mGen').textContent < 1) fails.push('no generations ran: mGen = ' + (els.mGen &&
  el('mGen').textContent));

// watch the reference policy, which needs no training
el('policy').value = 'reference';
el('watch').onclick();
if (typeof updateMetrics === 'function') updateMetrics(); // the frame loop is stubbed out
if (el('mMode').textContent !== 'watch') fails.push('watch did not start, mode = ' + el('mMode').textContent);
const before = { ...ops };
for (let i = 0; i < 40; i++) { /* frame loop body */ }
// drive the frame body directly: stepWatch + draws are inside the IIFE, so re-run pieces
if (typeof stepWatch === 'function') for (let i = 0; i < 60; i++) stepWatch();
if (typeof drawView === 'function') drawView();
if (typeof drawGait === 'function') drawGait();
if (typeof drawNet === 'function') drawNet();
if (ops.fill <= before.fill && ops.stroke <= before.stroke) fails.push('renderer drew nothing');
if (typeof gaitHist !== 'undefined' && gaitHist.length === 0) fails.push('gait diagram recorded no contacts');

// score every rung with each ablation
for (const ab of ['none', 'rec', 'thr']) {
  el('ablate').value = ab;
  el('report').onclick();
  if (!el('rep').innerHTML.includes('trot')) fails.push('stage report empty with ablation ${ab}');
}
// save and reload a champion
el('policy').value = 'evolved';
el('save').onclick();
if (!el('box').value.includes('protoneuron-quad4')) fails.push('save produced no payload');
const saved = el('box').value;
el('load').onclick();
if (!el('io').textContent.includes('Loaded')) fails.push('load rejected its own payload: ' + el('io').textContent);
el('box').value = '{"kind":"nope"}';
el('load').onclick();
if (!el('io').textContent.includes('Could not load')) fails.push('bad payload was not rejected');
el('box').value = saved;

if (fails.length) {
  console.error('FAIL');
  for (const f of fails) console.error(' - ' + f);
  process.exit(1);
}
console.log('PASS curriculum, training, watching, 3D + gait + connectome render, ablations, save/load');
console.log(' canvas ops: ${ops.fill} fills, ${ops.stroke} strokes, ${ops.text} labels');

```